



SPRINT 3 REPORT

COS40006 COMPUTING TECHNOLOGY PROJECT B

Prepared by Group 12:

Kenneth Hui Hong CHUA	102782494
Ivan Jia Wei HOAN	102786580
Kelvin Wen Kiong FONG	102782287
Nigel Zi Jun LING	102779140

Unit code: COS40006/EAT40006

Unit Name: Computing/Engineering Technology Project B

Submission date: 8th May 2026

Table of Contents

Contribution Summary (This Sprint).....	1
Contribution Details (This Sprint)	2
1 Sprint Plan	24
1.1 Sprint goal.....	24
1.2 Scope	24
1.3 Assumption & Dependencies	25
1.3.1 Assumption.....	25
1.3.2 Dependencies	26
1.4 Capacity & Forecast.....	29
1.5 Definition of done.....	29
1.6 Sprint Milestone & Review Artifacts	30
1.7 Sprint Backlog.....	32
2 Final Documentation Deliverables	34
2.1 Introduction.....	34
2.2 System Overview	35
2.2.1 System Architecture	35
2.2.2 Key Features	36
2.2.3 Technologies Used.....	38
2.3 User Manual	40
2.3.1 Installation Guide	41
2.3.2 How to Start the Application	42
2.3.3 Controls Instructions	43
2.3.4 UI Explanation	49
2.3.5 Features Guide	56

2.3.6	Troubleshooting	62
2.4	System Installation / Deployment Manual.....	68
2.4.1	System Requirements.....	68
2.4.2	Project Structure	76
2.4.3	Configuration.....	78
2.5	Design & Implementation Details	82
2.5.1	Architectural Overview.....	82
2.5.2	Database Design	83
2.5.3	Key Technical Decisions.....	87
2.5.4	Core Feature Implementation	90
2.5.5	Security and Permissions.....	97
2.6	Testing	99
2.7	Limitations & Future Improvements	101
2.7.1	Current Limitations.....	101
2.7.2	Future Enhancements	102
2.8	Conclusion	103
3	Sprint Progress	104
3.1	Sprint Deliverables	104
3.1.1	UI Upgrade.....	104
3.1.2	AI-Fine-Tuning	107
3.1.3	AI Chat History.....	108
3.1.4	My Books	109
3.1.5	Notification.....	111
3.1.6	Reading Assistant	112
3.1.7	Learning Hub	113

3.2	Repository and Workflow Evidence	118
3.2.1	GitHub.....	118
3.2.2	WhatsApp Chat.....	119
3.2.3	Microsoft Teams.....	120
3.2.4	Linear	121
4	Sprint Review	123
4.1	Demonstration Session and Client Communication.....	123
4.2	Feedback Received	124
4.3	Critical analysis of progress	126
5	Retrospect (Critical Review of the Process)	129
5.1	What went well	129
5.2	What did not go well	130
5.3	What can be improved	132
5.4	Action items to be done	133
5.5	General observations from Sprint 3	134
5.6	Risk Registry and Ethical Considerations.....	135
6	Lessons Learned	137
6.1	Lessons Learned	137
6.2	To do and future improvement.....	139

Contribution Summary (This Sprint)

	Contribution	Initiatives	Communication					Respect
Team Member	Finished tasks in time or on time with an acceptable quality	Self-assigned tasks and proactively found solutions to problems	Attended all team meetings	Attended all supervisor meetings	Attended all client meetings	Replied every time within an acceptable timeframe	Kept the team updated about the status	The student respected others and listened to different opinions
Kenneth Hui Hong CHUA	Yes, in case of can't make on time will inform the team	Self-researching, find out problem, provide solution to the team	Yes	Yes	Yes	Yes	As leader of team, always updating the status	Yes, listening to team member opinion
Ivan Jia Wei HOAN	If cannot finish will try adjust the task	Will find problem and solution for the project	Yes	Yes	Yes	Reply as soon as possible	Will notice if have any changes	Try to discuss with others when different opinion
Kelvin Wen Kiong FONG	Built the foundation of the system and made the webpage responsive	Helped with tasks when needed and tried my best to solve problems	Yes	Yes	Yes	Reply ASAP	Regularly informed the team about our progress and any changes in the project	Talk with all team members to share feedback and check on any updates or changes
Nigel Zi Jun LING	Follow the requirement and finish task as soon as possible with acceptable quality	Help teams to solve	Yes	Yes	Yes	Reply as soon as possible	Notice every change in physical meetings and WhatsApp group	Communicate with every member to exchange feedback and follow up changes

Contribution Details (This Sprint)

Kenneth Hui Hong CHUA

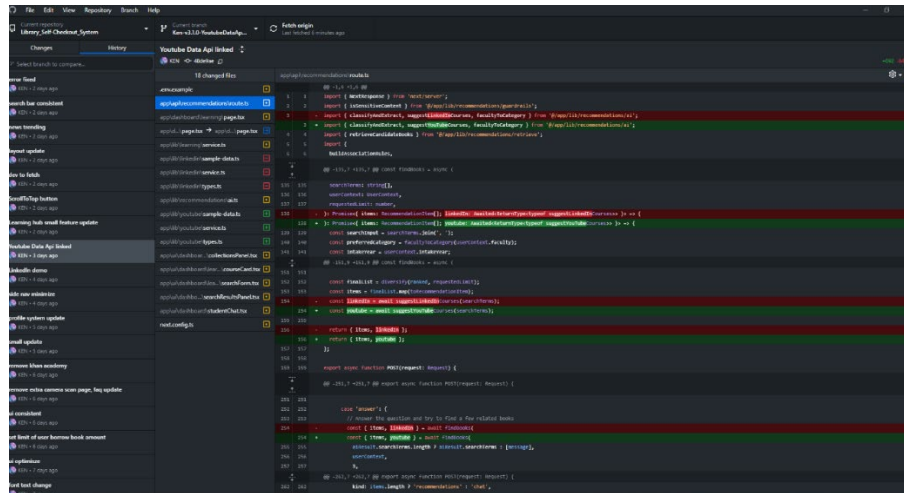


Figure 1 Kenneth's GitHub History

In Sprint 3, I was solely responsible for designing and implementing the Learning Hub, a dedicated self-study page available exclusively to student-role accounts at learning hub. The Learning Hub was conceived as a one-stop browsing destination for online learning content that complements the AI Reading Assistant, instead of asking a question to find a resource, students can browse and discover content passively on their own terms. The page is structured as three tabs, YouTube, Community, and News, each pulling from a different external content source. I integrated all three sources, built the tab navigation, implemented the per-topic server-side caching strategy, designed the responsive card layouts for desktop and mobile viewports, and handled the graceful fallback behaviour for when external API keys are unavailable. I also refined the overall UI of the page to match the warm cream and charcoal design system adopted across the dashboard in this sprint.

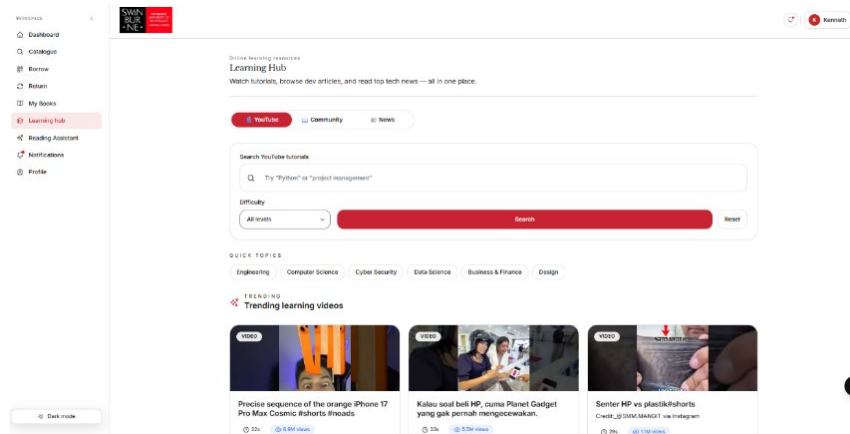


Figure 2 Learning Hub Page

The YouTube tab is the centrepiece of the Learning Hub and was built using the YouTube Data API v3. The integration allows students to search for tutorial videos directly within the library system without leaving the dashboard. When a student enters a search query or selects one of the six preset faculty quick-topic chips (Engineering, Computer Science, Cyber Security, Data Science, Business & Finance, and Design), the system sends a server-side request to the YouTube Data API using the configured API key. Results are filtered to return only educational and relevant content and are rendered as playable video cards inside an in-app modal so students do not need to be redirected to YouTube itself. A “Trending” feed is displayed by default when no search query has been entered, drawing from YouTube’s Science and Technology category (category ID 28) filtered to the MY region. To stay within the free API quota, results are cached per search topic so repeated queries within the same session do not consume additional quota. If the YOUTUBE_API_KEY environment variable is not configured, the tab falls back gracefully to bundled sample video data and displays a clear “Demo mode” amber banner so students and administrators are aware that live data is unavailable.

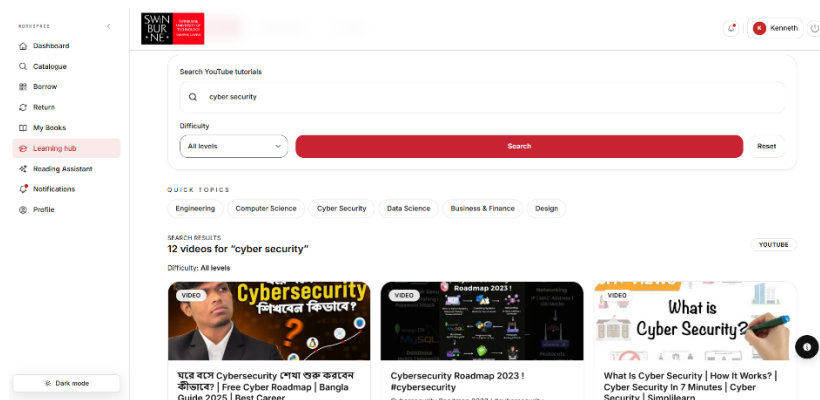


Figure 3 Learning Hub Search result



Figure 4 YouTube playing window

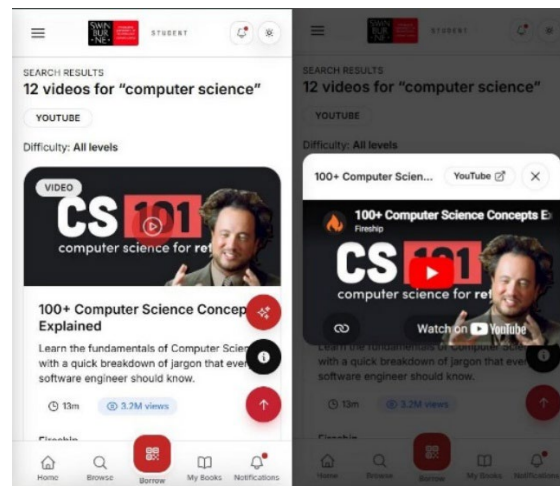


Figure 5 Learning Hub mobile view

The Community tab of the Learning Hub was built by integrating the DEV.to public API, which provides access to thousands of community-written developer and technology articles without requiring authentication or incurring cost. When a student navigates to the Community tab, the system fetches the latest trending articles from DEV.to and filters them by tag category. Students can narrow the feed using emoji-tagged category filter chips covering topics such as Programming, Cyber Security, Cloud, AI/ML, Design, and Career. Each article card displays the title, author, publication date, reading time, tag labels, and a reaction count so students can gauge community interest at a glance. Clicking a card opens a detail panel within the dashboard that shows a summary of the article alongside a link to read the full post on DEV.to. The mobile layout stacks the cards vertically so the Community tab remains fully usable on a phone screen. Article results

are cached for one hour per category to reduce outbound API calls and keep the page load fast even on slower campus Wi-Fi connections.

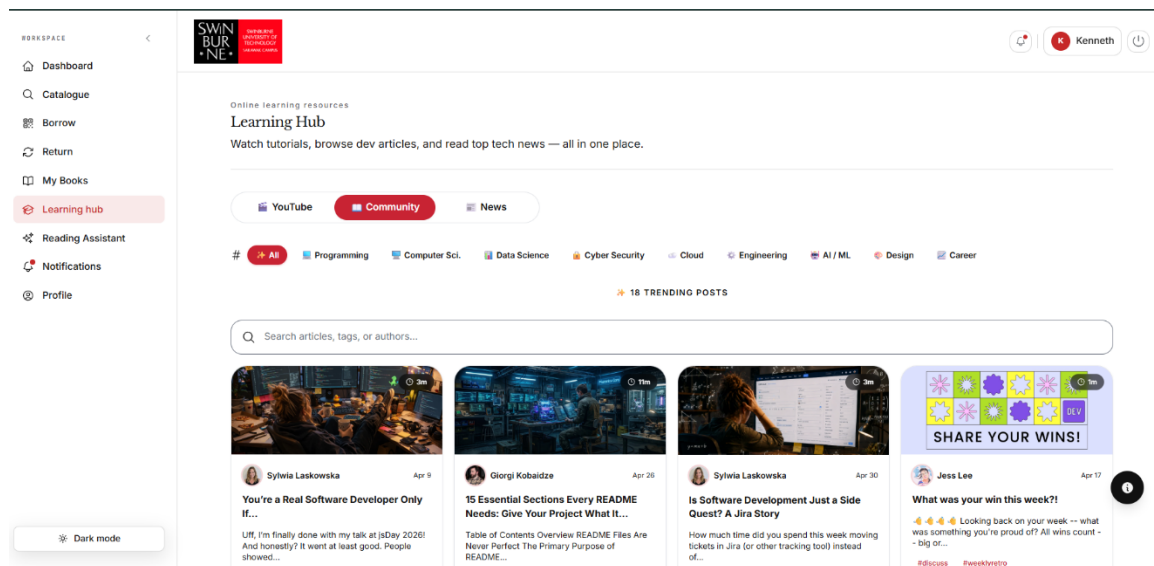


Figure 6 Learning Hub Community

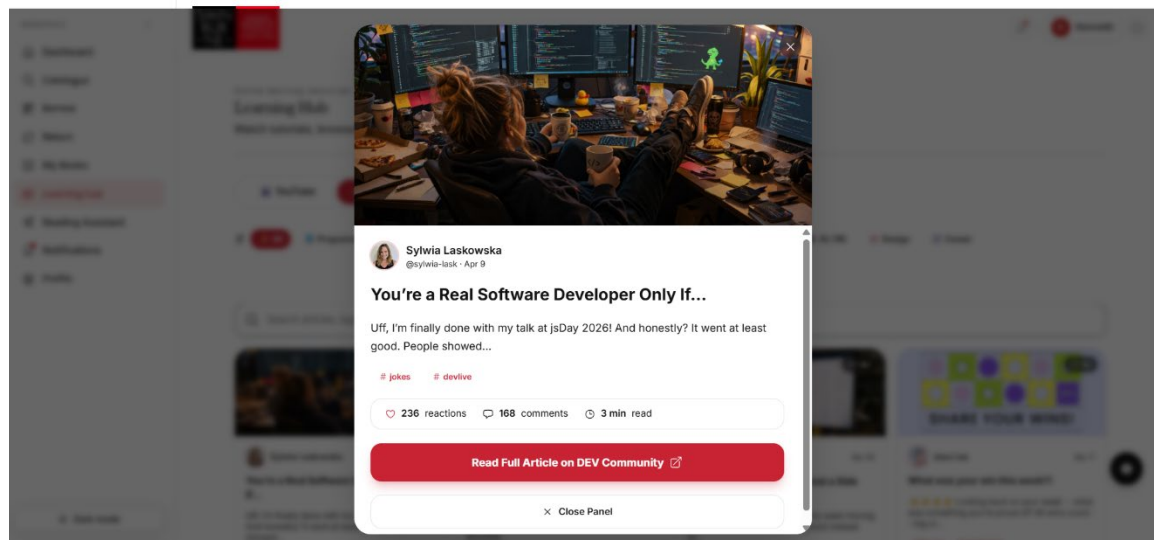


Figure 7 Learning Hub Community details

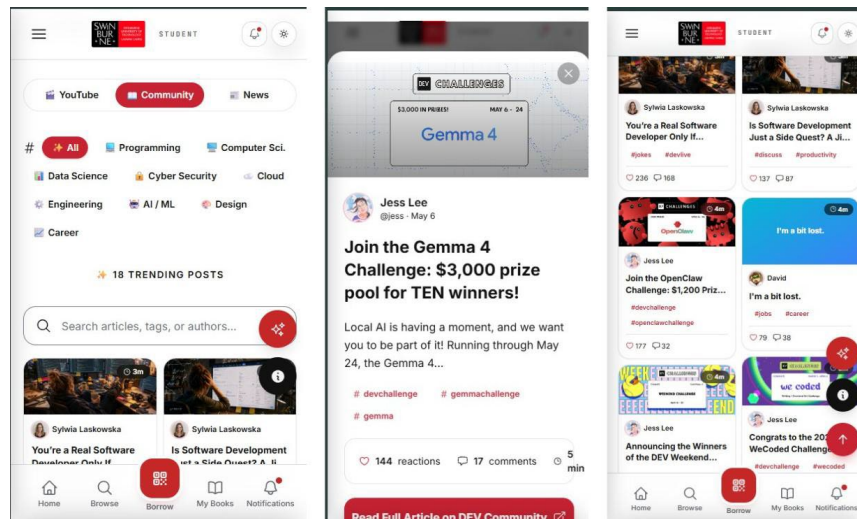


Figure 8 Learning Hub Community Mobile View

The News tab aggregates the latest technology and learning-related headlines from five major RSS feeds: BBC Technology, TechCrunch, The Verge, Ars Technica, and Engadget. Rather than pulling from a single outlet, the system fetches each feed in parallel on the server and interleaves the results so that the displayed news list feels varied and representative of the broader technology landscape rather than any one publication's editorial angle. Each news item shows the headline, source outlet, publication date, and a short excerpt. Clicking a news card opens a detail panel inside the dashboard that surfaces the full article excerpt and provides a direct link to the original article for students who want to read further. The feed is refreshed every 30 minutes via server-side caching, keeping the content current without placing excessive load on the RSS endpoints. The mobile view of the News tab collapses each card into a compact headline-and-source format so the feed remains easy to scroll on smaller screens. The News tab was designed to give students passive awareness of developments in their field, complementing the more active search-driven experience in YouTube and Community tabs.

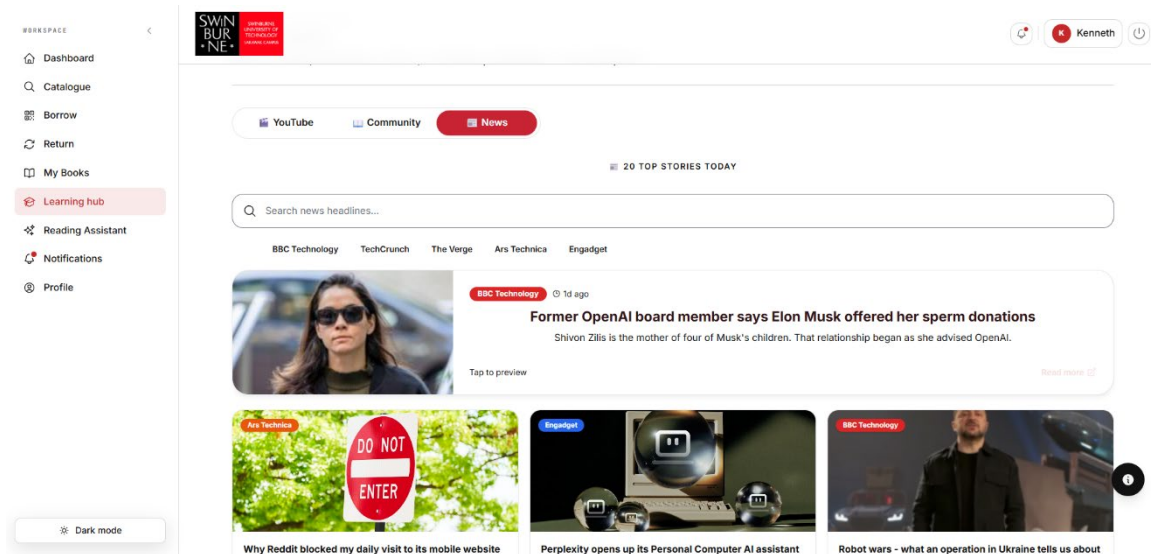


Figure 9 Learning Hub News sections

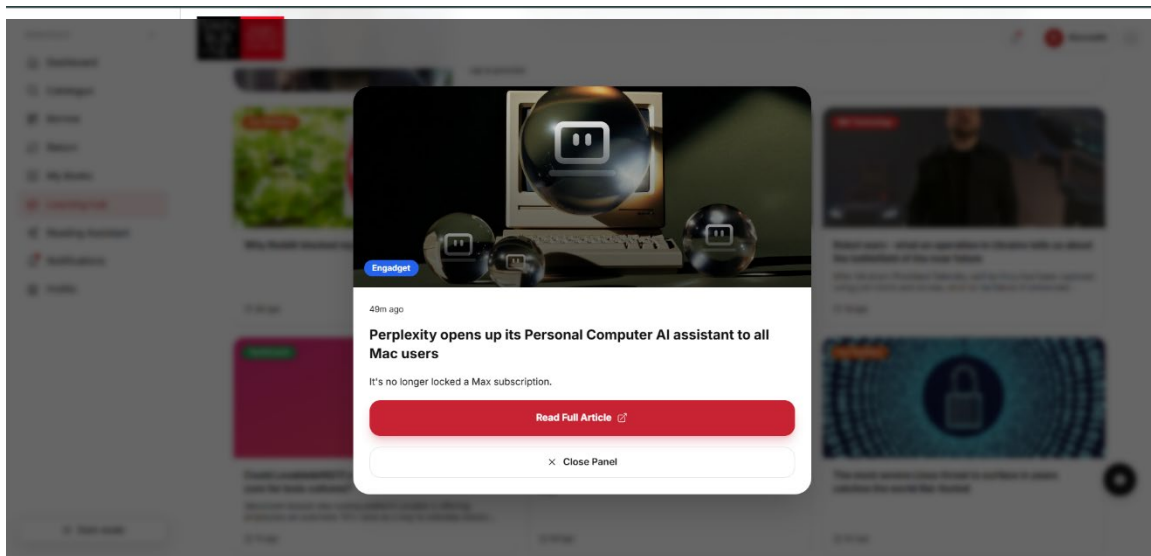


Figure 10 Learning Hub News Details

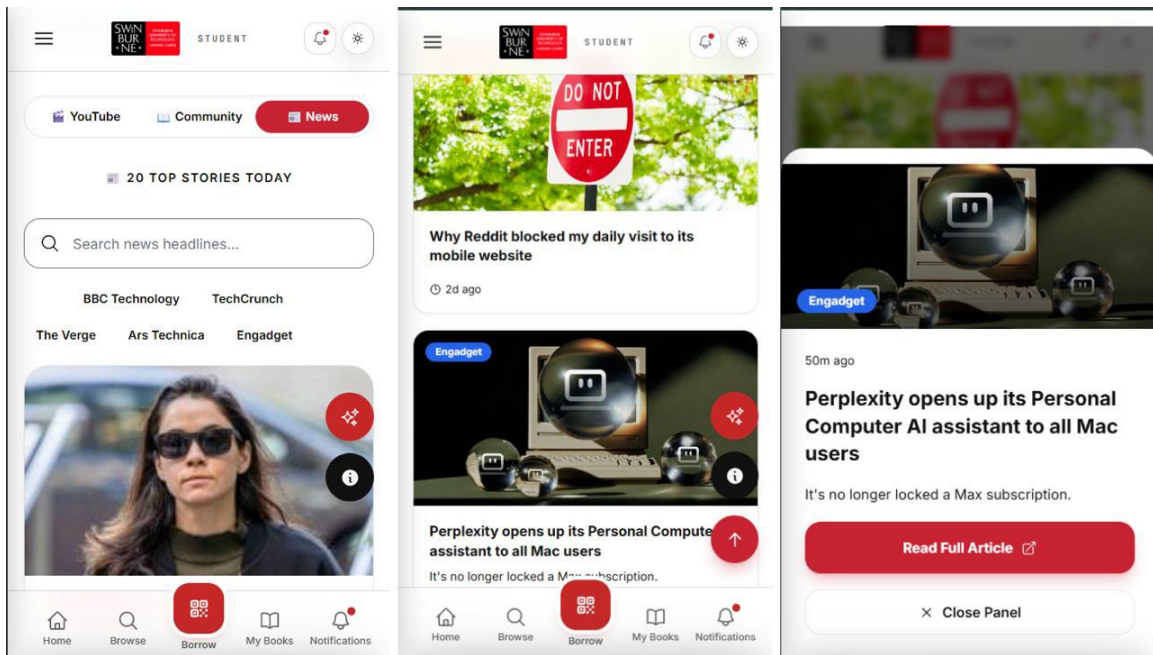


Figure 11 Learning Hub News Mobile View

Alongside the Learning Hub implementation, I contributed to the wider UI refinement effort on the student-facing dashboard. Working within the shared design system established this sprint, warm cream and charcoal palette, rounded corners, and consistent spacing defined in `tailwind.config.ts`. I applied visual polish across the student dashboard pages that fell within my ownership. This included aligning button styles and icon placement to the new design tokens, ensuring interactive elements such as search inputs, category filter chips, and action buttons responded consistently to hover and focus states across both light and dark mode, and verifying that every page I worked on rendered correctly on narrow mobile viewports. I also reviewed the overall flow of the student dashboard from a first-time user's perspective, identifying and correcting instances where the visual hierarchy made key actions less obvious than they should be. The aim throughout was for the interface to feel unified and intentional, so that a student moving between the catalogue, the Learning Hub, and their notifications would not notice any visual inconsistency between sections.

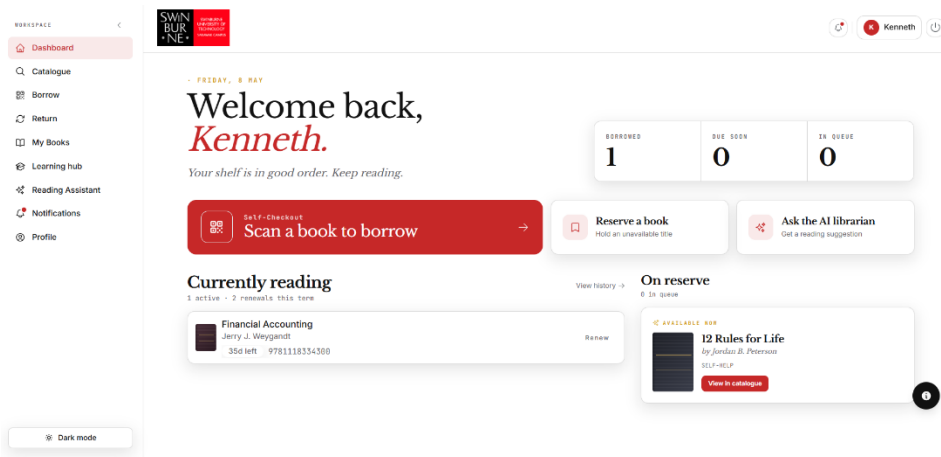


Figure 12 Dashboard UI

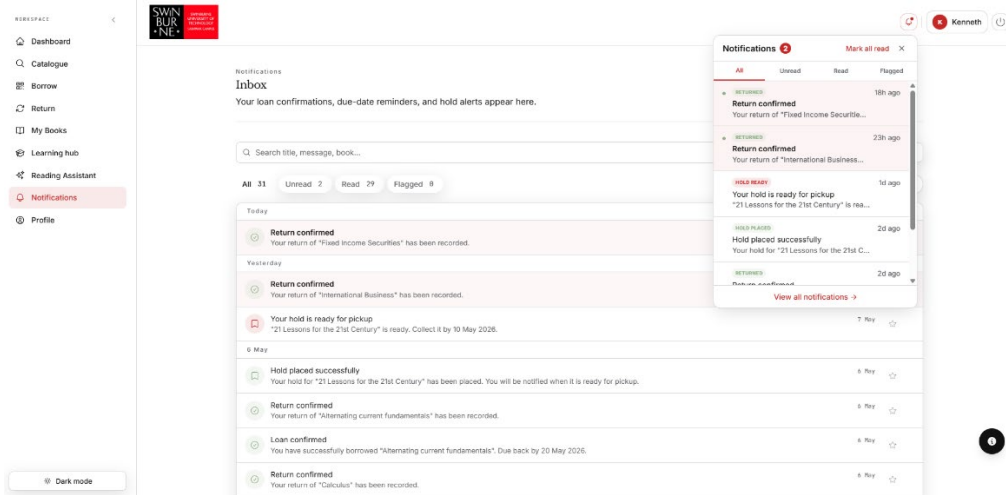


Figure 13: Notification

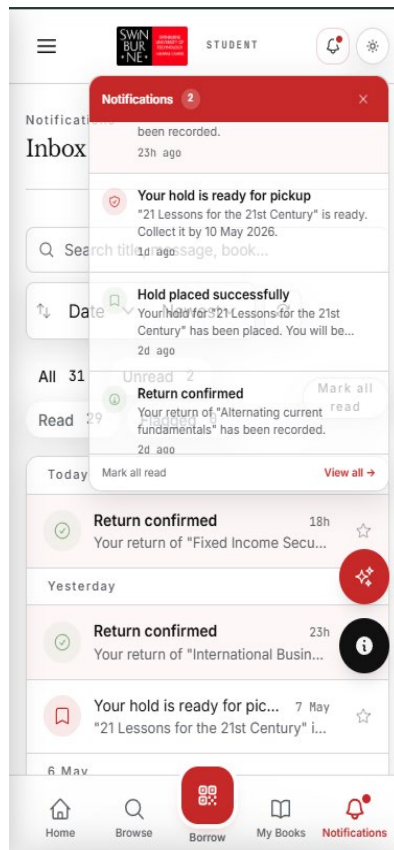


Figure 14: Notification Mobile view

Ivan Jia Wei HOAN

In this sprint, my responsibilities were mainly limited to carrying out smaller tasks like testing, tweaking, and optimization.

Previously, we faced difficulties running our GitHub and Jest tests due to constant updates. However, we have addressed this problem, and now we can confidently execute our GitHub Actions tests without any issues.

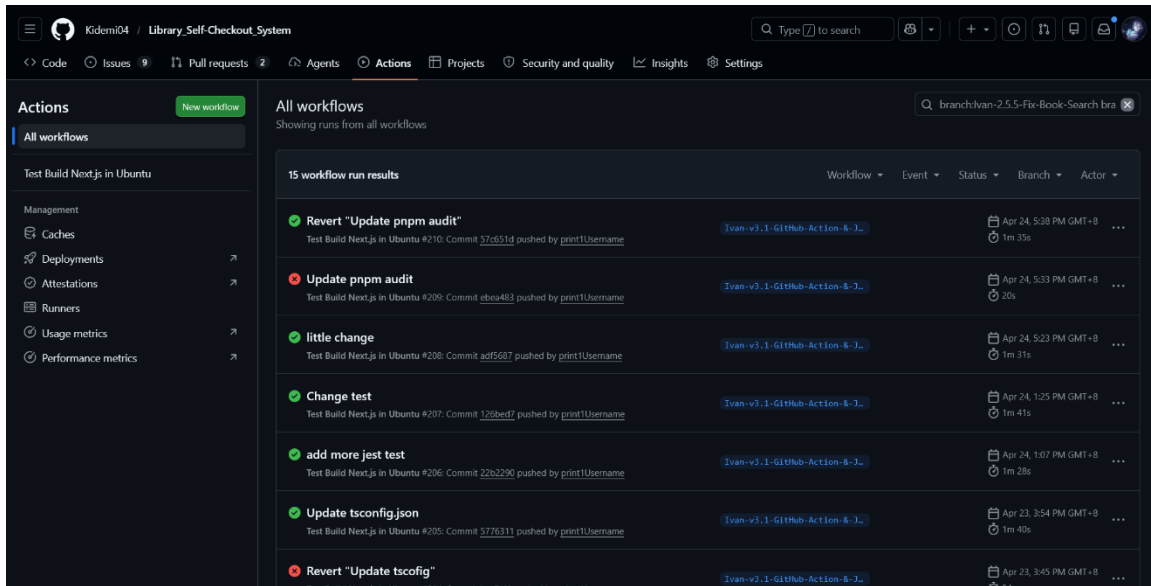


Figure 15 Ivan's GitHub workflows

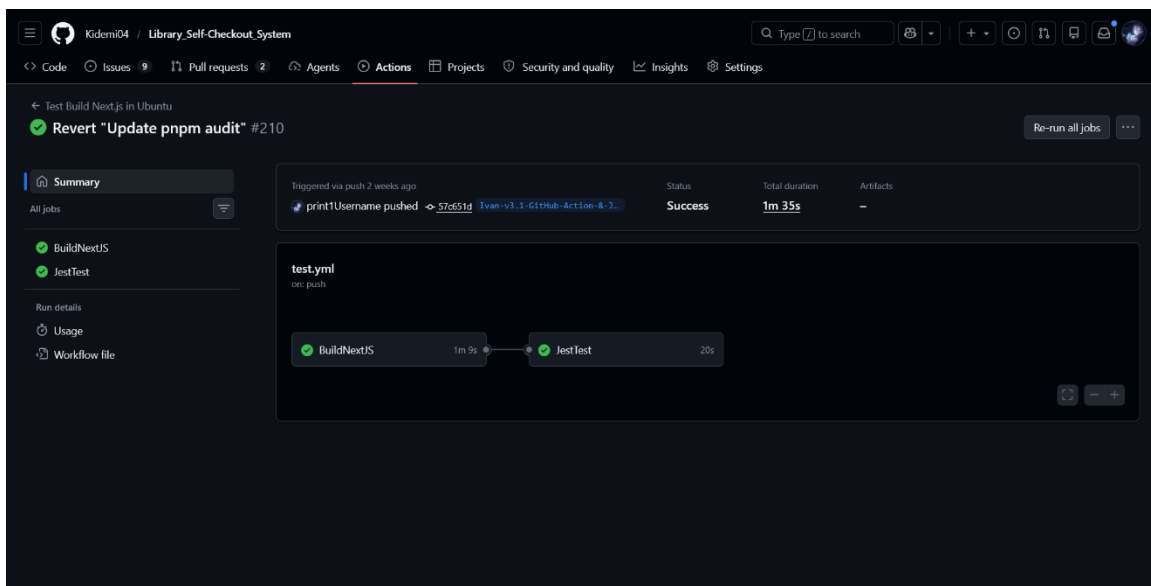


Figure 16 GitHub test build summary

Although we have managed to develop the AI, there are still lots of hidden problems with the recommendations of our AI.

Previously, when developing our project, we managed to create the AI; however, the AI was not able to preserve the chat history. Thus, every time when the user would refresh the website or log out of the website, all of the chat history with the AI would disappear from the system, making the overall experience worse for the user.

To tackle this problem, we decided to add the AI history table into Supabase.

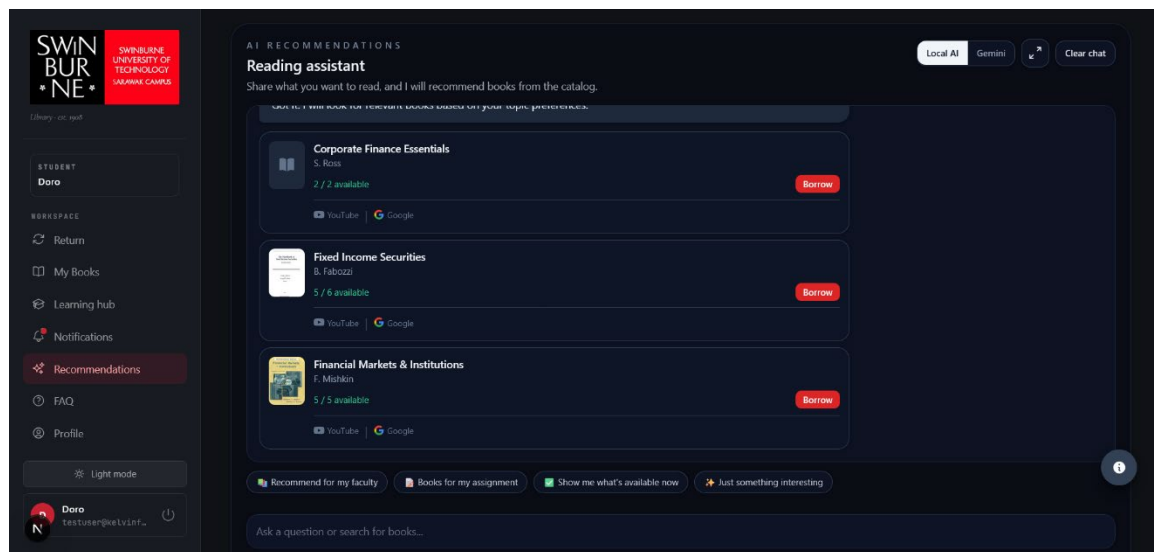


Figure 17 AI chat History

The screenshot shows the Supabase Table Editor interface. On the left is a sidebar with a list of tables: 'ActiveLoans', 'AIChatHistory' (selected), 'AuditLog', 'AvailableCopies', 'BookRecommendation', 'BookTags', 'BookTagsLinks', 'Books', 'Community', 'CommunityBookRec...', 'CommunityMem...', 'CommunityPostCo...', 'CommunityPosts', 'CommunityRecomm...', 'Copies', 'DamageReports', and 'FriendList'. The main area shows the 'AIChatHistory' table with columns: 'id', 'user_id', 'message_id', 'sender', and 'text'. The table contains 100 rows of data. The first few rows are visible, showing chat history entries with their respective IDs and content. The bottom of the interface shows 'Page 1 of 1' and '100 rows 8 records'.

Figure 18 AI Chat History table

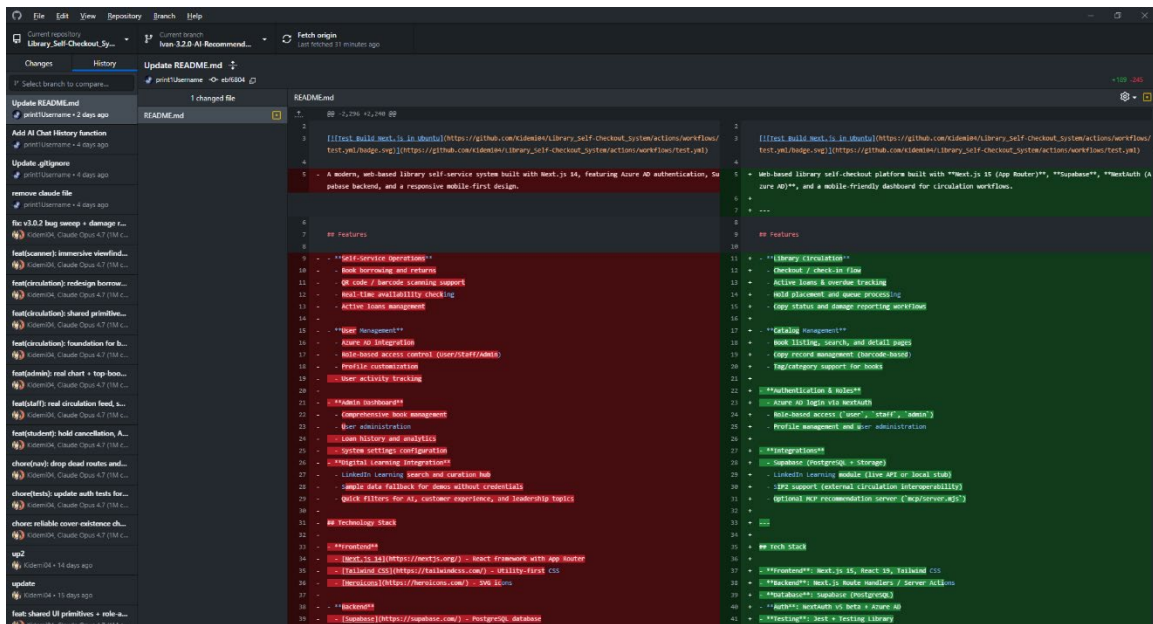


Figure 19 Ivan's GitHub History

Kelvin Wen Kiong FONG

1. UI redesign across the system

I led the full UI redesign across every student, staff, and admin page in the dashboard. The visual direction drew on Anthropic's Claude interface as a design reference, adapted to a library dashboard context. The work included defining the design tokens (warm cream and charcoal palette, self hosted typography, spacing conventions) in tailwind.config.ts, implementing the dark mode toggle and the responsive layouts for narrow viewports, and migrating every page to the new system in three planned batches across the sprint. The redesign was delivered as roughly 17 atomic commits for traceability. (See Figure for a before and after comparison.)

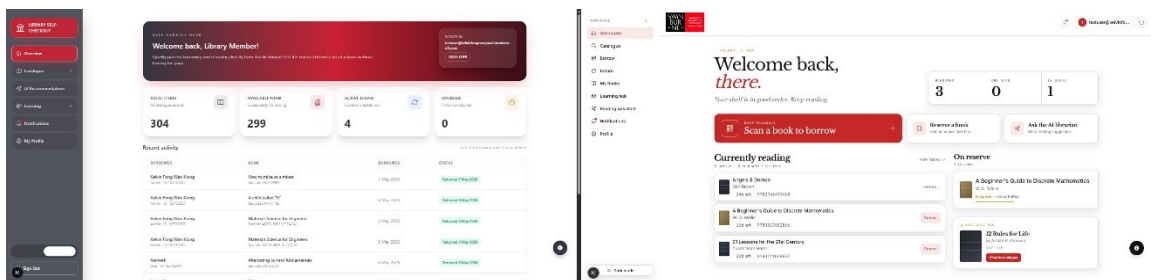


Figure 20: Student Dashboard Before And After

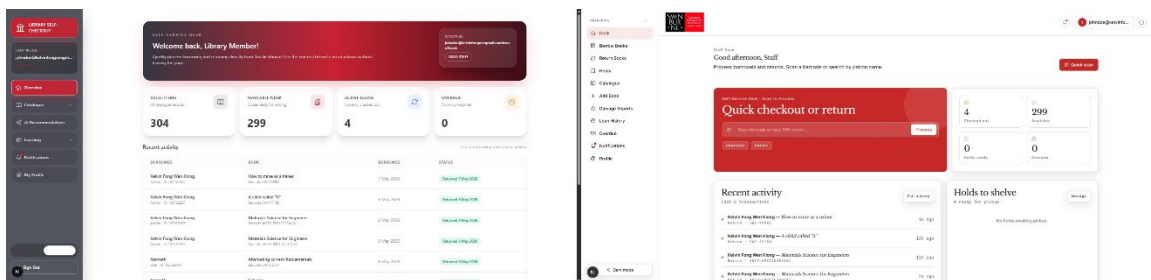


Figure 21: Staff Dashboard Before And After

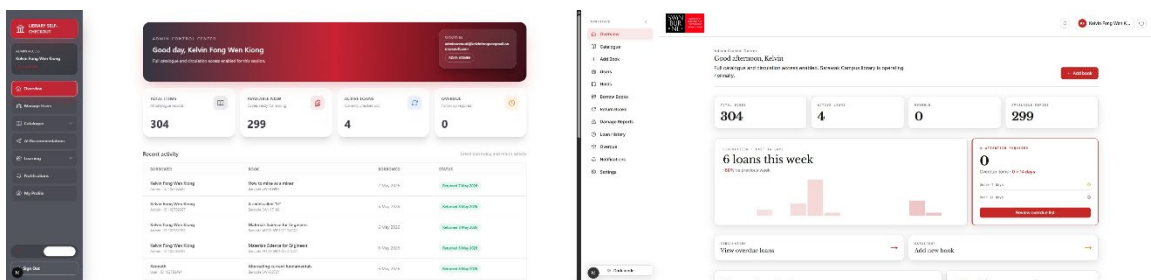


Figure 22: Admin Dashboard Before And After

2. Admin and Staff Operational Pages

I built four operational pages for the admin and staff roles during Sprint 3. The admin Overdue page lists overdue loans with multi facet filters (faculty, department, date range, search). The admin Users page introduces role based tab navigation and an integrated detail panel that lets admins manage user roles directly from the dashboard, removing the need to update the Supabase tables manually. The admin Books catalogue includes an add book form with ISBN lookup that prevents duplicate entries by redirecting to the existing book when a match is found. The staff Loan History page lists every past loan with filter controls and a filtered CSV export. I also restructured every other admin and staff page during the dashboard redesign so that they conform to the new design system. (See Figure 2 for screenshots of the four primary pages.)

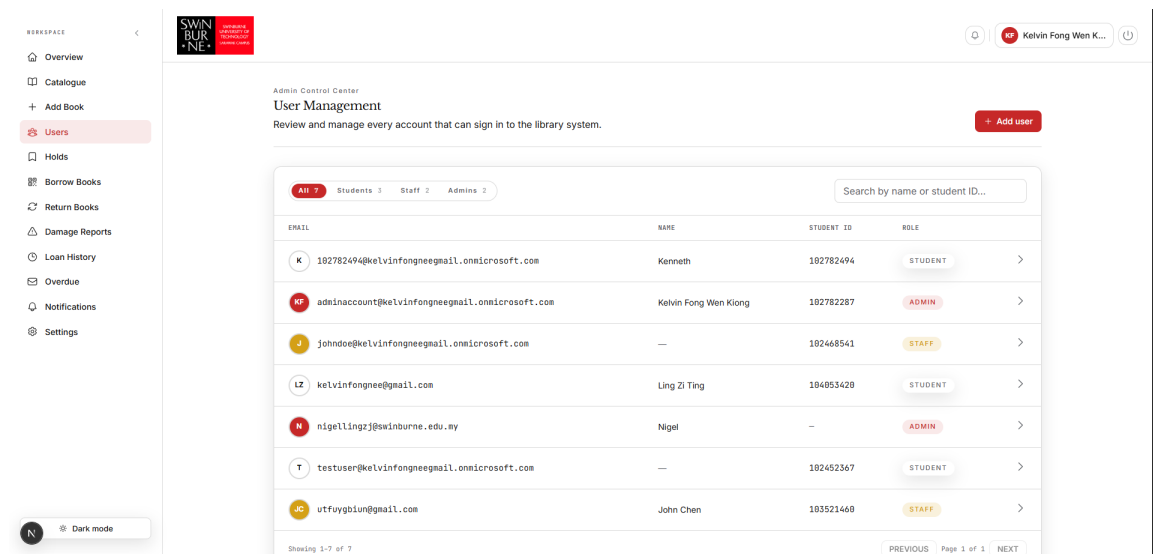


Figure 23: Users page with role tabs and integrated detail panel

WORKSPACE

Overview

Catalogue

+ Add Book

Users

Holds

Borrow Books

Return Books

Damage Reports

Loan History

Overdue

Notifications

Settings

Dark mode

SWINBURNE

Q

Kelvin Fong Wen K...

U

Circulation - Archive

Loan history

All borrowing records. Filter, search, and export.

All

Active

Returned

Overdue

30 days

6 months

Semester

All time

Borrower (name / student ID)

Book (title / author / barcode)

Handler (staff name)

Showing 32 of 32 Loans · 4 active · 28 returned · 0 overdue

Export CSV

Book	Borrower	Borrowed	Due	Returned	Duration	Status
How to mine as a miner People from Niger	Kelvin Fong Wen Kiong 192782287	07/05/2026	14/05/2026	07/05/2026	8d	Returned
A child called "It" David J. Pelzer	Kelvin Fong Wen Kiong 192782287	06/05/2026	20/05/2026	08/05/2026	2d	Returned
Materials Science for Engineers W. Callister	Kelvin Fong Wen Kiong 192782287	06/05/2026	20/05/2026	08/05/2026	2d	Returned

Figure 24: Loan History page with filtered export

WORKSPACE

Overview

Catalogue

+ Add Book

Users

Holds

Borrow Books

Return Books

Damage Reports

Loan History

Overdue

Notifications

Settings

Dark mode

SWINBURNE

Q

Kelvin Fong Wen K...

U

Catalog - New title

Add new book

Create a new title and its physical copies.

ISBN duplicate check *

9780132350884

Lookup

Scan ISBN

This book already exists in the library:

"Clean Code" by Robert C. Martin (1 copy)

Go to that book

Continue creating new entry

Basic information

Title *

Author *

Publisher

Publication year

Classification

Cover image URL

e.g. 005.1 MAR

https://...

Figure 25: Add a new book page with ISBN duplicate check

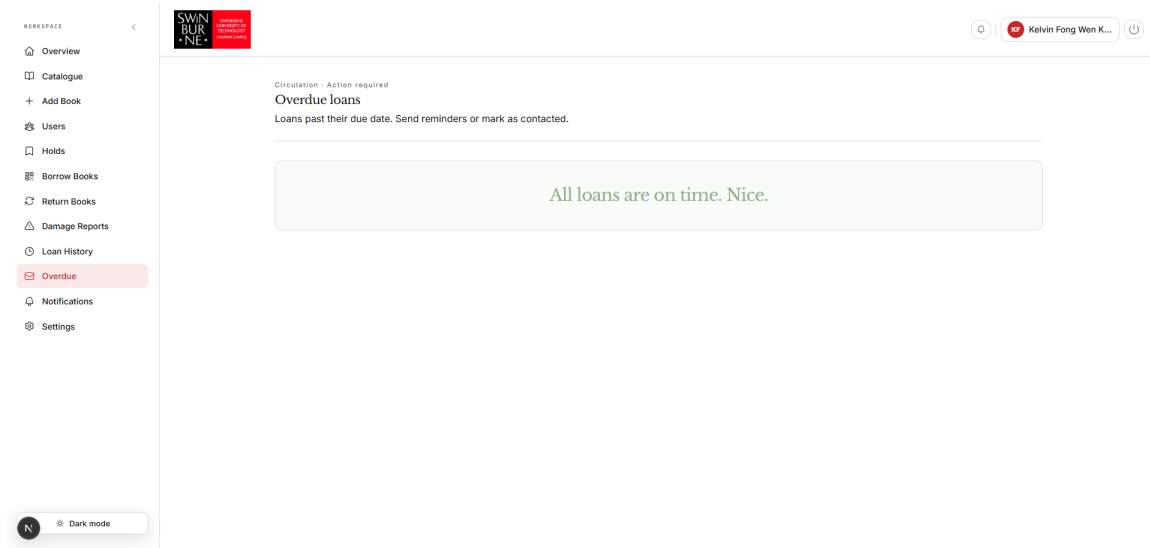


Figure 26 : Overdue Page

3. AI Reading Assistant consolidation

The recommendation engine was originally authored by Kenneth in March, then extended by Nigel with embeddings, retrieval, and a learning path AI in April. Ivan added chat history persistence in early May. On May 6 I merged the three teammates' AI branches and fixed the resulting type errors. On May 7 I consolidated every AI surface into a single /dashboard/reading-assistant page, authoring the system prompt builder (the full prompt is reproduced in section 4.5.4 of this report), the /api/reading-assistant route, the five readingAssistant UI components, and the GeneralChatHistory migration. I also dropped the LM Studio fallback so that Gemini became the sole AI provider, and rewired the AI call to consume multi turn conversation history together with the user's profile, active loans, and recent returns as context.

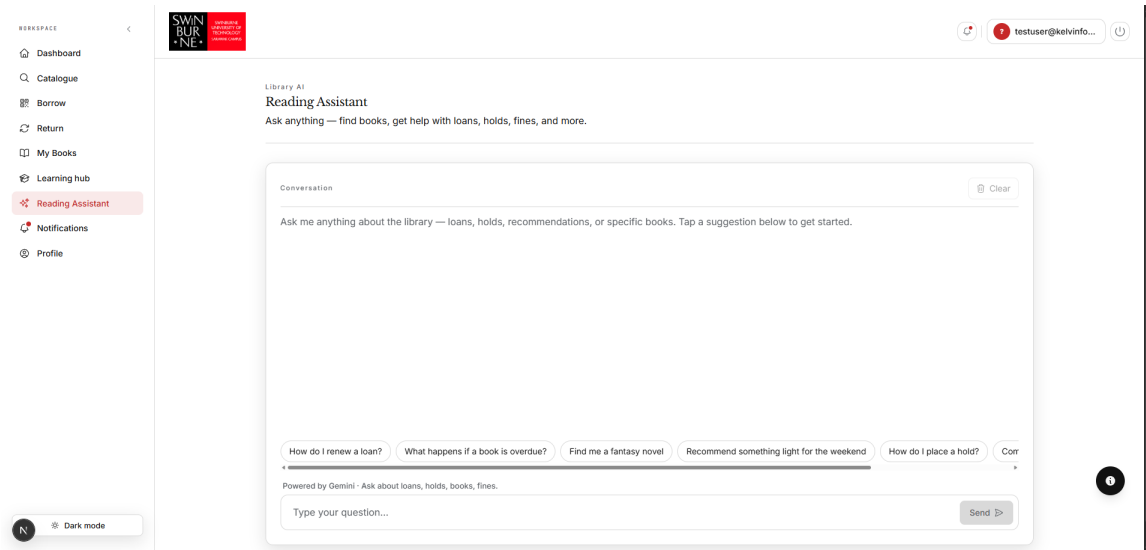


Figure 27: The empty state with the welcome message and the quick prompt chips

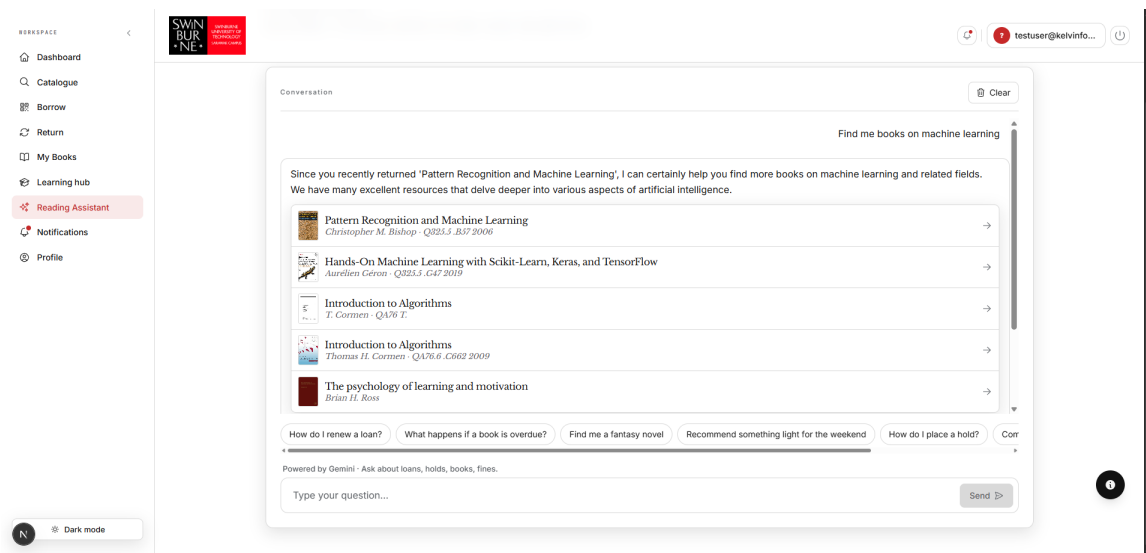


Figure 28: 1/2 An active multi turn dialogue showing book recommendations and grounded answers about the user's current loans

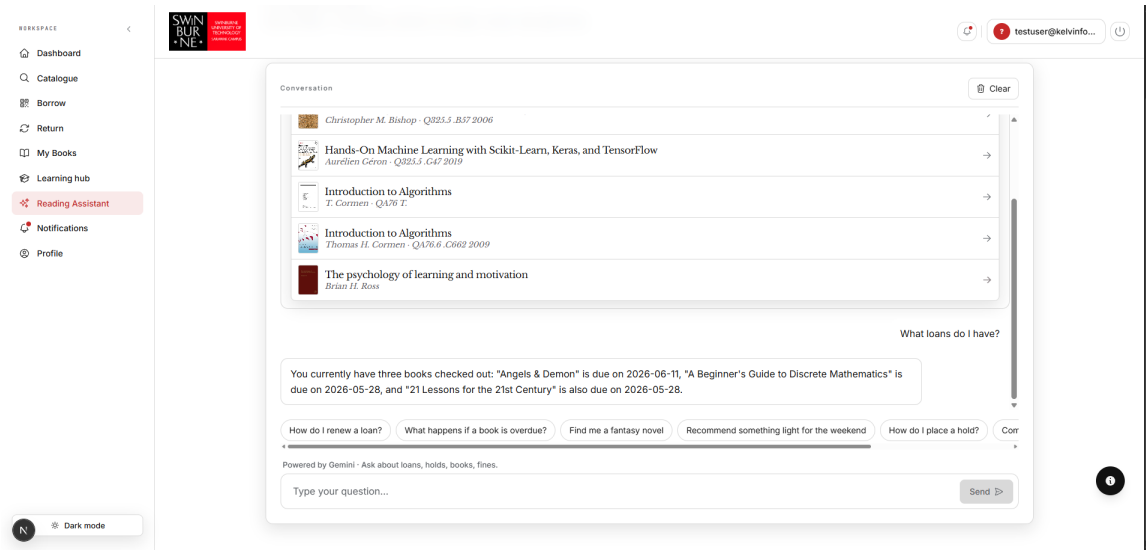


Figure 29: 2/2 An active multi turn dialogue showing book recommendations and grounded answers about the user's current loans

4. Integration and merge orchestration

I handled the integration of every teammate's branch during Sprint 3, merging Kenneth's, Nigel's, and Ivan's multi commit feature branches into the consolidated working branch. Because the branches had diverged during parallel development, several integration points produced interface or UI mismatches that I had to rewrite before the merged build would compile and run correctly. Verifying each merged build and resolving the resulting type errors is core Tech Lead work and accounts for several commits in the git history that do not appear as standalone features.

Nigel Zi Jun LING

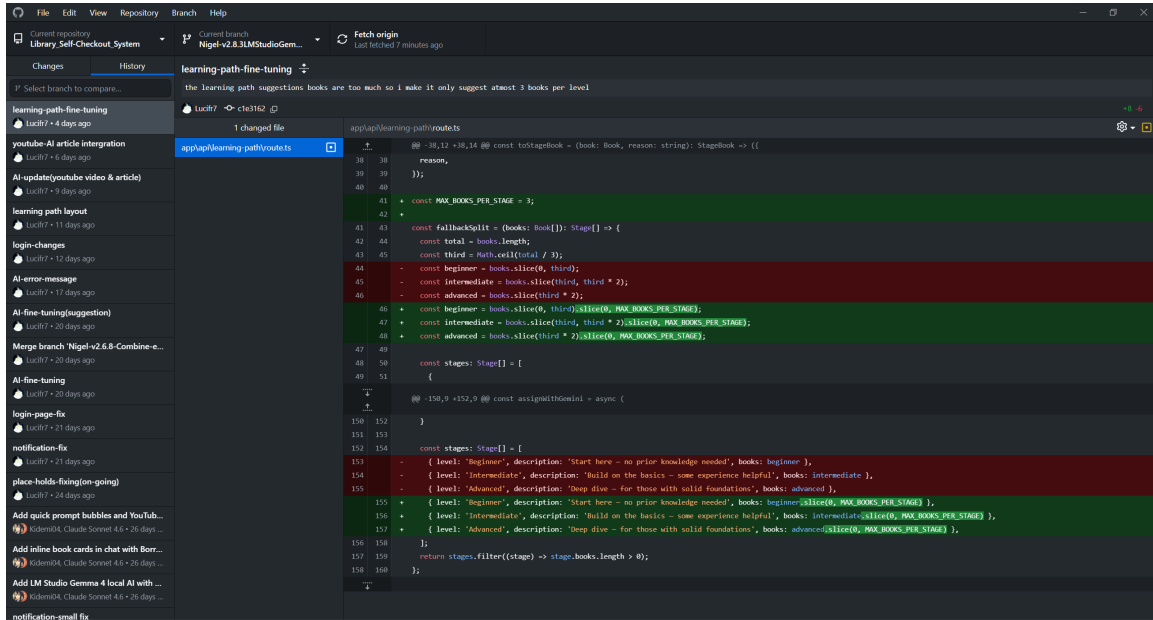


Figure 30 Nigel's GitHub History

I usually make a lot of changes in 1 branch instead of creating multiple branches per update.

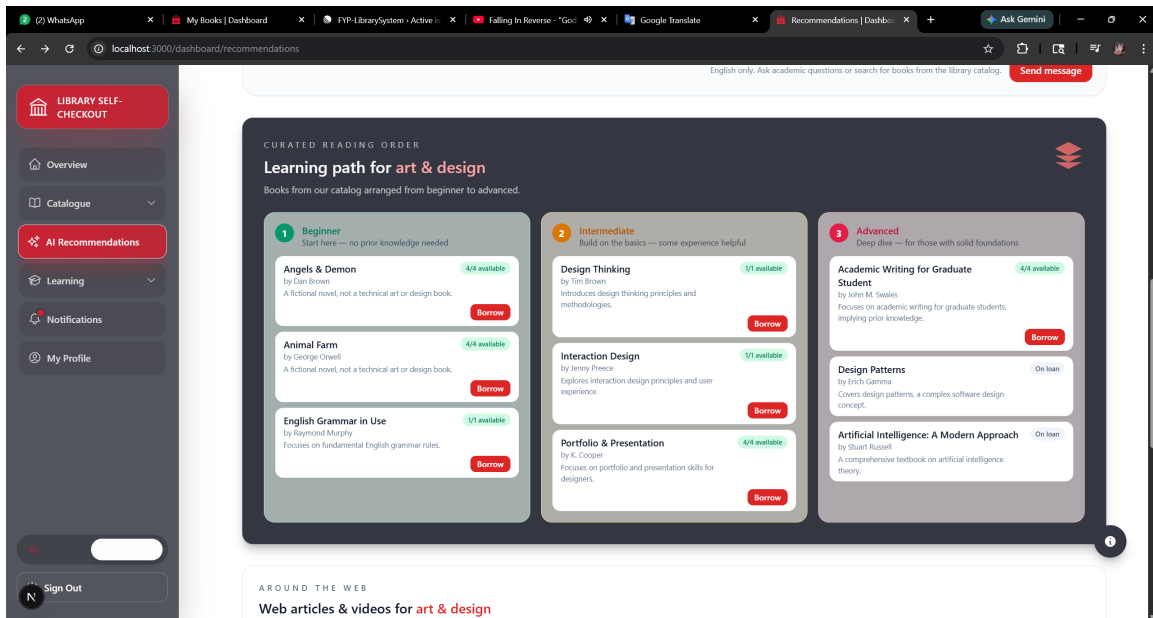


Figure 31 Learning Path that generate based on user prompt in AI chat is now more accurately

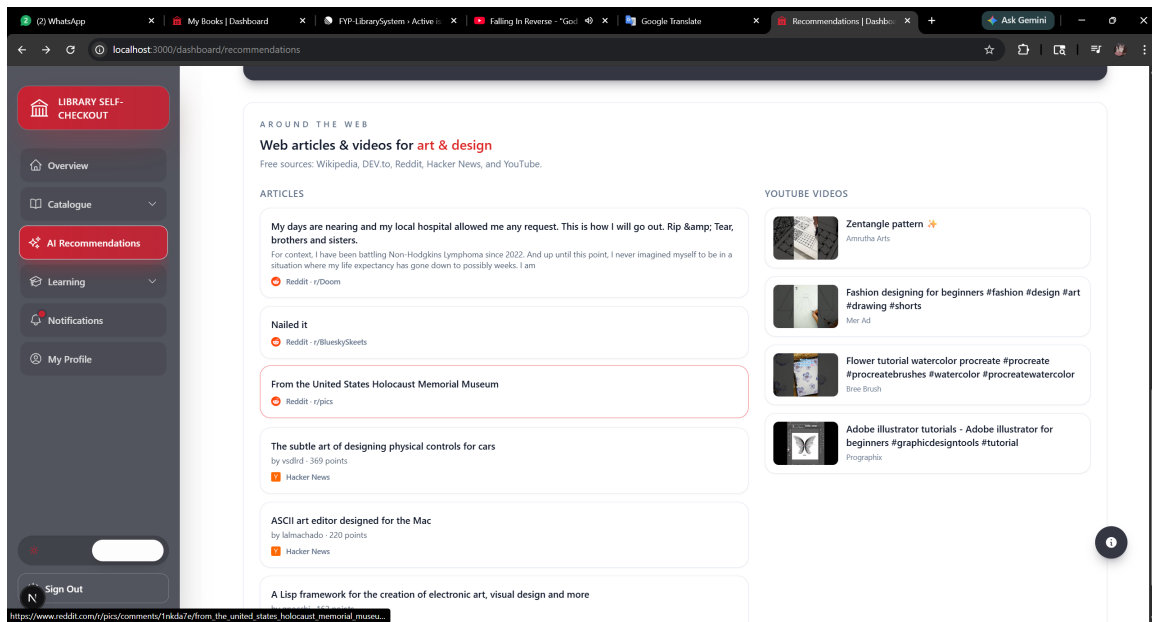


Figure 32 AI recommendation system that grabs online articles and YouTube videos from the Internet

Added features in the AI recommendation system

I have added new features such as like grabbing YouTube videos from the internet by using the YouTube Data API key, and suggesting online articles by using a free multi-source aggregator that pulls content in parallel from **Wikipedia** (one authoritative intro summary per topic), **DEV.to** (community-written developer articles), **Reddit** (real student/community discussions), and **Hacker News** (high-signal tech stories). All four sources are deduplicated by URL, capped at 8 articles total, and merged into a single "Around the web" panel that renders next to a YouTube videos panel — together giving students a Google-search-style sidebar of related reading and watching for whatever topic the AI detects from their message. Both panels are cached per-topic for 24 hours to stay well within free quota limits.

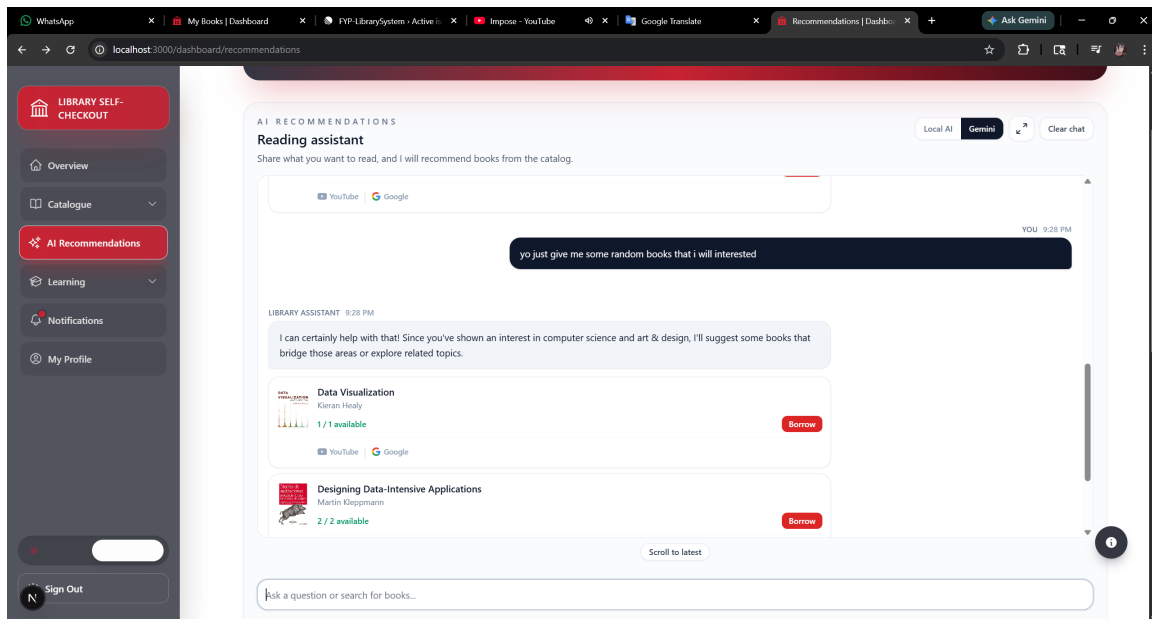


Figure 33 AI chat responds accurately to casual talks

Improving the AI recommendation system, I use gemini-2.5 flash lite for more quota, preventing ran out of usage. To fine-tune the AI model i use a couple ways to achieve the desired standard such as Made the learning path auto-trigger from the chat's detected interests (no separate topic input). Switched book retrieval from token matching to vector semantic search using Xenova embeddings + Supabase pgvector. To do that i added an "Embed all books" button for staff to generate embeddings. I also improved AI accuracy with stop word expansion, acronym mapping, and few-shot examples. Tightened the greeting regex so phrases like "yo i need books" don't get treated as small talk. Then I also made the AI read user's borrow history (last 20 books) to inform recommendations.

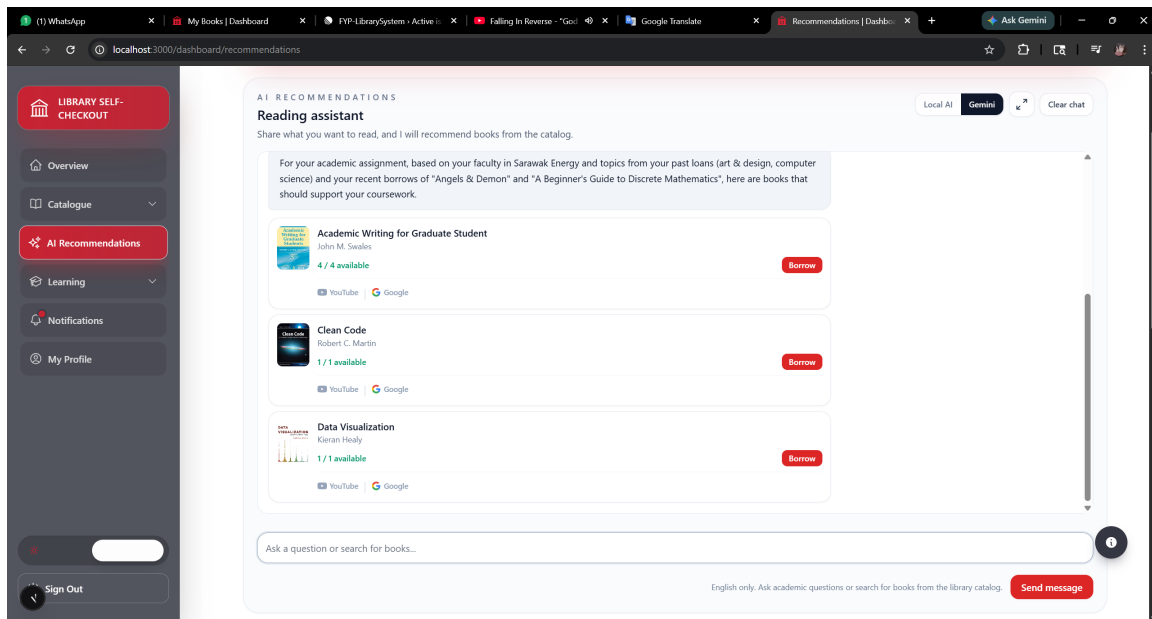


Figure 34 AI recommendation system is now able to suggest books that related to user's faculty or borrow history

I also added a personalized request handler ("based on my faculty/interests/history") that uses real user context and cites the reasons. I also made the Borrow button on AI recommendations route to the checkout page with the book pre-selected.

Manual Testing (Internal test)

I help the team to do manual test out the new version of system every time when it's done, and report the bugs and errors to the team.

1 Sprint Plan

1.1 SPRINT GOAL

In this Sprint, our team goal is adding more features to improve our system before getting into User Acceptance Testing phase such as UI upgrade, AI model fine-tuning, and learning hub implementation. Then, we plan to collect as much as possible of user feedback from Swinburne students and librarian staff to get more ideas and suggestions to improve our system. We first decided to change our entire user interface to make it more refreshing and user-friendly. We found out that our system still has couple bugs remain after trying to merge all the branch together, such as button non-functioning and logic error. Since we aim to present a fully implemented system that functions perfectly, we want to find out and fix all the bugs in our system.

1.2 SCOPE

The scope of Sprint 3 is enhanced throughout the entire system by upgrading the user interface, fine-tuning AI model, adding more practical features such as YouTube and online article suggestion and collecting feedback from user acceptance testing phase.

This research project will cover the following four major components. Firstly, from the perspective of UI/UX, there is the need to create an intuitive interface with interactive features and animations. Secondly, it will have a notification system for alerting and communicating with the user. Thirdly, it will develop an intelligent AI chatbot that will function as a smart system that will answer questions and give advice to users. Lastly, a learning center needs to be created that functions like social media.

1.3 ASSUMPTION & DEPENDENCIES

1.3.1 Assumption

To ensure the successful delivery of Sprint 3, the team has proceeded on the following assumptions. Each assumption identifies what the system takes for granted and the consequence if that assumption does not hold.

#	Assumption	Description	Consequence if Broken
1	Supabase as Central Data Store	Supabase (PostgreSQL) is the sole source of truth for all library data, including books, copies, loans, holds, damage reports, notifications, and the AuditLog table. The system assumes Supabase remains operational, correctly configured, and reachable from the Vercel deployment at all times.	If Supabase is unavailable, all core features — borrowing, returning, holds, notifications, and the admin dashboard — will cease to function. There is no offline fallback or secondary database.
2	Microsoft Azure AD as Primary Identity Provider	Users authenticate primarily through their Swinburne Microsoft 365 account via Azure AD and NextAuth, with an optional LinkedIn OAuth provider available as a secondary fallback when LINKEDIN_CLIENT_ID and LINKEDIN_CLIENT_SECRET are configured. The system assumes the Azure AD tenant is active, that the App Registration redirect URIs are correctly configured, and that Swinburne accounts remain valid in the Microsoft 365 directory. New accounts are automatically provisioned with the user role on first sign-in.	If Azure AD is unavailable or misconfigured, no user can sign in. The development bypass (DEV_BYPASS_AUTH) exists for local testing only and must never be enabled in production.
3	Role-Based Access is Enforced Server-Side	The system normalises three roles after sign-in: user, staff, and admin. Legacy or alias values (student → user, librarian → staff) are coerced through <code>resolveRole</code> in <code>middleware.ts</code> and <code>toDashboardRole</code> in <code>auth.ts</code> . Roles are stored in the Supabase Users table and re-read on every JWT rotation, and route access is enforced by the Next.js middleware before any page renders. The system	If the Supabase Users table role column is corrupted or out of sync, users may land on incorrect dashboards. An immediate manual SQL correction to the affected rows is the only recovery path.

		does not rely on Azure AD claims to determine permissions.	
4	External Integrations are Optional and Configurable	Beyond Azure AD and Supabase, the system integrates with a wider set of external services, all treated as optional with graceful fallback. The AI Reading Assistant calls Google Gemini (gemini-2.5-flash-lite) and an "Around the web" panel that aggregates Wikipedia, DEV.to, Reddit, Hacker News, and Brave Search. The Learning Hub uses YouTube Data API v3 (with bundled sample data fallback and a "Demo mode" banner), DEV.to (Community tab), and 5 RSS feeds — BBC Technology, TechCrunch, The Verge, Ars Technica, and Engadget — on the News tab. Local sentence embeddings use Xenova Transformers. Catalogue seeding scripts use Open Library and Google Books. Circulation events are synced to the legacy ILS via SIP2. Each integration has documented fallback behaviour.	If a fallback is not properly implemented for an integration, a quota breach or service outage in that integration could cause user-visible errors or broken pages. The team has verified fallback behaviour for all integrations during Sprint 3.

1.3.2 Dependencies

The system relies on the following external services and platforms. Services marked Required must be operational for the system to start. Services marked Optional have documented fallback behaviour when unavailable.

#	Dependency	Type	Purpose	Failure Impact
1	Vercel	Required	Production hosting and deployment platform. Vercel serves the Next.js application via its edge network, handles SSL termination, and automatically deploys the latest build on every push to the main branch. Preview deployments are also generated per branch to support parallel development review. Environment variables are managed	If Vercel is unavailable, the system is inaccessible to all users. No offline or alternative hosting fallback exists. Redeployment is possible by linking a new Vercel project to the same GitHub

			through the Vercel dashboard and are injected at build and runtime.	repository and re-entering environment variables.
2	GitHub	Required	Source code hosting, version control, and continuous integration. The repository hosts all application code, database migration scripts, and documentation. GitHub Actions runs the CI workflow (.github/workflows/test.yml) on every push, executing the Next.js production build and the full Jest test suite on an Ubuntu runner. Pull requests and branch protection are managed here.	If GitHub is unavailable, the team cannot push code or trigger deployments. The local codebase remains intact. CI would need to be re-established on an alternative platform if the outage were prolonged.
3	Microsoft Azure AD (Entra ID)	Required	Institutional identity provider for all user authentication. Every sign-in flow routes through the Swinburne Microsoft 365 tenant via OAuth 2.0. The Azure App Registration holds the client ID, client secret, and redirect URIs. Role assignment is managed separately in Supabase, but identity verification depends entirely on Azure AD being reachable.	If Azure AD is unavailable, no user can sign in regardless of role. All dashboard access is blocked. The development bypass exists for local testing only and must not be used in production.
4	Supabase (PostgreSQL)	Required	Backend database and file storage. Supabase stores all application data across 27 tables including users, books, copies, loans, holds, damage reports, notifications, and chat history. It also hosts two storage buckets: avatars (public) and damage-reports (private). Server-side API calls use the service role key; browser-side calls use the anon key.	If Supabase is unavailable, all data reads and writes fail. The system has no caching layer that would allow partial operation during an outage. Recovery depends on Supabase restoring service; no self-hosted fallback is in place for Sprint 3.
5	Gemini API (Google AI Studio)	Optional	Powers the AI Reading Assistant chat interface. The system uses gemini-2.5-flash-lite by default, configured via the GEMINI_MODEL environment variable. Each request sends a structured system prompt containing the student's profile, active loans, and recent return history, and forces JSON output via responseType. The model is called once per chat message.	If the Gemini API is unavailable or the daily quota is exhausted, the Reading Assistant returns a static fallback message. All other features — catalogue, borrowing, returns, holds,

				notifications — continue to work normally. The quota limit of gemini-2.5-flash-lite is approximately 1,000 requests per day on the free tier.
6	YouTube Data API v3	Optional	Powers the YouTube tab of the Learning Hub. Requests are sent server-side using the YOUTUBE_API_KEY environment variable. Results are cached per search topic to minimise quota consumption. The trending feed uses category ID 28 (Science and Technology) filtered to the MY region.	If the API key is missing or the daily quota is reached, the YouTube tab falls back to bundled sample video data and displays a clear "Demo mode" amber banner. The Community and News tabs are unaffected.
7	DEV.to API and RSS Feeds	Optional	Powers the Community and News tabs of the Learning Hub. The DEV.to public API requires no authentication. The News tab aggregates five RSS feeds (BBC Technology, TechCrunch, The Verge, Ars Technica, Engadget) fetched in parallel. Results are cached for one hour (Community) and 30 minutes (News) to reduce external load.	If DEV.to or the RSS endpoints are unreachable, the affected tab displays an empty state. No other features are impacted.

1.4 CAPACITY & FORECAST

Team Member	Estimated Hours
Kenneth Hui Hong CHUA	70-75 hours
Ivan Jia Wei HOAN	25-30 hours
Kelvin Wen Kiong FONG	75-80 hours
Nigel Zi Jun LING	30-35 hours
Total	Around 200-220 hours

The estimated number of hours per individual was calculated based on the difficulty of the tasks assigned to them and the role each member played in this sprint. Most hours were spent on the AI Reading Assistant consolidation (merging three teammates' AI branches into a single chat surface, fine-tuning the unified system prompt, and adding multi-turn history persistence), the full-system UI redesign rolled out in three batches across the student, staff, and admin dashboards, the Learning Hub three-tab implementation (YouTube, Community, News RSS), and the new admin and staff operational pages that move circulation tasks out of the Supabase studio and into the dashboard. Several feature-driven schema additions were also required this sprint, including the DamageReports table, the GeneralChatHistory table, the NotificationFlags table, the search_books RPC with trigram indexes, the avatars storage bucket, and the NotificationReads composite primary key fix. The remaining hours were divided among debugging, manual testing, and the multi-branch merge orchestration that the Tech Lead carried at the integration points. It is important to highlight that team capacity this sprint was slightly affected by concurrent academic obligations.

1.5 DEFINITION OF DONE

A story is considered done when once all feature are fully functional, and there are no any issue with either the frond-end or back-end, and the whole website has been deploy successfully deployed. Regarding basic function, the frond-end and back-end of the website must work well together. For instance, when borrow book, return book, scanning barcode and other function, there should be no system error. For AI recommendation, the AI need to analyse and understand the users need, and recommend suitable book for user. For the learning hub, system need to help user to plan their study methods. In addition, the website's interface has undergone a huge optimized, the interface should be more user-friendly and intitutive, making it easier for user to use it.

1.6 SPRINT MILESTONE & REVIEW ARTIFACTS

Throughout the 4 weeks of Sprint 3, we are aiming to finish the implementation of our system and start user acceptance testing phase by collecting feedback from Swinburne students and librarian staff, so we can get more ideas to improve our system. Also, we are planning to add some more practical features based on what we already have to provide a more satisfactory and user-friendly experience. These sections also included the work of progress that we have done in the past four weeks, presented by Linear issues, GitHub commit history, screenshots from the websites, also with test logs from backend testing. Our group always have 2 online meetings per week through Microsoft Teams for progression tracking and updating each member's current situation in our WhatsApp group. Not only that, but we also often meet at Swinburne open lab to discuss about our plan.

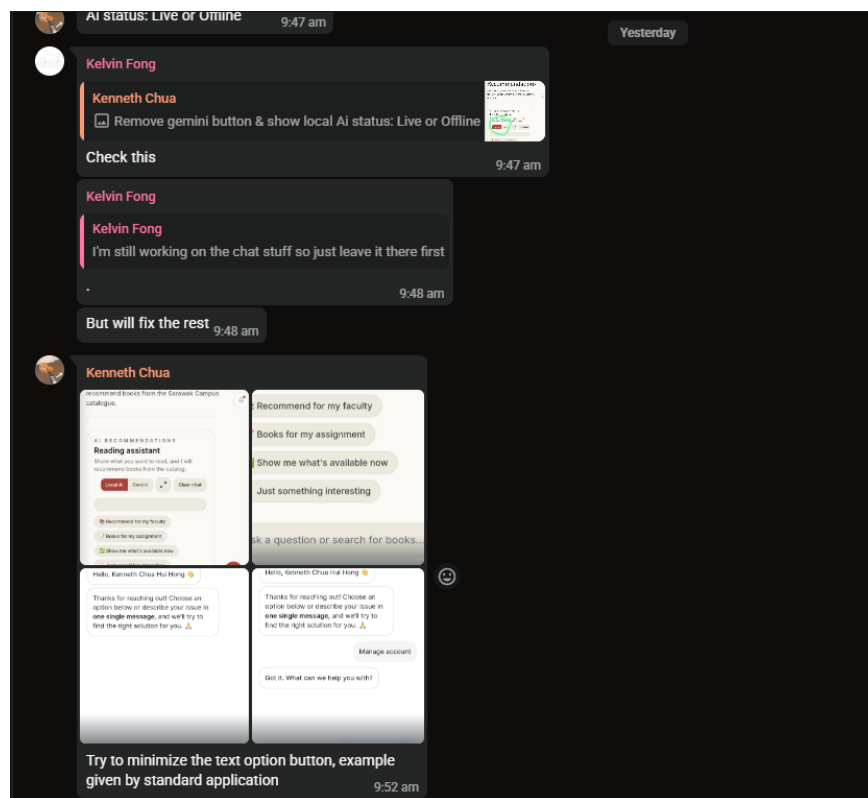


Figure 35: WhatsApp Chat

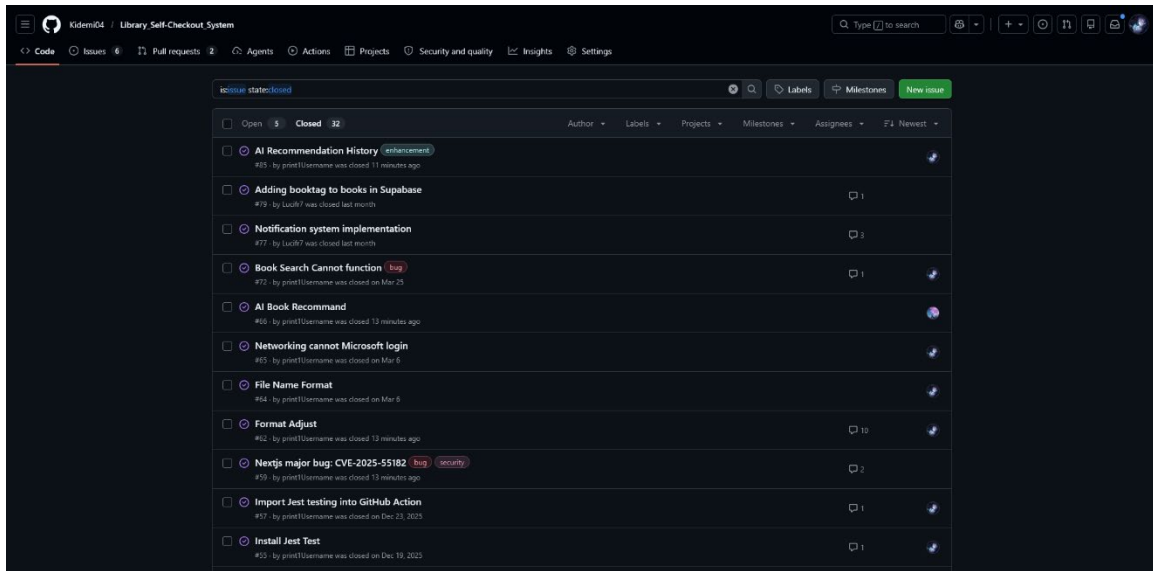


Figure 36 : GitHub Issue

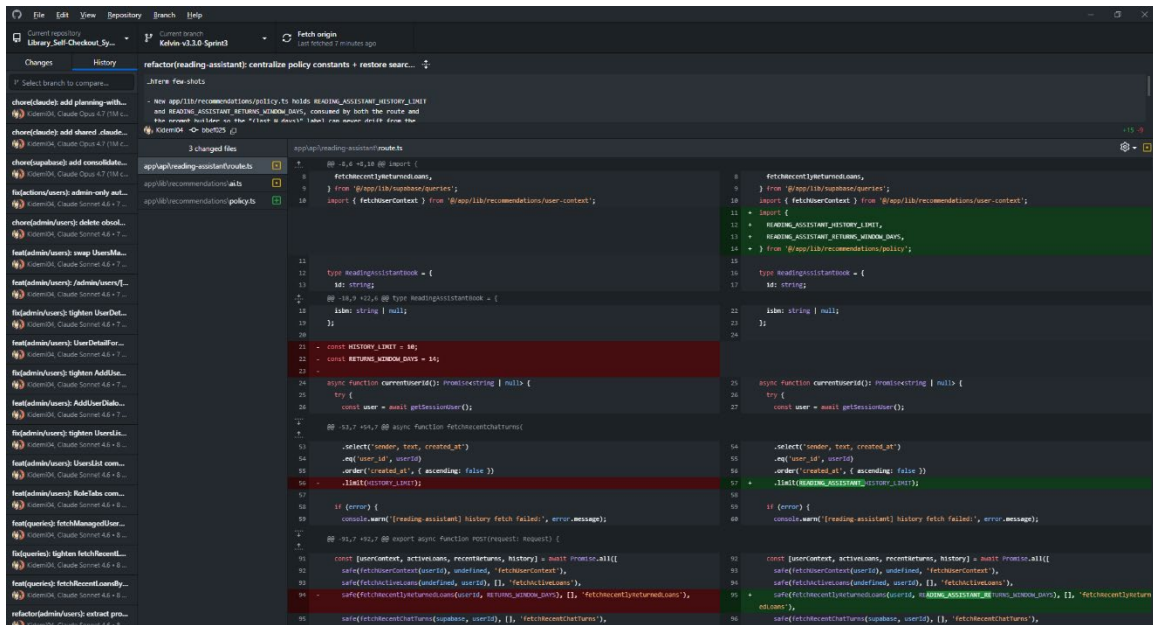


Figure 37: GitHub Change

1.7 SPRINT BACKLOG

ID	Item	Acceptance criteria	Est hrs	Assignee
S3-01	UI Remake & Refine	A user-friendly design system applied across student, staff, and admin dashboards as a three-batch rollout. Warm cream and charcoal palette in light mode, deep charcoal palette in dark mode, with the Swinburne red as the accent for critical CTAs. Elegant serif fonts for titles and clean sans-serif for body. Rounded corners on every shape, with fully rounded tag chips. Every interactive component responds correctly to hover, focus, and click states in both modes. Layouts adapt cleanly to mobile, tablet, and desktop breakpoints. Design tokens centralised in <code>tailwind.config.ts</code> .	50	Kelvin, Kenneth
S3-02	AI Optimize	The AI Reading Assistant should understand student requirements regardless of phrasing or acronyms, recommend appropriate books, avoid duplicates, and ground its replies in the student's profile and recent loan history. Includes vector semantic search using Xenova local embeddings with Supabase pgvector, automated book tagging via Gemini, stop-word and acronym expansion, tightened greeting detection, and a dedicated "Around the web" panel that aggregates Wikipedia, DEV.to, Reddit, Hacker News, and Brave Search alongside YouTube videos for the topic detected from the chat.	30	Nigel
S3-03	Learning Hub	A self-study browsing destination at <code>/dashboard/learning</code> organised into three tabs. The YouTube tab integrates the YouTube Data API v3 with six faculty-aligned quick-topic chips (Engineering, Computer Science, Cyber Security, Data Science, Business & Finance, Design) and a trending feed, falling back to bundled sample data with a "Demo mode" banner when no API key is configured. The Community tab pulls trending DEV.to articles filtered by emoji-tagged categories. The News tab aggregates 5 RSS feeds (BBC Technology, TechCrunch, The Verge, Ars Technica, Engadget) in parallel and interleaves the results. All tabs are responsive and cached server-side to stay within free quota limits.	35	Kenneth
S3-04	AI Chat History	Chat between the student and the AI Reading Assistant is persisted to Supabase (<code>GeneralChatHistory</code> table) so messages survive page refresh and sign-out. The last 10 turns are loaded back into the chat panel on mount and sent as conversation context to Gemini on every subsequent request. Students can clear their entire history through a confirmation modal.	15	Ivan
S3-05	System Testing & CI	The Jest test suite is stabilised and expanded to cover utility functions, API call shape (with mocked Supabase and Gemini SDKs), and key UI primitives. The GitHub Actions workflow at <code>.github/workflows/test.yml</code> runs both <code>pnpm build</code> and the	10	Ivan

		full Jest suite on every push, on Ubuntu, with no required secrets.		
S3-06	Notification	The notification system is extended with read / unread state, mark-all-read, per-user starring (NotificationFlags table), search, sort, and filter on the dedicated /notifications page. A bell popover shows the latest notifications without leaving the current page. The NotificationReads composite primary key drift is corrected via migration so mark-as-read actually persists across reloads.	20	Kenneth
S3-07	Camera Scanner Reliability	The camera scanner launches correctly on both mobile and desktop, reliably detects printed barcodes (including on glossy paper) using a multi-resolution decode strategy with quiet-zone padding, supports BarcodeDetector with a ZXing fallback, and degrades gracefully to manual entry when decoding fails.	8	Kelvin
S3-08	Admin & Staff Operational Pages	Four operational pages added or refactored. The admin Overdue page lists overdue loans with multi-facet filters (faculty, department, date range, search) and a filtered CSV export. The admin Users page introduces role-based tab navigation, an AddUserDialog modal, a dedicated /dashboard/admin/users/[id] detail route, and a danger zone for delete actions — replacing direct Supabase Studio edits. The admin Add Book page includes ISBN duplicate-check that redirects to the existing record on match. The staff Loan History page provides filter controls and a filtered CSV export. All four pages conform to the redesigned dashboard design system.	20	Kelvin
S3-09	Reading Assistant Consolidation	The previously separate AI surfaces (chat, recommendations, FAQ, Help Centre) are consolidated into a single /dashboard/reading-assistant page with multi-turn dialogue, intent classification across six intents (find_books, answer, both, greeting, off_topic, loan_status), and grounded answers about the student's current loans and recent returns. Includes a new /api/reading-assistant route handler, five readingAssistant UI components, and a unified system prompt builder. The LM Studio fallback path is removed so that Gemini becomes the sole AI provider.	12	Kelvin
S3-10	Multi-branch Integration	Tech Lead integration of every teammate's feature branch into the consolidated working branch. Resolves the type errors and interface mismatches that surface only at merge time, verifies each merged build runs end-to-end, and keeps the deployable branch green for the demo.	10	Kelvin

2 Final Documentation Deliverables

2.1 INTRODUCTION

The Library Self-Checkout System is a web-based library platform developed for Swinburne University of Technology Sarawak Campus. The system allows students to borrow, return, renew, and place holds on library books directly from their own mobile devices using a built-in barcode scanner. Beyond standard circulation, the system provides an AI powered reading assistant for personalised book recommendations and library queries, a curated learning hub with YouTube tutorial videos and community articles, and role-based dashboards tailored to the needs of library staff and administrators. The system operates alongside the library's existing Integrated Library System by communicating through the SIP2 protocol so that records stay consistent across both platforms.

This document is the final technical documentation for the Library Self-Checkout System. Its purpose is to give users, system administrators, and future developers a clear and complete reference for the system. The document covers how to use the system, how to install and deploy it, how the system was designed and implemented, what testing was carried out, and what the known limitations are. It should be read alongside the project codebase, which is hosted on GitHub under the repository name Library Self-Checkout System.

The system serves three distinct groups of users. All three groups must authenticate using a Swinburne Microsoft 365 account before accessing any feature of the platform. Because access is tied to the Swinburne institutional directory, the system is not available to the general public and operates exclusively within the campus community.

Students are the primary and highest volume audience. They use the system daily to manage their borrowing activity, discover new books through the AI reading assistant, and access learning resources through the learning hub. Every checkout, return, hold placement, damage report, and AI interaction originates from a student account. The first time a student signs in, the system automatically creates their account with no registration form required.

Library staff form the second group. Their focus is on operational tasks that require human judgement, such as reviewing damage reports submitted by students and searching historical loan records when disputes need to be resolved. Staff can also assist students at the counter by processing checkouts and returns on their behalf using the same scanner interface.

Administrators form the third and smallest group. They have the broadest level of access in the system, including the ability to manage the book catalogue, edit user accounts and roles, and export overdue loan reports. In practice, this role corresponds to library IT personnel or senior library management.

2.2 SYSTEM OVERVIEW

2.2.1 System Architecture

A student opens the platform in their mobile browser, and the request first reaches Vercel's edge network, which handles SSL termination and routes it into a Next.js 15 App Router application. Before any page loads, the NextAuth middleware intercepts the request, verifies the student's Microsoft 365 session through Azure AD, and enforces role-based route guards. Students cannot access staff or admin routes, and any unauthenticated request is redirected to the login page before any content is served.

Once authenticated, pages are rendered as React Server Components on the server. These components fetch data directly from Supabase using a server only service role client, meaning database credentials are never exposed to the browser. When a student takes an action such as borrowing or returning a book, the browser triggers a React Server Action. The action validates the payload using Zod, writes the result to Supabase, pushes the same event to the library's existing SIP2 service with a five second timeout, records the action in an audit log, and revalidates the affected pages so the interface reflects the latest state straight away.

AI features follow a different path. The Reading Assistant sends requests to a Next.js route handler, which fetches the student's profile, active loans, and recent return history from Supabase and passes that data as context to the Gemini API. The response is requested in structured JSON format so the application can parse and render it reliably in the chat interface. The Learning Hub follows a similar pattern, calling the YouTube Data API and Reddit feeds from the server before streaming the rendered content to the browser.

The browser is intentionally kept lean. Most content arrives as rendered HTML from the server. Only the parts of the interface that need immediate interactivity, such as the camera scanner, the chat input, and form fields, are loaded as client components in the browser.

2.2.2 Key Features

The first feature is mobile self-checkout through barcode scanning. Students open the Borrow page, grant camera permission, and scan a book barcode using their phone. The system validates copy availability, checks the student's loan count against the policy limit, records the loan in Supabase, and notifies the legacy ILS through SIP2 all without any staff involvement.

The second feature is full end to end circulation. Students can borrow books, return them, renew active loans, place holds on titles currently on loan, and submit damage reports on returned copies. The loan policy is enforced on the server, allowing a maximum of three books per student at a time with a thirty day loan period per title.

Catalogue browsing and search is the third feature. Students and staff can browse the full book catalogue, filter by category, and search using keywords. The search is backed by a Supabase stored procedure that performs full text matching. The search input is debounced so queries are only sent when the user pauses typing, keeping the interface responsive.

The AI Reading Assistant is the fourth and central feature introduced in Sprint 3. It is a conversational chat interface powered by Google Gemini that answers questions about the catalogue, checks loan status, and provides personalised book recommendations. Every AI call is enriched with the student's profile, active loans, and recent return history. Gemini is configured to return structured JSON output so the application can parse responses reliably.

Personalised book recommendations form the fifth feature. The system maps each student's faculty and stated interests to relevant book categories using a predefined mapping in the codebase. These recommendations are delivered through the Reading Assistant rather than as a standalone page, consolidating the discovery experience in one place.

The sixth feature is the Learning Hub, a curated resource surface that aggregates YouTube tutorial videos through the YouTube Data API v3 and community news and articles through Reddit and RSS feeds.

An in app notification system is the seventh feature. Students, staff, and admins all receive alerts for events relevant to their role, including overdue loan reminders, hold ready alerts, and damage report status updates. A red indicator on the sidebar icon shows unread items. Deduplication logic in the notification service prevents the same reminder from being sent more than once for the same event.

Role based dashboards are the eighth feature. The system provides three entirely different dashboard experiences depending on the user's role. Students get a consumer oriented interface focused on borrowing and discovery. Staff get an operational interface built around damage report triage and loan history lookup. Admins get a management interface with system wide statistics and administrative tools. Route access is enforced at the middleware layer before any page begins to render.

Authentication and account management form the ninth feature. All users sign in through their Swinburne Microsoft 365 account via Azure AD single sign on. First time sign ins are automatically provisioned as student accounts. Students can update their display name, faculty, interests, and profile avatar. An optional LinkedIn sign in provider can be enabled when credentials are configured.

The tenth feature is SIP2 interoperability with the library's legacy Integrated Library System. Every checkout and return event is sent to the SIP2 service so the existing ILS stays in sync. The call uses a five second timeout and is treated as best effort, meaning if SIP2 fails the action still succeeds in Supabase. This approach keeps the self-checkout experience reliable even on slower network connections.

An audit trail is the eleventh feature. Every significant circulation action in the system, including checkouts, returns, holds, and damage reports, is written to an audit log table. Each entry captures who performed the action, what the target was, and the outcome. This provides full traceability for operational reviews.

Accessibility and UX polish is the twelfth feature. The interface supports both light and dark mode through a theme toggle on the profile page. Smooth animations built with Framer Motion are applied to page transitions and interactive elements. The layout is mobile first, meaning all flows are usable on a phone, though wider screens provide more comfort for pages with large data tables.

2.2.3 Technologies Used

Layer	Technology	Version	Purpose in this Project
Framework	Next.js (App Router)	15.5.14	Full stack React framework covering routing, server components, server actions, and API routes
Runtime Language /	TypeScript	5.7.3	Static typing across the entire codebase
UI Library	React	19.2.4	Component model using server components with small interactive client islands
Bundler (dev)	Turbopack	Bundled with Next.js 15	Faster hot module replacement during development
Styling	Tailwind CSS	3.4.17	Utility first CSS with custom design tokens defined in tailwind.config.ts
Tailwind Plugins	@tailwindcss/forms	0.5.10	Sensible default styles for form inputs
Theming	next-themes	0.4.6	Light and dark mode toggle with persistent user preference
Animation	Framer Motion	12.23.24	Page transitions, blur fade, and shimmer effects in the UI
Icons	Heroicons	2.2.0	Outline and solid icon set used throughout the dashboards
Class Merging	clsx + tailwind-merge	2.1 / 3.4	Conflict free conditional Tailwind class composition
Database	Supabase (PostgreSQL)	Hosted	Source of truth for users, books, copies, loans, holds, damage reports, notifications, and the audit log
Supabase Clients	@supabase/supabase-js + @supabase/ssr	2.99 / 0.8	Server side service role client and browser client
Direct PG Driver	postgres	3.4.6	Available for raw SQL queries; Supabase client is the preferred data access path
Authentication	NextAuth.js (Auth.js v5)	5.0.0-beta.25	Session management and OAuth flow handling
Identity Provider	Microsoft Azure AD (Entra ID)	Hosted	Institutional single sign on via the Swinburne Microsoft 365 tenant
Password Hashing	bcrypt	5.1.1	Available for future credential based providers; not used in the primary Azure AD flow
Validation	Zod	3.25.17	Schema validation for server actions and API payloads
AI / LLM	Google Gemini (gemini-2.5-flash-lite by default)	Via REST	Powers the Reading Assistant, recommendation generation, and intent classification with enforced JSON output
Local Embeddings	@xenova/transformers	2.17.2	On device text embeddings; optional and marked as an external server package

Barcode Scanning	ZXing (@zxing/library + @zxing/browser)	0.21 / 0.1	Camera based barcode scanner for the self checkout and return flows
External Catalog	YouTube Data API v3	Hosted	Learning Hub video suggestions based on student faculty and interests
External Feeds	Reddit JSON and RSS XML	Hosted	Community and developer news tab in the Learning Hub
Legacy ILS Bridge	SIP2 (Standard Interchange Protocol 2)	Custom HTTP wrapper in lib/sip2.ts	Keeps the library's existing ILS in sync with checkout and return events via a five second timeout
Optional AI Tooling	Model Context Protocol SDK	1.27.1	mcp/server.mjs exposes Supabase library data to external AI agents over stdio
Hooks Utility	use-debounce	10.0.4	Debounces catalogue search inputs to reduce unnecessary database queries
Testing	Jest + jsdom + Testing Library	Jest 30.2	Unit and integration tests located in tests/
Package Manager	pnpm	Latest	Dependency management with security patch overrides declared in package.json
Hosting / Deployment	Vercel	Hosted	Production hosting, preview deployments per branch, and environment variable management
CI	GitHub Actions	Hosted	.github/workflows/test.yml runs the full Jest suite on Ubuntu for every push

2.3 USER MANUAL

The Library Self-Checkout System is a web based application that does not require any installation on your device. It can be accessed from any modern browser on a phone, tablet, or computer. The system is hosted on Vercel and the URL will be provided by the library or your system administrator.

Before using any part of the system, you need to sign in with your Swinburne Microsoft 365 account. Personal email addresses such as Outlook or Gmail accounts are not accepted because the system is configured specifically for the Swinburne institutional directory. If you do not have a Swinburne account, you will not be able to access the system.

To sign in, open the system URL in your browser. You will see a landing page with a Log In button at the top right. Click it to reach the login page, then click Sign In with Microsoft and follow the Microsoft authentication steps. If this is your first time signing in, the system will automatically create a student account for you. No registration form is needed.

After signing in you will be taken straight to your dashboard. The exact page depends on your role. Students land on the student dashboard, staff land on the staff desk view, and admins are taken to the admin overview. If you need staff or admin access, a system administrator must manually update your role from the admin users page. There is no way to request a role upgrade from within the application itself.

To sign out, scroll to the bottom of the sidebar on any page and click the Sign Out button. This will clear your session and take you back to the login page.

2.3.1 Installation Guide

The Library Self-Checkout System is a web based application, which means there is nothing to install on your phone, tablet, or computer. As long as you have a modern browser and an active internet connection, you can open the system and start using it right away.

For the best experience, we recommend using a recent version of Google Chrome, Microsoft Edge, or Mozilla Firefox on a desktop or laptop, the default Chrome browser on Android, or Safari on iPhone and iPad. The barcode scanner uses your device camera through the browser, and Chromium based browsers such as Chrome and Edge tend to give the smoothest scanning experience on mobile devices. Older browsers that do not support modern camera access will not be able to use the scanner at all.

The system needs a steady internet connection at all times. There is no offline mode, so if your connection drops while you are using the system, pages will fail to load and any action that needs to reach the server, such as borrowing or returning a book, will not go through until your connection comes back. On campus, the Swinburne network or eduroam is the most reliable option. Off campus, any home or mobile internet connection that can load regular websites will work fine.

To sign in to the system, you must have an active Swinburne Microsoft 365 account. Personal email accounts such as Outlook, Gmail, or Yahoo are not accepted because the system is configured to authenticate only against the Swinburne institutional directory. If you do not yet have a Swinburne Microsoft 365 account, please contact the Swinburne IT helpdesk before trying to use the system. The system URL itself is not published publicly. The library or your system administrator will share the URL with you directly through email or through an internal Swinburne notice.

If you plan to use the self checkout scanner on your phone, your browser will ask for camera permission the first time you open the Borrow or Return page. Tap Allow when the prompt appears. Without camera permission, the scanner will not be able to read book barcodes and you will need to fall back on manual entry instead. For convenience, you can also add the system URL to your phone home screen using your browser's Add to Home Screen option, which gives you a tappable icon rather than having to type the URL each time you visit.

2.3.2 How to Start the Application

To open the system, visit the URL provided by your library or system administrator in your browser. The landing page shows the Swinburne logo, a campus background image, and a welcome section in the middle of the screen with the heading "Swinburne Library Self-Checkout" and a short description. To enter the system, click either the Get Started button in the welcome section or the Log In button at the top right of the screen. Both take you to the same login page. There is also a Learn More button next to Get Started, which opens the official Swinburne library website in a new browser tab in case you want to read more about library services first.

Once you reach the login page, you will see a centred panel titled "Swinburne Sarawak Library" with the Swinburne logo and one or two sign in options. The Sign in with Microsoft button is the primary option and is always shown. Click it to begin the Microsoft authentication process. The system is configured to always show the Microsoft account picker, so even if you are already signed in to a Microsoft account elsewhere in your browser, you will still be asked to choose which account to use. Pick your Swinburne Microsoft 365 account and complete any verification steps that Microsoft requires, such as multi factor authentication. While the sign in is in progress, the button label changes to "Signing in..." so you know your request is being processed.

After Microsoft confirms your identity, the system signs you in automatically. If this is the first time you are using the system, an account is created for you in the background using the email address linked to your Swinburne Microsoft 365 account. New accounts are created with the student role by default. There is no registration form to fill in and no separate password to set up, since the system relies entirely on Microsoft for authentication.

You will then be taken straight to the dashboard that matches your role. Students land on the student dashboard, which shows your active loans, any holds you have placed, and a selection of available books to browse. Staff land on the Desk page, which is the operational home screen with quick access to checkout, return, and the holds queue. Administrators are taken to the Admin Overview page, which shows system wide statistics such as total books, active loans, and overdue items, along with a recent activity feed. If you ever need staff or administrator access and you only see the student view, contact your system administrator. Role changes can only be made through the admin Users page, and there is no self service option to upgrade your own role from within the system.

When you are finished using the system, scroll to the bottom of the sidebar on any page and click the Sign Out button. This will end your session and return you to the login page. Closing the browser tab without signing out will also clear your session after a short period, but using the Sign Out button is the cleanest way to leave the system, especially on a shared device.

2.3.3 Controls Instructions

This sub-section explains the interactive controls available across the system. Many controls appear on multiple pages, since the sidebar, the theme toggle, search inputs, and the camera scanner are reused throughout the system. For that reason, controls are grouped by type rather than by page. For details on how these controls fit together to perform a complete task, refer to 2.3.5 Features Guide.

Navigation controls:

The sidebar on the left side of the screen is your main way of moving between pages. The exact items in the sidebar depend on your role, but the structure is the same across all three roles. Each sidebar item is a link, so clicking it opens the corresponding page in the main content area. The Sign Out button is always at the bottom of the sidebar.

Control	Location	What it does
Sidebar items	Left side of every page	Navigates to the corresponding section such as Catalogue, Borrow, My Books, or Notifications
Theme toggle	Top of the student dashboard and on the profile page	Switches between light and dark mode
Sign Out button	Bottom of the sidebar	Ends your session and returns you to the login page
Top search bar	Top of the desktop student dashboard	Searches the catalogue by title, author, or ISBN
Notification icon	Page header	Opens the Notifications page; shows a red dot when you have unread items

Search and filter controls:

Most list pages in the system support searching and filtering. Search inputs accept free text and update the list when you press Enter or click the search icon. Filter pills are small rounded buttons that toggle between options. The active pill is highlighted, and you change the filter by clicking a different pill or by clicking the Clear link to reset the filter group.

Control	Where you find it	What it does
Search input	Catalogue, Loan History, Damage Reports, Users, Notifications	Filters the list to items matching the keyword
Category pills	Catalogue page	Filters books by category such as All, Computing, Design, Business, Engineering, Psychology, Self help, and History
Status pills	Holds management, Loan history	Filters by status such as Active, Returned, Overdue, Ready, or Queued
Range pills	Loan history, Damage reports, Overdue list	Filters by date range such as 30 days, 6 months, Semester, or All time
View toggle (Grid or List)	Catalogue page	Switches between card grid view and compact list view
Sort dropdown	Notifications	Sorts notifications by date or title, ascending or descending
Clear link	Filter rows	Clears the active filter or search query

Scanner controls:

The barcode scanner is used for the self checkout flow, the return flow, and the admin book creation flow. Opening any of these pages presents a Scan with camera button. Clicking it opens the scanner modal, which uses your device camera to detect a book barcode automatically. If the camera is unavailable or the barcode is unclear, the modal also offers a photo upload option and a manual entry input as fallbacks.

Control	What it does
Scan with camera	Opens the camera and starts detecting barcodes automatically
Upload a photo instead	Lets you pick an image from your device that contains a barcode
Camera switch icon	Switches between the front and rear camera; only shown when multiple cameras are detected
Settings icon	Opens a drawer with a camera source dropdown, a refresh button, and a debug log panel
Manual entry input	Lets you type or paste an ISBN, a SWI barcode, or a loan ID by hand
Submit button (manual entry)	Looks up the typed code
Back button	Closes the scanner modal without scanning
Refresh	Restarts the camera preview if it gets stuck

AI Reading Assistant controls:

The Reading Assistant is available only to students. The interface is a simple chat with a text input at the bottom, suggested prompt chips above the input, and a Clear Chat button. Your previous chat history loads automatically when you open the page.

Control	What it does
Quick prompt chips	Tappable suggestions shown above the chat input; tapping a chip fills the input box with that prompt so you can edit it before sending
Chat input	A free text box where you can type any question about books, loans, or library policy
Send button	Sends your message to the AI assistant and waits for a reply
Clear Chat	Permanently deletes your entire chat history after a confirmation prompt

Form controls and field rules:

Forms across the system use standard inputs for text, numbers, dropdowns, file uploads, and dates. Most fields have a maximum length so the database stays consistent. The table below summarises the validation rules for the most common fields you will encounter when filling in a form.

Field	Type	Limit
Display name	Text	No maximum
Username	Text	50 characters
Phone	Text	20 characters, accepts only digits, spaces, plus signs, and dashes
Preferred language	Text	50 characters
Faculty	Text	100 characters
Department	Text	100 characters
Bio	Multi line text	500 characters
Avatar image	File upload	2 MB maximum, must be JPEG, JPG, PNG, GIF, or WEBP
Book title (admin)	Text	200 characters
Book author (admin)	Text	200 characters
Publication year (admin)	Text	4 characters
Number of copies (admin book create)	Number	Between 1 and 20
Tags (admin book create)	Tag chips	Maximum of 6 tags

Action buttons:

These are the main action buttons that appear across different pages. Primary actions are marked with red styling and secondary or destructive actions are usually grey or outlined. Many of these buttons trigger a confirmation modal before they actually run, especially for destructive or irreversible actions.

Button	Where it appears	What it does
Borrow	Catalogue card, Book detail page	Opens the checkout page with the book pre selected
Confirm loan	Checkout page	Records the loan after all policy checks pass
Mark returned	Return page	Records the return after a confirmation modal
Renew	My Books active loan card	Extends an active loan by another 30 days; only enabled if the loan is not overdue, has not hit the 2 renewal limit, and no other student has placed a hold
Place a hold	Book detail page when no copies are available	Adds you to the queue for that book
Cancel hold	My reservations, Books Holds management	Removes you or the patron from the queue
Mark ready or (staff admin)	Holds management page	Marks a queued hold as ready for pickup and sends a pickup notification
Send reminder (admin)	Overdue list	Sends an overdue reminder; cannot be sent more than once within 24 hours per loan

Mark as read	Notifications page	Marks a single notification as read
Mark all read	Notifications page	Marks every unread notification as read in one click
Export CSV	Loan history (staff), Overdue list (admin)	Downloads a CSV file of the currently filtered list
Add User (admin)	Admin Users page	Creates a new staff or admin account
Save (admin user row)	Admin Users page	Saves edits to that user's details
Delete (admin user row)	Admin Users page	Permanently deletes that user account after confirmation

Confirmation modals:

Many actions in the system that change or delete data show a confirmation modal before they complete. The modal explains exactly what is about to happen and gives you two buttons: a primary action button such as "Yes, mark returned" or "Yes, delete", and a Cancel or Go back button. Clicking outside the modal does not confirm or cancel it. You must click one of the two buttons to either commit the action or abort it. This pattern applies to confirming a book return, marking a hold ready, cancelling a reservation, creating or editing a book, adding or deleting a user, sending an overdue reminder, and clearing your AI chat history.

2.3.4 UI Explanation

This sub-section explains the structure of the system's interface, walking through the page layout, the differences between the three role specific views, the theming options, and the visual cues such as colour coded status pills and empty states. Together these explain not just where things are on each page but what you are actually looking at when you see them on screen.

Overall page layout:

Every page in the system uses the same overall layout: a fixed sidebar on the left and a main content area on the right that fills the rest of the screen. The sidebar contains the navigation links for the current role and the Sign Out button at the bottom. The main content area shows the page title or a search input near the top, depending on the page, and the body of the area holds the actual content such as dashboard cards, the catalogue grid, or a form. On mobile devices, the sidebar collapses into a top header with a menu icon. Tapping the icon slides the sidebar in from the left edge of the screen, and tapping outside the sidebar or selecting an item closes it again.

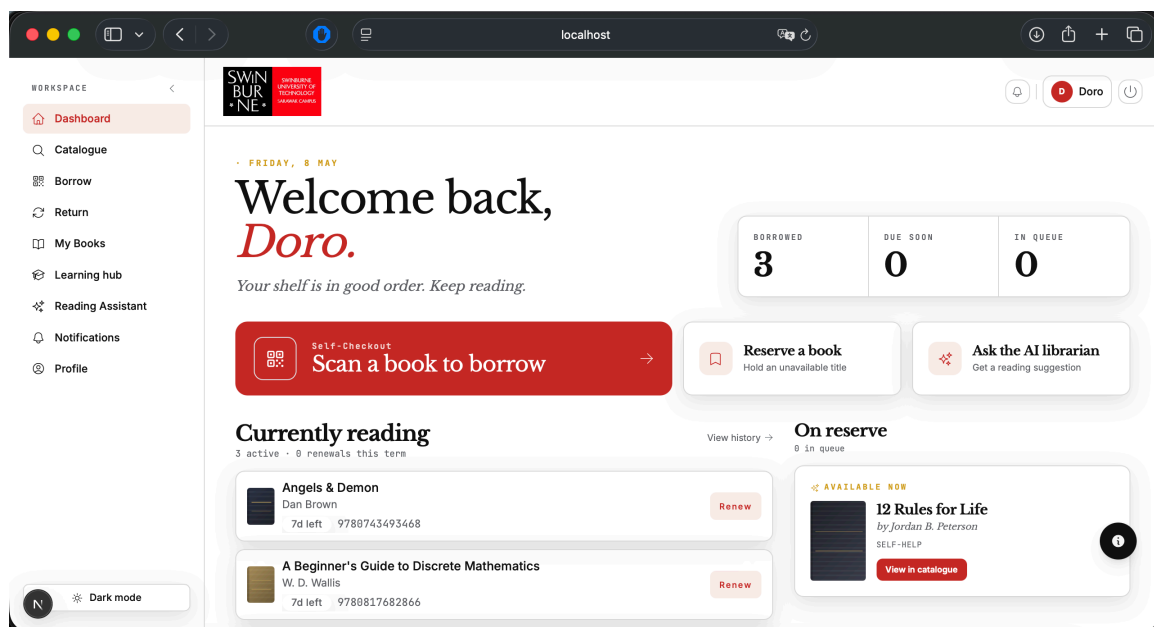


Figure 38: The student dashboard showing the sidebar on the left, the main content area in the centre, and the theme toggle in the top right.

The three role specific views:

The system uses the same shell across every role, but the contents differ significantly. Students see a discovery focused interface with the AI Reading Assistant, the Learning Hub, and personal book picks. Staff see an operational interface centred on the Desk page, with damage report triage, holds management, and loan history search as their main daily tools. Admins see a management interface that opens with a KPI overview page including total books, active loans, overdue counts, a fourteen day circulation chart, and the top borrowed titles. Each role's sidebar shows only the

items relevant to that role, so a student will not see admin only options, and a staff member will not see anything that is admin only.

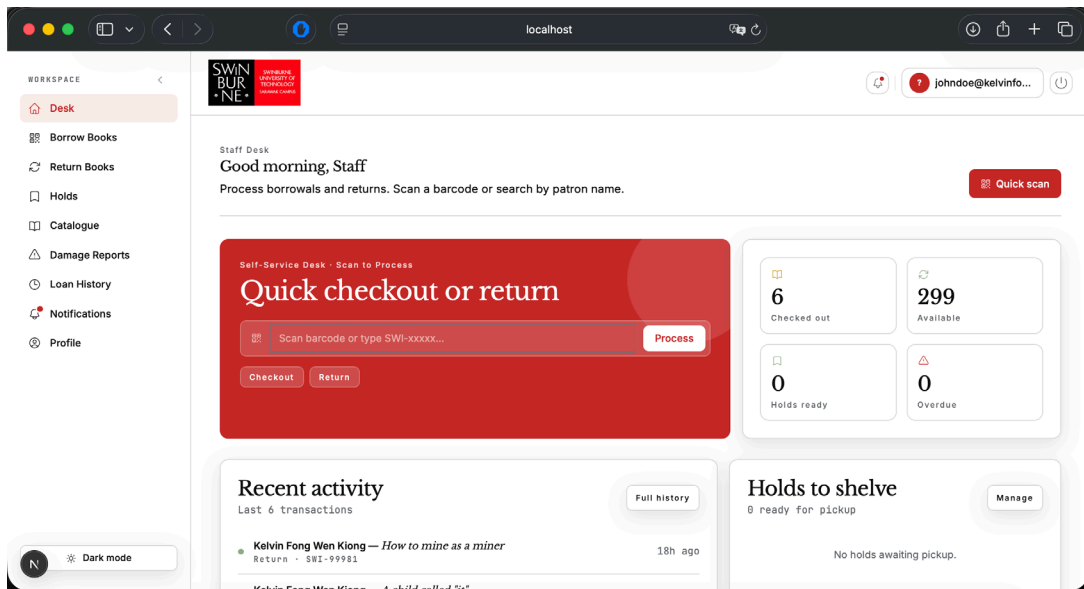


Figure 39: The staff Desk view showing the quick scan input, stats cards, recent activity table, and holds to shelve.

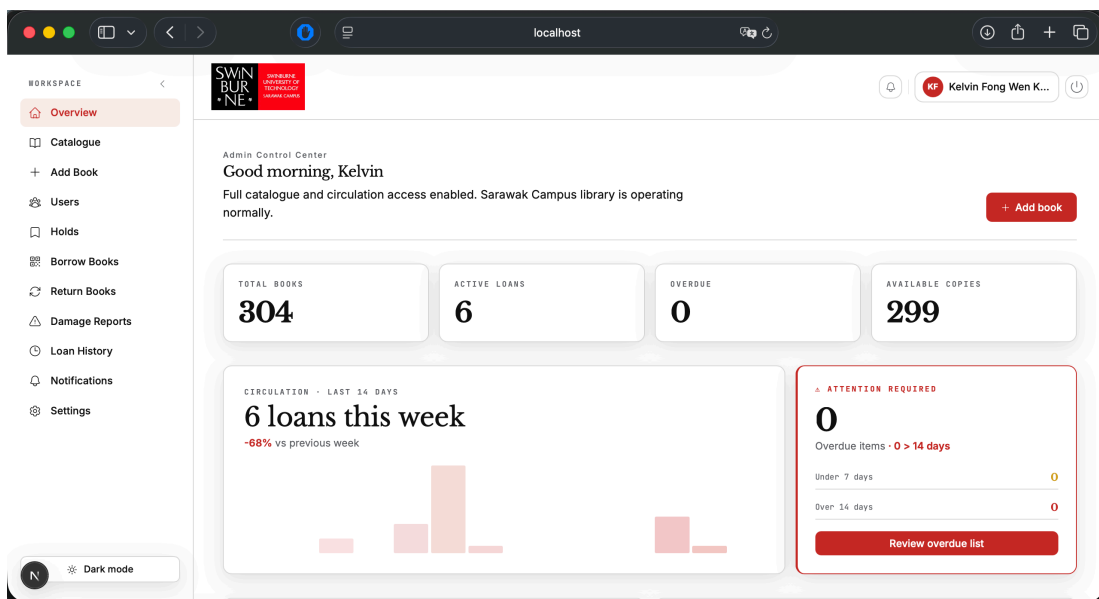


Figure 40: The admin Overview page showing the KPI cards, the fourteen day circulation chart, and the most borrowed titles list.

The table below summarises which capabilities are visible to each role, so you can see at a glance how the views differ.

Capability	Student	Staff	Admin
Browse catalogue and book details	Yes	Yes	Yes
Borrow, return, and renew books	Yes	Yes	Yes
Place and cancel a hold	Yes	Yes	Yes
Submit a damage report	Yes	Yes	Yes
Use the AI Reading Assistant	Yes	No	No
Use the Learning Hub	Yes	No	No
Receive personalised recommendations	Yes	No	No
Triage damage reports	No	Yes	Yes
Search all user loan history	No	Yes	Yes
Manage the holds queue	No	Yes	Yes
Add, edit, or delete books	No	No	Yes
Manage user accounts and roles	No	No	Yes
Export an overdue loans report	No	No	Yes
View system wide KPI dashboard	No	No	Yes

Light and dark mode:

The system supports both light and dark mode through a theme toggle that appears at the top of the student dashboard and on the profile page. Tapping the toggle switches between the two modes immediately and the choice is remembered, so the next time you open the system it will use whichever mode you last selected. Light mode uses a warm cream background with dark text and the Swinburne red accent. Dark mode uses a deep charcoal background with light text and the same red accent. Both modes have been tuned so that buttons, status pills, and text remain readable in either setting.

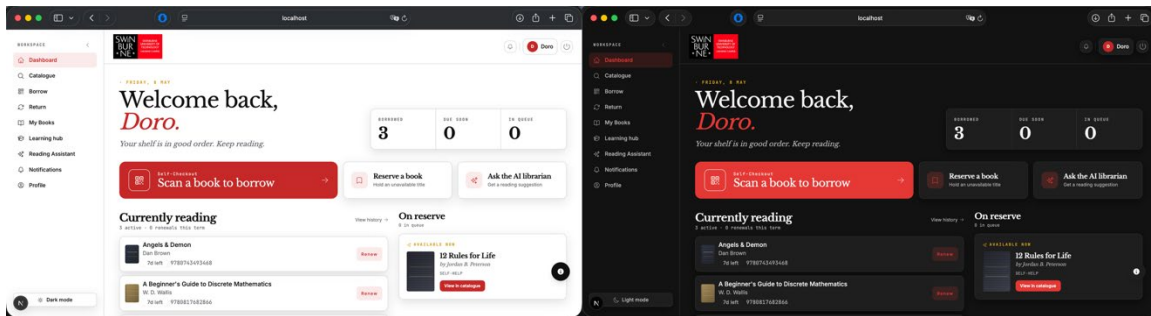


Figure 41: Side by side comparison of light mode (left) and dark mode (right) on the same page.

Status pills and colour coding:

Across the system, short rounded labels called status pills are used to communicate the current state of an item at a glance. Each colour has a consistent meaning so once you learn the colour key, you can read any page faster. The table below summarises what each colour means in context.

Colour	Meaning	Where you commonly see it
Green	Available, returned, success, hold ready for pickup	Catalogue card "Available" badge, return confirmation, holds queue ready state
Red	Overdue, blocked action, lost copy, attention required	Days overdue badge on My Books, lost severity on damage reports, blocked checkout warnings
Amber or orange	Warning, damaged copy, demo mode, soft warning before action	Damaged severity on damage reports, LinkedIn Learning demo mode banner, staff override warnings on checkout
Gold	Active loan in good standing, borrowed but not overdue	Loan history table, recent activity rows
Teal	Needs inspection, awaiting human review	Damage report severity for ambiguous cases
Grey or outline	Disabled, neutral, or inactive	Disabled buttons such as Renew when not eligible, inactive filter pills

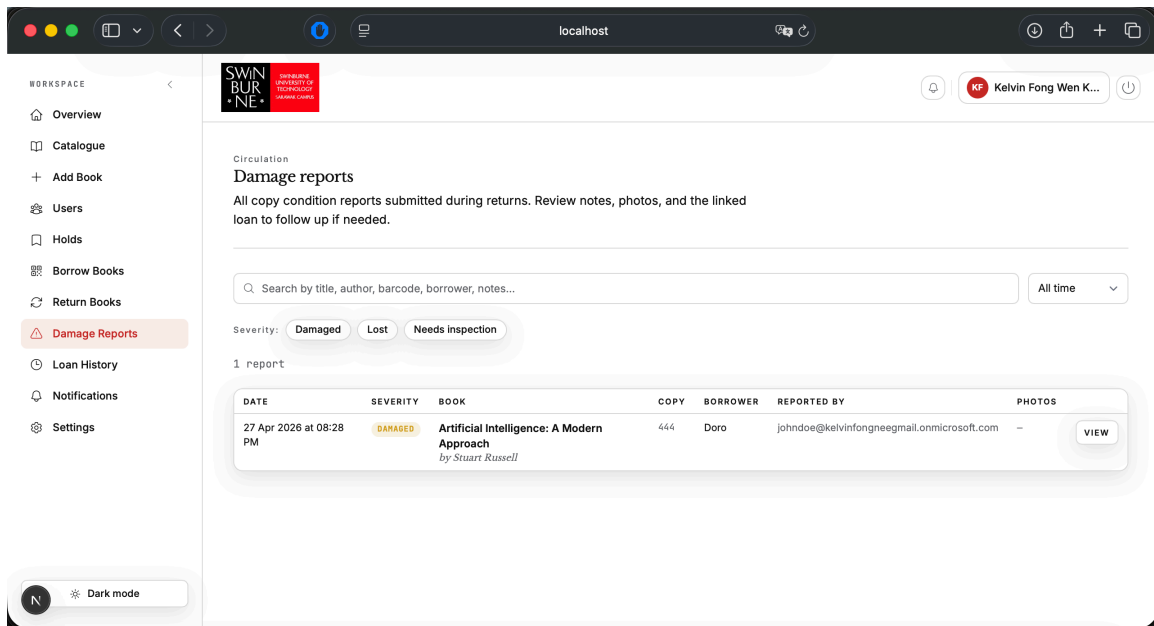


Figure 42: Examples of the main status pill colours as they appear in the system.

Empty states:

When a list page has nothing to show, the system displays a friendly empty state message instead of leaving the area blank. These are not errors. They simply mean there is no data yet for the filter or page you are looking at. For example, the My Books Current tab shows "No books borrowed" with a link to the catalogue when you have no active loans. The Notifications page shows "All caught up" when you have read every notification, and "No notifications yet" when you have never received one. The admin Overdue page shows "All loans are on time. Nice." when there are no overdue items at all. Each empty state is designed to confirm that the system is working correctly and to suggest a next step where one is appropriate.

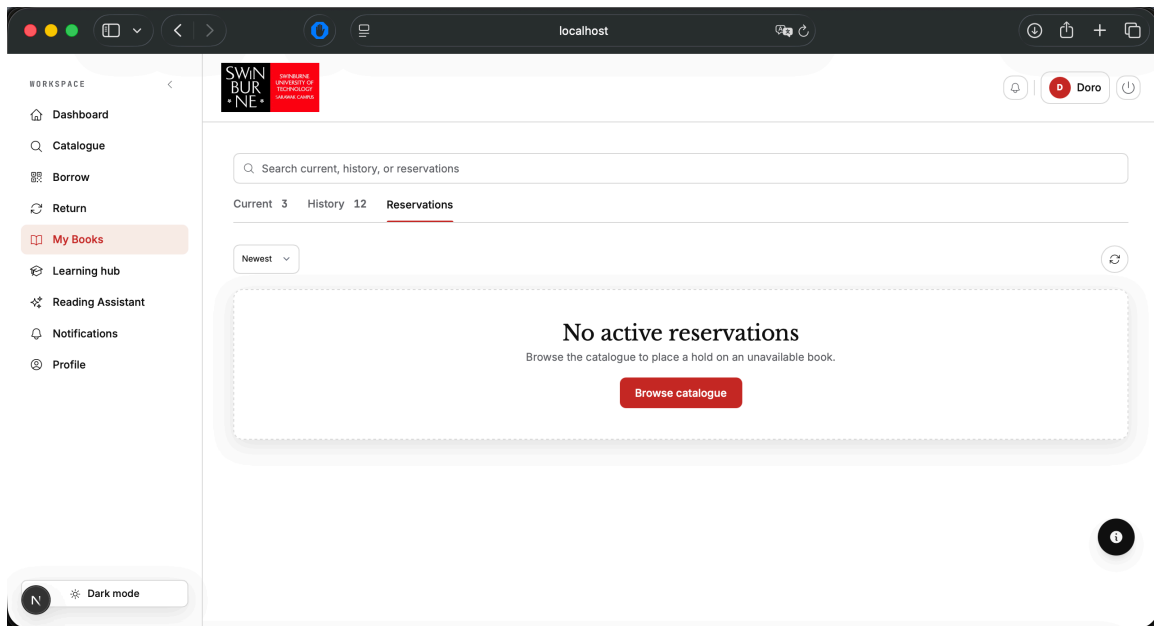


Figure 43: Example of an empty state on the My Books page when no books are currently reserved.

Loading skeletons:

While a page is fetching its data from the server, the system shows shimmer placeholders called loading skeletons in place of the actual content. These look like greyed out rectangles in the rough shape of the content that is about to load, with a soft animated shimmer running across them. Loading skeletons appear briefly on most pages, especially the catalogue, the admin overview, and the loan history page. They are not errors and there is nothing for you to do other than wait a moment for the real content to appear. If a skeleton stays on screen for more than a few seconds, it usually means the network is slow or the server is unreachable, in which case refreshing the page is the right next step.

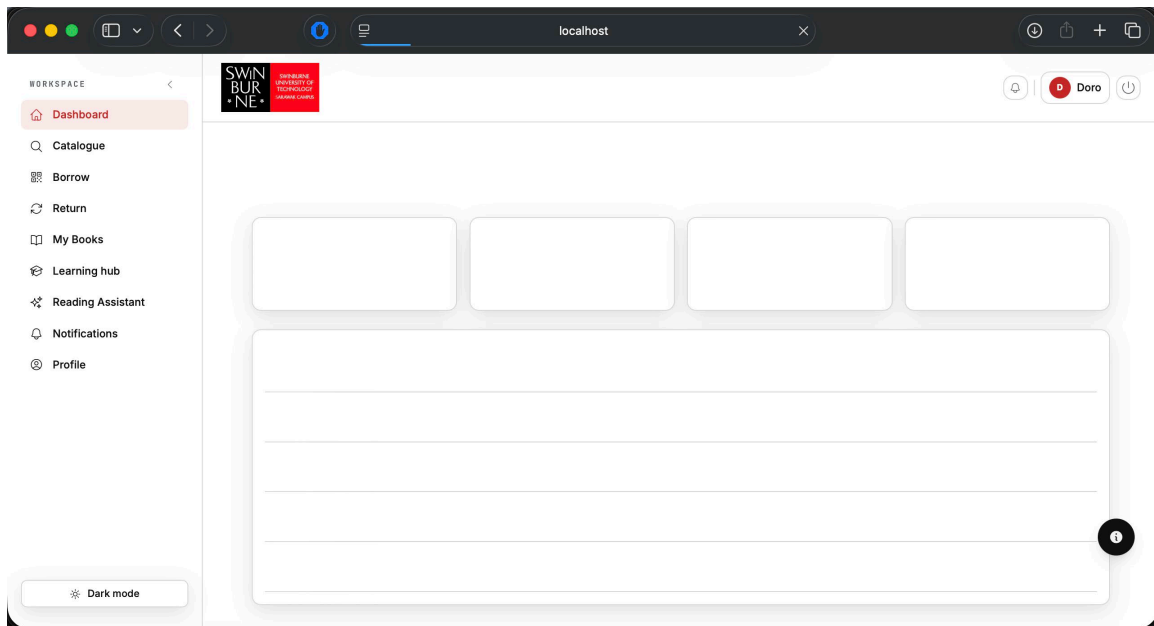


Figure 44: Example of the loading skeletons.

2.3.5 Features Guide

This sub-section walks through how to use each feature of the system from start to finish. Features are grouped by the role that uses them: student facing features come first, followed by staff specific tools, and finally admin specific tools. The table below maps every feature against the roles that can use it, so you can quickly identify what is relevant to you before reading further.

Feature	Student	Staff	Admin
Borrowing a book	Yes	Yes	Yes
Returning a book	Yes	Yes	Yes
Renewing a loan	Yes	Yes	Yes
Placing a hold	Yes	Yes	Yes
Cancelling a hold	Yes	Yes	Yes
Using the AI Reading Assistant	Yes	No	No
Using the Learning Hub	Yes	No	No
Managing your notifications	Yes	Yes	Yes
Updating your profile	Yes	Yes	Yes
Triaging damage reports	No	Yes	Yes
Managing the holds queue	No	Yes	Yes
Searching loan history and exporting CSV	No	Yes	Yes
Adding a new book	No	Yes	Yes
Editing existing books	No	Yes	Yes
Managing user accounts	No	No	Yes
Sending overdue reminders	No	Yes	Yes
Viewing the admin overview	No	No	Yes

Student features

Borrowing a book

To borrow a book, the easiest path is to open the Borrow page from your sidebar. The Borrow page opens a barcode scanner that uses your phone or computer camera. Find the book you want, hold its barcode flat in front of the camera, and the system will detect it automatically. Once detected, a confirmation card appears showing the book title, the copy you are about to borrow, and your due date which is set thirty days from today. Click Confirm loan to record the loan. The system runs several checks before the loan is recorded. It confirms that you have not already reached the three book limit, that you have no overdue loans blocking new borrowing, and that no other student has placed a hold on this title. If any of these checks fail, you will see an explanatory message and the loan will not go through. After a successful checkout, you are taken to a receipt screen where you can choose to borrow another book or jump to your loans list.

You can also start the borrow flow from a book's catalogue detail page. If a book is currently available, the detail page shows a Borrow this book button which takes you straight to the checkout page with that book already selected.

Returning a book

To return a book, open the Return page from your sidebar. Like the Borrow page, the Return page uses the camera scanner. Scan the barcode on the book you are returning and the system will look up the matching loan and show a confirmation. If you have multiple books to return at once, tick the "Continue scanning (bulk mode)" checkbox before you start scanning. In bulk mode, after each successful return the form clears and is ready for the next scan, and a strip of recent receipts appears showing your last three returns so you can verify them at a glance. If the book has any visible damage or is missing pages, click "Report damage or missing" before confirming the return to attach a damage report to the return. Damage reports are queued for staff to review, and the return itself still completes normally on your end.

If the camera scanner cannot read the barcode, you can switch to "Find by borrower" mode, which is the second tab on the Return page. In this mode, you select your name from a dropdown and then choose which of your active loans you are returning.

Renewing a loan

To renew an active loan, open the My Books page from your sidebar and stay on the Current tab. Each active loan card has a Renew button. Clicking Renew extends the loan by another thirty days from today. There are three rules around renewal. You cannot renew an overdue loan, you cannot renew the same book more than twice in a row, and you cannot renew if another student has placed a hold on the same book. If any of these conditions apply, the Renew button is disabled and hovering over it shows a tooltip explaining why.

Placing a hold

If a book you want is currently on loan, you can place a hold so that you are next in line when it becomes available. Open the book's detail page from the catalogue. When all copies are unavailable, the detail page shows a Place a hold button instead of Borrow this book. Click it to add yourself to the queue. Your queue position is shown straight away. When the book is returned and a copy becomes available, a staff member marks the hold as ready and you receive a notification telling you to collect the book within three days. After three days, the hold expires and the book moves to the next person in the queue.

Cancelling a hold

If you no longer need a book you placed on hold, you can cancel the hold from the My Books page. Open the Reservations tab and find the hold you want to cancel. Each hold card has a Cancel hold button. Clicking it shows a confirmation modal explaining that you will lose your place in the queue. Click "Yes, cancel hold" to confirm or "Keep reservation" to back out.

Using the AI Reading Assistant

The AI Reading Assistant is a chat interface that helps you find books and get answers to library related questions. To use it, open Reading Assistant from your sidebar. The chat will load with any previous conversation you had with the assistant. Above the message input, six suggested prompts are always shown so you have a starting point if you do not know what to ask. The suggestions cover both how-to questions ("How do I renew a loan?", "How do I place a hold?", "What happens if a book is overdue?") and recommendation requests ("Find me a fantasy novel", "Recommend something light for the weekend", "Computer science textbooks I can borrow now"). Tap any suggestion to drop the prompt into the input box where you can edit it before sending. To send a message, type it into the box and click the Send button. The assistant takes a few seconds to respond because it queries Google Gemini in the background and waits for a complete answer. The assistant has access to your profile, your active loans, and your recent return history, which is why its recommendations feel personal. To start a fresh conversation, click the Clear Chat button. You will be asked to confirm because clearing the chat permanently deletes all your previous messages.

Using the Learning Hub

The Learning Hub is a curated resource page with three tabs: YouTube, Community, and News. The YouTube tab shows tutorial videos that are relevant to your faculty and interests, drawn from the YouTube Data API. You can use the search box to look for specific topics, or use the quick topic buttons (Engineering, Computer Science, Cyber Security, Data Science, Business and Finance, Design) to jump straight to a category. The Community tab shows recent articles and discussions from developer focused communities. The News tab shows learning related news pulled from RSS feeds. If you see a small amber banner saying "Demo mode — showing sample video data", that means the system administrator has not configured a live YouTube API key and you are seeing example data only.

Managing your notifications

To check your notifications, open the Notifications page from your sidebar. The page lists every notification you have received, with the most recent first. Each row shows the notification type icon, the message, and how long ago it was received. Click anywhere on a row to expand it and see the full details. Clicking on an unread notification automatically marks it as read. To mark a single notification as read without expanding it, expand the row and click the Mark as read button inside. To mark every unread notification as read at once, click the Mark all read button at the top of the page. You can also filter the list (All, Read, Unread), search across messages, and sort by date or title. There is currently no way to delete individual notifications. The red dot on the Notifications icon in the sidebar will clear once you read or mark all your notifications as read.

Updating your profile

To update your profile, open the Profile page from your sidebar. The page is divided into two main forms: an avatar form for uploading or replacing your profile photo, and a details form for editing your display name, username, phone number, preferred language, faculty, department, and bio.

Change any field and click Save Changes. The button label changes to "Saving..." while the system processes your update, then switches back once complete. If anything goes wrong, an error message appears in red below the form. The avatar uploader accepts JPEG, JPG, PNG, GIF, and WEBP images up to 2 MB in size. Once uploaded, your avatar appears in the sidebar and on any pages that show your name.

Staff features

The features in this group are available to both staff and admin users. Anything an admin can do here, a staff member can also do.

Triaging damage reports

When a student returns a book and reports damage, the report appears in the Damage Reports queue. Open the Damage Reports page from your sidebar to see the queue. Each row shows the date the report was filed, a severity pill (Damaged, Lost, or Needs inspection), the book title and author, the copy barcode, the borrower's name, who reported it, and how many photos are attached. To review a report in detail, click the View button on its row. A modal opens showing the full report, including the borrower details, the dates, any notes the student wrote, and any photos they attached. Click on a photo thumbnail to open it in a new tab at full size. After reviewing, you can mark the report as resolved or escalate it. The page also has a search bar and filter pills for severity (Damaged, Lost, Needs inspection) and date range (All time, Last 7 days, Last 30 days, Last 90 days), which you can use to narrow down the queue when it gets long.

Managing the holds queue

The Holds page (under Holds in your sidebar) shows every active hold across the system. Use the filter pills at the top (All, Ready, Queued) to focus on whichever subset you need. The KPI cards at the top show total holds, ready holds, and queued holds. The main table lists each hold with the book, the patron, the status, the date the hold was placed, and the pickup deadline if the hold is ready. When a book is returned and a copy becomes available, the first hold in the queue for that book gets a Mark ready button. Click it to mark the hold as ready, which sets a three day pickup window and sends an automatic notification to the patron. The system will not let you mark a hold as ready if no copies are available. The button shows as Unavailable with a tooltip explaining why. To cancel a hold (for example, if a patron asks to be removed from the queue), click the Cancel button on the row. If there are holds that cannot be fulfilled because no copies exist anywhere in the system, an Attention required alert appears at the top of the page listing them, with a Cancel hold button per row.

Searching loan history and exporting CSV

The Loan History page lets you search every loan that has ever been recorded in the system. Open it from the Loan History link in your sidebar. The page has three search fields (borrower name or student ID, book title or author or barcode, handler staff name) and two filter rows (status pills All, Active, Returned, Overdue, plus range pills 30 days, 6 months, Semester, All time). Set any combination of these and the table updates to show matching loans. The summary line above the

table shows the total count along with breakdowns by status. To download the current filtered view as a CSV file, click the Export CSV link. The CSV will contain only the rows that match your current filters. If the result set is large, use the Previous and Next pagination buttons at the bottom to move through pages.

Adding a new book

To add a new book to the catalogue, open Add Book from your sidebar. The page has a multi part form. The first section is an ISBN lookup which can save you typing. Enter or scan an ISBN and click Lookup to check whether the book is already in the catalogue. If it is, the page shows an alert with a button to jump straight to that book or to continue creating a new entry anyway. If the book is not already in the catalogue, fill in the rest of the form: Title, Author, ISBN, Publisher, Publication year, and Classification (such as a Dewey Decimal code). Add a cover image URL if you have one. Pick the Category from the dropdown (Computer Science, Business, Art and Design, or Engineering). Add up to six tags by typing each tag and pressing Enter. Set the Number of copies you want to create, which must be between 1 and 20. The system will automatically generate copy barcodes in the SWI prefix, and a preview shows the range you are about to create. When everything looks right, click the Create button. A confirmation modal asks you to verify the title and the number of copies before the book is actually saved.

Editing existing books

To edit a book that already exists, navigate to the book in the catalogue and click on it. As a staff or admin user, the book detail page includes an edit option that opens a simplified version of the Add Book form. You can update the title, author, ISBN, category, classification, publication year, publisher, and cover image. ISBN lookup, copies counter, and tag editing are not shown in edit mode because those only apply when creating a new book. Click Save changes to apply your edits.

Sending overdue reminders

The Overdue page (Overdue in your sidebar) lists every loan that is currently past its due date. The page is structured similarly to the Loan History page: filter pills for the overdue bucket (All overdue, 1 to 7 days, 8 to 30 days, 30 plus days), a search field, and a table showing book, borrower, dates, and days overdue. Each row has a Send reminder button. Clicking it sends an in app notification to the borrower reminding them about the loan. The system prevents you from sending more than one reminder for the same loan within a 24 hour window. If you try to send another reminder too soon, you will see a message saying a reminder was already sent recently. The Reminder column on each row shows when the last reminder was sent and by whom. To download the overdue list as a CSV file, click the Export CSV link.

Admin features

The features in this group are available to admin users only. Staff members will not see these pages in their sidebar and cannot reach them by typing the URL directly.

Managing user accounts

The Users page (Users in your admin sidebar) shows every account in the system. Use the search bar to find a user by any field. Each row shows the email, full name, student ID, and role, with action buttons. To add a new staff or admin account, scroll to the top of the page and use the Add staff member form. Enter the email address (which must end in swinburne.edu.my for staff or admin accounts), the full name, and the role from the dropdown. Click Add User and confirm in the modal that appears. To edit a user, click on their row to make the fields editable. Update what you need and click Save. To delete a user, click Delete on the row, then confirm in the warning modal. Deletion is permanent and cannot be undone. To see or edit additional profile fields beyond the basics, click the Details toggle on the row to expand the full profile editor, which includes display name, username, phone, preferred language, faculty, department, intake year, visibility, bio, and links.

Viewing the admin overview

The Admin Overview page is your default landing page after sign in and gives you a quick read of the system's overall health. Four KPI cards at the top show the total number of books, the number of active loans, the number of overdue loans (highlighted in red when greater than zero), and the number of available copies. Below the cards, a circulation chart visualises the last fourteen days of loan activity, with a trend indicator showing the percentage change compared to the previous week. An Attention required panel appears when there are overdue loans, with a button to jump straight into the Overdue page. Two side by side panels show the most recent activity (returns, borrows, holds) and the top five most borrowed titles over the last thirty days. There is a Full history button at the bottom of the recent activity panel that takes you straight into the staff Loan History page where you can dig deeper.

2.3.6 Troubleshooting

This sub-section covers common issues you may run into and what to do about them. The issues are grouped by feature area so you can jump to the section that matches what you were doing when the problem appeared. As a general rule, when something does not work, the first thing to try is refreshing the page. If that does not help, sign out and sign back in. Many issues are resolved by a fresh session before any deeper troubleshooting is needed.

Sign in and access:

Symptom or message	What to do
Microsoft authentication fails or shows an "Access denied" error	Confirm you are using your Swinburne Microsoft 365 account and not a personal Outlook, Gmail, or other email provider. The system only accepts Swinburne tenant accounts.
You signed in but only see the student view, even though you should be staff or admin	Role changes can only be done by an administrator from the Users page. Contact your system administrator to have your role updated. There is no self service option to upgrade your own role.
You see the Microsoft account picker every time, even though you are already signed in to Microsoft elsewhere	This is intentional. The system always asks you to choose an account so you can switch between accounts on shared devices. Just pick your Swinburne account each time.
Buttons stop responding after a long session	Your session may have expired. Sign out using the Sign Out button in the sidebar, then sign in again to refresh your session.

Camera scanner:

Symptom or message	What to do
"Camera access is not supported in this browser."	Your browser does not support camera access. Update to a recent version of Chrome, Edge, Firefox, or Safari, or try a different device.
"Camera permission denied. Allow access and try again."	Open your browser's site permissions settings, allow camera access for the system URL, then reload the page. The exact location of this setting depends on your browser.
"Unable to access the camera. Check permissions or try another device."	Another app may be using the camera. Close any apps that use the camera, such as video call apps, and try again.
Scanner camera turns on but does not detect the barcode	Make sure the barcode is in good lighting, held flat, and at a steady distance of around ten to twenty centimetres from the camera. If the barcode is damaged or dirty, use the Manual entry input and type the code by hand instead.
Camera preview is frozen or not updating	Open the scanner Settings drawer and click the Refresh button to restart the camera preview.
Wrong camera is in use, such as the front camera instead of the rear camera	Click the Camera switch icon in the scanner controls to switch between cameras. The icon only appears when more than one camera is detected.
"No available book matched that code."	The barcode does not match any book in the catalogue. Double check the code or try the manual entry input. If the book exists but all copies are on loan, you can place a hold from the book detail page instead.

Borrowing, returning, renewing, and holds:

Symptom or message	What to do
"Loan limit reached (3/3). Return a book before borrowing another."	You already have three active loans, which is the maximum allowed for students. Return at least one book before borrowing another.
"You have an overdue book. Please return it before borrowing another."	Return your overdue book first. Until that loan is closed, the system will not let you borrow anything new.
"This title is reserved for another patron."	Another student has placed a hold on this book and is next in the queue. You cannot borrow it directly. Place a hold yourself if you want to join the queue, or pick a different title.
"That copy is not currently available."	Someone else may have just borrowed the copy you tried to scan. Refresh the page and check the catalogue for other available copies.
"No active loan matches that reference."	The barcode or loan ID you entered does not match any active loan. Make sure the book is actually checked out and that you are using the correct copy barcode.
"Multiple active loans found. Scan the barcode or enter the specific loan ID."	The borrower has more than one active loan, so the system cannot tell which one you want to return. Switch to scanning the specific copy barcode instead of searching by borrower name.
Renew button shows "Cannot renew an overdue loan"	Overdue loans cannot be renewed. Return the book and borrow it again if no one else has placed a hold.
Renew button shows "Maximum renewals reached (2/2)"	You have already renewed this loan twice in a row, which is the maximum. Return the book, then borrow it again later if it is still available.
Renew button shows "Cannot renew, another student has a hold on this book"	Someone else is waiting for this book. Return it on time so the next person can collect it. You can place your own hold afterwards if you still need the book.
The damage section on the return page shows a flagged chip you did not intend to add	Click the small × button on the chip to clear the damage flag, then try the return again.
Hold expired before the patron collected the book	Holds expire after three days if not collected. The patron will need to place a new hold if they still want the book.

AI Reading Assistant and Learning Hub:

Symptom or message	What to do
"Sorry — I couldn't reach the AI just now. Please try again in a moment, or ask a librarian directly."	The AI service is temporarily unavailable. Wait a few seconds and resend your message. If it keeps failing, ask a librarian directly.
The assistant takes a long time to reply	Responses normally take a few seconds because the system waits for a complete answer from Google Gemini. The first message in a session is usually the slowest. Wait for the loading indicator to finish instead of resending the same message.
The assistant suggests a book that does not appear in the catalogue	This is an error and should be reported. The assistant is supposed to recommend only books that exist in the library catalogue. Let a librarian know which book it suggested.
You want to delete a single past message in the chat	The Reading Assistant does not support deleting individual messages. Use the Clear Chat button to wipe the entire conversation, or simply leave the past messages and continue with a new question below.
"Demo mode — showing sample video data" appears on the Learning Hub	The system administrator has not configured a live YouTube API key, so the Learning Hub is showing example data only. Contact the administrator if you need live data.
The Learning Hub YouTube tab is empty or has unusually few results	The daily YouTube API quota may have been reached. Try again the next day, or use the Community or News tabs in the meantime.

Profile and notifications:

Symptom or message	What to do
Avatar upload fails	Check that your image file is under 2 MB and that its format is JPEG, JPG, PNG, GIF, or WEBP. Other formats and larger files are rejected.
Phone number field rejects your input	The phone field accepts only digits, spaces, plus signs, and dashes. Remove any other characters. The maximum length is 20 characters.
A profile field will not save and the page shows no change	You may have hit the maximum length for that field. Username and preferred language are limited to 50 characters, faculty and department to 100 characters, and bio to 500 characters.
Red dot on the Notifications icon will not clear	Open the Notifications page and either click each unread notification (which marks it as read automatically) or click the Mark all read button at the top of the page.
You want to delete a notification	The system does not currently support deleting individual notifications. You can mark them as read so they disappear from the unread filter, but they remain in your full notification history.

Staff and admin tools:

Symptom or message	What to do
Mark ready button on a hold shows "Unavailable" with a tooltip	No copies of the book are currently available, so the hold cannot be marked ready yet. Wait until a copy is returned and the system updates its status to available.
"A reminder was already sent within the last 24 hours."	The system prevents sending more than one reminder for the same loan within 24 hours to avoid spamming the borrower. Wait until the cooldown period ends and try again.
"This loan is no longer overdue." (when sending a reminder)	The loan has already been returned or its due date has been extended. Refresh the Overdue page to update the list.
Trying to add a staff or admin user fails with "Staff and admin accounts must use a swinburne.edu.my Outlook email address."	The email you entered is not a Swinburne address. Use the staff member's official Swinburne email when creating a staff or admin account.
"User added successfully." but the new user cannot sign in	The user record exists but the user must still sign in through Microsoft using their Swinburne account on their own device. Their first sign in will link the Microsoft identity to the record you created.

Connection and general issues:

Symptom or message	What to do
Pages take a long time to load on the first visit of the day	The first request after a long idle period or after a deployment can be slow because the server has to warm up. Wait a moment and the page should load normally. Subsequent requests will be much faster.
Pages fail to load entirely	Check your internet connection first. The system requires an active connection at all times. If your connection is fine, the server may be temporarily down. Try again in a few minutes.
A loading skeleton stays on screen for more than several seconds	The page is taking longer than usual to fetch data. Refresh the page. If the issue persists, the server may be slow or unreachable.
The system shows old data even after you made a change	Some pages do not update in real time. Refresh the page manually to see the latest data.

If you cannot find your issue listed above, and the standard fixes (refreshing the page, signing out and back in, and checking your connection) do not help, contact the library directly. Library staff can investigate further or escalate the issue to the system administrator on your behalf if it turns out to be a system level problem rather than a user side one.

2.4 SYSTEM INSTALLATION / DEPLOYMENT MANUAL

2.4.1 System Requirements

This sub-section lists the system level requirements for installing, building, and running the Library Self-Checkout System. The requirements split into three groups: the development machine running the code, the external services the application depends on, and the network conditions needed for it to function correctly. Meeting all three groups is necessary for a successful deployment.

Development and runtime environment:

The application is built with Next.js 15 and uses several modern JavaScript features that require an up to date Node.js runtime. The continuous integration workflow currently builds and tests the project against Node.js 22.19.0 with pnpm 10.17.1, so we recommend matching these versions exactly. Earlier Node.js versions may work but have not been verified, and using a different package manager such as npm or yarn is not supported because the project relies on the pnpm flat tree configuration declared in the .npmrc file.

The operating system can be macOS, Linux, or Windows. Linux is the most thoroughly tested environment because the GitHub Actions runners use Ubuntu, but macOS and Windows are both fine for local development. On Windows, we recommend using Windows Subsystem for Linux (WSL) because some Node.js native modules can be difficult to install on plain Windows.

Hardware requirements are modest. Any machine capable of running a modern web browser and a Node.js development server is sufficient. In practice, around 4 GB of RAM and 5 GB of free disk space are enough for development work. For production deployment, the system runs on Vercel's serverless infrastructure which scales automatically based on traffic.

External service accounts:

Several third party services are integral to the system and need to be set up before the application can run end to end. The following table summarises each service, what it is used for, and whether it is required for the system to start.

Service	Purpose	Required to start?
Supabase	Source of truth database, authentication record storage, file storage for avatars and damage report photos	Yes
Microsoft Azure AD (Entra ID)	Single sign on identity provider tied to the Swinburne Microsoft 365 tenant	Yes
Google AI Studio	Gemini API key for the AI Reading Assistant and personalised recommendations	Optional, AI features degrade if missing
Google Cloud Console	YouTube Data API v3 key for the Learning Hub video tab	Optional, falls back to demo data if missing

Vercel (or any Node.js compatible host)	Production hosting platform with serverless function support	Yes for production, optional for local development
GitHub	Source code hosting and continuous integration	Yes
Library SIP2 service	Connection to the existing Integrated Library System for circulation sync	Optional, falls back to best effort logging if missing

Free tier accounts are sufficient for development and small scale testing on every service except SIP2, which is a campus library service rather than a public API. For production use, the team should review each service's pricing and quota limits ahead of time, especially for the Gemini API and the YouTube Data API since they have daily request limits.

Network requirements:

The application makes outbound HTTPS calls to Supabase, the Microsoft authentication endpoints, the Gemini API, the YouTube Data API, and the SIP2 service when those features are in use. Standard outbound port 443 must be reachable from both the development machine and the production host. There are no inbound port requirements beyond the default web port. For phone barcode scanning during development, the development server is bound to 0.0.0.0 so that other devices on the same local area network can reach it. This means the development machine and any phone used for testing the camera scanner need to be on the same network.

Dependencies

This sub-section covers the dependencies that the project relies on, divided into three groups: code dependencies (the npm packages installed by pnpm), development tools (the command line tools a developer or system administrator needs on their machine), and external services (the third party platforms the application connects to at runtime). The first and third groups are documented elsewhere in this report, so the focus here is on practical setup rather than a duplicate inventory.

Code dependencies:

The complete list of code dependencies, with version numbers and the role each one plays in the project, is presented in the technology stack table in section 2.2.3 of this report. All code dependencies are managed by pnpm and are installed automatically when you run pnpm install from the project root. The dependency tree is locked by the pnpm-lock.yaml file committed in the repository, so every developer and every production deployment use the same exact versions. The lock file should never be deleted or hand edited. If a dependency needs to be updated, use the appropriate pnpm command such as pnpm update and commit the updated lock file alongside the package.json change.

Development tools:

In addition to Node.js and pnpm covered in section 2.4.1, the developer or system administrator setting up the project needs the following tools on their machine.

Tool	Purpose	Required?
Git	Cloning the repository and managing source control	Yes
pnpm 10.17.1	Package manager. Other package managers such as npm or yarn are not supported by this project	Yes
Supabase CLI	Running database migrations and managing the Supabase project from the command line	Recommended
Vercel CLI	Linking the local project to the Vercel deployment and syncing environment variables	Recommended
A code editor such as Visual Studio Code	Editing the codebase	Recommended

The Supabase and Vercel command line tools are technically optional because the same actions can be performed through their respective web dashboards. However, for repeatable operations such as applying migrations or pulling the production environment variables to a local file, the CLI tools are significantly faster and reduce the chance of human error.

External services:

The full list of external services that the application connects to is documented in the service accounts table in section 2.4.1 of this report. To summarise: the system requires Supabase and Microsoft Azure AD to start, optionally connects to Google Gemini and the YouTube Data API for AI and Learning Hub features, and optionally connects to a SIP2 service for legacy library system synchronisation. Each of these requires an account with the relevant provider and a set of credentials, which are configured through the environment variables documented in section 2.4.5.

Build Instructions

This sub-section walks through the steps required to set up a working copy of the Library Self-Checkout System from scratch, configure all required services, run the application locally, and deploy it to production. The instructions assume familiarity with the command line, basic Git workflows, and standard web application deployment concepts. Read through the entire sub-section once before starting so you understand the overall flow, then return to step one and follow the steps in order.

The pnpm scripts referenced throughout this sub-section are summarised in the table below for quick reference.

Script	Command	When to use it
dev	pnpm dev	Local development with hot reload, bound to 0.0.0.0 so phones on the same network can reach the dev server for camera scanner testing
dev:webpack	pnpm dev:webpack	Fallback development server using Webpack rather than Turbopack, useful if Turbopack misbehaves
build	pnpm build	Production build, generates the optimised .next directory
start	pnpm start	Run the production build (after pnpm build has completed)
test	pnpm test	Run the full Jest test suite once
test:watch	pnpm test:watch	Run Jest in watch mode for test driven development
clean	pnpm clean	Wipe the .next build cache before a clean rebuild

Step 1: Prerequisites

Before you begin, make sure your machine has the following tools installed and reachable from the command line. The exact versions are listed in section 2.4.1.

node --version	should report 22.19.0 or compatible
pnpm --version	should report 10.17.1 or compatible
git --version	any modern version is fine

If any of these commands fail or report an unexpected version, install or update the missing tool before continuing.

Step 2: Clone the repository and install dependencies

Clone the repository from GitHub and install the dependencies using pnpm. The dependency installation may take several minutes the first time because the project has a large dependency tree.

```
git clone https://github.com/Kidemi04/Library_Self-Checkout_System.git
cd Library_Self-Checkout_System
pnpm install
```

When pnpm install completes, you should see a success message and a node_modules folder in the project root.

Step 3: Set up the Supabase project

Create a new Supabase project at <https://supabase.com> if you do not already have one. Once the project is created, navigate to Settings → API and note the following values, which you will need in Step 7:

Project URL
anon public key
service_role secret key

The damage-reports and avatars storage buckets will be created automatically by the schema script in Step 4, so no manual bucket setup is needed at this point.

Step 4: Apply the database schema

The project includes a complete schema script at `supabase/schema.sql` that sets up every table, view, stored procedure, extension, and storage bucket the application needs. Open the file, copy the entire contents, paste it into the Supabase SQL Editor on your new project, and click Run. The script is idempotent where possible and reports any errors clearly in the editor output.

If the storage section of the script fails on your plan (some Supabase plans restrict direct INSERT INTO storage.buckets through SQL), create the avatars bucket (Public) and the damage-reports bucket (Private) manually through the Storage dashboard, then re-run only the storage policy section of the script so the access policies are applied to your buckets.

For reference, the `supabase/migrations/` directory contains the 10 incremental migration files that were used during development to track schema changes over time. These are useful for understanding how the schema evolved but are not needed for a fresh deployment, since the `schema.sql` file already includes everything those migrations applied. The migration files and what each one sets up are listed below.

#	File	What it sets up
1	<code>20260326_notifications.sql</code>	Notifications and NotificationReads tables
2	<code>20260326_notifications_user.sql</code>	Adds <code>target_user_id</code> column and expands notification types
3	<code>20260423_damage_reports.sql</code>	DamageReports table
4	<code>20260423_extend_book_categories.sql</code>	Adds Psychology, Self help, History to the book category list
5	<code>20260428_loans_reminder_tracking.sql</code>	Reminder tracking columns on Loans
6	<code>20260428_search_books_rpc.sql</code>	<code>search_books</code> stored procedure with trigram indexes

7	20260430_avatars_bucket.sql	avatars storage bucket and access policies
8	20260507_fix_notification_reads_pk.sql	Restores composite primary key on NotificationReads
9	20260507_general_chat_history.sql	GeneralChatHistory table
10	20260508_notification_flags.sql	NotificationFlags table for per user starring

Step 5: Configure Microsoft Azure AD

The application uses Microsoft Azure AD as its identity provider. In the Azure Portal, create a new App Registration for the project. Add a Redirect URI of the form <https://your-domain/api/auth/callback/azure-ad> for production, and <http://localhost:3000/api/auth/callback/azure-ad> for local development. After registration, note the following values:

Application (client) ID
Directory (tenant) ID
A client secret created from Certificates & secrets

These three values are required environment variables. The application will not start without them.

Step 6: Get optional API keys

Two more API keys are useful but not strictly required.

The Google Gemini API key powers the AI Reading Assistant. Create one at <https://aistudio.google.com/app/apikey>. If this key is missing, the AI features will degrade but the rest of the system will continue to work.

The YouTube Data API v3 key powers the Learning Hub. Create one in the Google Cloud Console under APIs & Services → Credentials. If this key is missing, the Learning Hub will display sample data with a Demo mode banner.

Step 7: Create the local environment file

Copy the .env.example file at the project root to a new file called .env.local in the same directory. Open .env.local and fill in the values you collected in Steps 3 to 6.

```
cp .env.example .env.local
```

The full list of environment variables, what each one does, and where to obtain its value is documented in section 2.4.5 Configuration. Pay particular attention to the variables marked Required, since the application will refuse to start without them.

Step 8: Promote your first admin user

The application creates new accounts with the student role automatically when someone signs in for the first time. There is no built in mechanism to create the first admin account, so this step must be done manually using SQL after at least one user has signed in.

After your intended administrator has signed in to the system at least once (which creates their user record), open the Supabase SQL Editor and run the following query, replacing the email with the actual administrator's Swinburne email.

update "Users"
set role = 'admin'
where email = 'their-swinburne-email@swinburne.edu.my';

Once this update is run, the next sign in by that user will land on the admin overview page.

Step 9: Optional but recommended - seed the initial book catalogue

The system runs against an empty book catalogue by default. To populate the catalogue with a starter set of books, run the seed scripts.

node scripts/seed-books.mjs --apply
node scripts/fill-cover-images.mjs

The first script imports books from the Open Library API. The second backfills cover images for any books that are missing one. These steps are optional but strongly recommended because an empty catalogue makes it difficult to test or demonstrate the system.

Step 10: Run the development server

Start the development server with the dev script.

pnpm dev

The server will start on <http://localhost:3000> by default and will also be reachable on your local network via your machine's LAN IP address such as <http://192.168.1.42:3000>. The LAN binding is useful for testing the camera scanner from a phone on the same network. Open the URL in your browser, click Get Started, and sign in with a Swinburne Microsoft 365 account to verify the system works end to end.

Step 11: Build and run the production version locally

Before deploying to production, verify that the production build completes without errors on your local machine.

```
pnpm build  
pnpm start
```

If pnpm build completes successfully and pnpm start runs the production server, you are ready to deploy.

Step 12: Deploy to Vercel

The project is designed to deploy on Vercel. The repository does not currently include a vercel.json manifest, so the deployment configuration is set up through the Vercel dashboard or the Vercel CLI rather than checked into the repository.

To deploy, sign in to Vercel and import the GitHub repository through the Vercel dashboard, or use the Vercel CLI to link the project from your local machine.

```
vercel link
```

After the project is linked, add every required environment variable from your .env.local file into the Vercel project settings. You can do this through the Environment Variables panel in the Vercel dashboard, or use the Vercel CLI to manage them from the command line. Once the variables are set, Vercel will automatically build and deploy the project on every push to the main branch. Preview deployments are also created automatically for every other branch and every pull request.

Step 13: Verify continuous integration

The project includes a GitHub Actions workflow at .github/workflows/test.yml that runs on every push to any branch. The workflow installs dependencies, builds the project, and runs the Jest test suite. After your first push to GitHub, check the Actions tab on the repository to confirm that both the Build_Nextjs and Jest_Test jobs complete successfully. The workflow does not require any GitHub secrets to be configured.

If both jobs pass, the build and deployment pipeline is fully working and the system is ready for ongoing development.

2.4.2 Project Structure

This sub-section gives an overview of the project's directory layout so a developer or system administrator can quickly find the file or folder they are looking for. The project follows the standard Next.js App Router convention at the top level, with additional folders for shared utilities, database migrations, tests, scripts, and the optional MCP server.

A simplified view of the repository root is shown below.

```
Library_Self-Checkout_System/
├─ app/                # Next.js App Router pages, API routes, UI, and shared lib
├─ lib/                # Root level utilities such as sip2.ts and
barcodeScanner.ts
├─ supabase/
│   └─ schema.sql      # Complete database schema script (used for fresh setup)
│   └─ migrations/     # 10 incremental migration files kept for reference
├─ __tests__/          # Jest test suite (about 23 test files)
├─ mcp/
│   └─ server.mjs      # Optional standalone MCP server
├─ scripts/            # Seed and verification scripts
├─ public/             # Static assets such as the Swinburne logo
├─ .github/
│   └─ workflows/      # CI workflow definitions
│   └─ setup/          # Custom CI setup action
├─ auth.ts             # NextAuth configuration
├─ middleware.ts        # Route guards and dev bypass logic
├─ next.config.ts      # Next.js configuration
├─ tailwind.config.ts  # Tailwind theme tokens
├─ jest.config.ts      # Jest configuration
├─ tsconfig.json       # TypeScript configuration
├─ package.json        # Project metadata and npm scripts
└─ .env.example        # Environment variable template
```

Figure 45: View of the project folder structure.

The table below describes the most important top level files and folders in more detail.

Path	Purpose
app/	All page, API route, UI component, and shared library code lives here. This is the largest and most actively edited folder in the project.
lib/	Root level utilities that do not fit cleanly under app/. Notable files include sip2.ts (SIP2 client wrapper) and barcodeScanner.ts (camera scanner abstraction).

supabase/schema.sql	Single file containing the complete database schema. Use this for a fresh setup as described in section 2.4.3 Step 4.
supabase/migrations/	Ten incremental SQL migration files kept for reference and historical context. Not needed for fresh deployments.
tests/	Jest test files. Contains unit tests for utility functions, API route tests, and component tests.
mcp/server.mjs	A standalone Model Context Protocol server that exposes catalogue data over stdio. Optional and not required for the main application to run.
scripts/	Node scripts run via node scripts/<name>.mjs. Used for one off tasks such as seeding books from the Open Library API and backfilling cover images.
public/	Static assets served at the root of the site, including the Swinburne logo.
auth.ts	NextAuth configuration. Defines the Azure AD and optional LinkedIn providers, the callback flow, and the JWT and session shapes.
middleware.ts	Edge middleware that runs on every protected request. Enforces role based route guards and short circuits authentication when the dev bypass is active.
next.config.ts	Next.js configuration. Whitelists external image hosts (Microsoft, Swinburne, YouTube, Unsplash) and marks the @xenova/transformers package as a server only external.
tailwind.config.ts	Tailwind CSS theme tokens including the warm cream and charcoal palettes for light and dark modes.
package.json	Project metadata, npm scripts (see section 2.4.3), and the full dependency list.
.env.example	Template for the local environment file. Copy this to .env.local during setup.

The app/ folder is where most of the development happens, so a closer look at its structure is useful.

The app/dashboard/ folder is the role specific dashboard shell. It contains the routes for student, staff, and admin views, including book/, my-books/, notifications/, recommendations/, learning/, profile/, and the camera based cameraScan/ route. It also holds the server actions (actions.ts, bookActions.ts, overdueActions.ts, damageActions.ts) and the loan policy constants (loanPolicy.ts).

The `app/api/` folder contains the REST API routes for endpoints such as `/api/checkout`, `/api/checkin`, `/api/books`, `/api/notifications`, `/api/reading-assistant`, and `/api/holds`. These are called by client components and server actions when a server side operation is needed.

The `app/lib/` folder holds shared logic used across pages and routes. Notable subfolders include `auth/` (session helpers and types), `supabase/` (database client and queries), `recommendations/` (Gemini integration and prompt building), and `validators/` (Zod schemas).

The `app/ui/` folder is the UI component library. It contains the dashboard shell, the sidebar navigation, reusable design primitives, the `magicUi/` folder for animated components, and the `theme/` folder for the light and dark mode provider.

The `tests/` folder mirrors the source structure roughly. Tests are organised by what they cover: API route tests (such as `readingAssistant.api.test.ts`), UI component tests (such as `homePage.test.jsx` and `signOutButton.test.jsx`), and library tests (such as `lib.barcode.test.ts` and `queries.activeLoans.isbn.test.ts`).

The `scripts/` folder holds operational scripts that are not part of the running application. These include `seed-books.mjs` and `seed-books-bulk.mjs` for populating the catalogue from the Open Library API, `fill-cover-images.mjs` for backfilling cover images, and `verify-books-openlibrary.mjs` for data quality checks. All scripts are run with `node scripts/<name>.mjs` from the project root.

2.4.3 Configuration

This sub-section provides the environment variable reference for the system. Variables are organised into four groups based on whether they are required for the system to start, recommended for full functionality, used only for advanced tuning, or restricted to development use only. For convenience, the `.env.example` file at the project root contains placeholders for the most common variables.

Build time versus runtime variables

Most environment variables are read at runtime on the server and are never sent to the browser. The only exceptions are variables prefixed with `NEXT_PUBLIC_`, which are bundled into the client side JavaScript at build time. This means two important things. First, `NEXT_PUBLIC_` variables are visible to anyone who inspects the page source, so they should never contain secrets. Second, changing a `NEXT_PUBLIC_` variable requires a fresh build before the new value takes effect; updating the value in Vercel and triggering a redeploy is the standard workflow.

Required for production

These variables must be set for the system to start in production. The application throws an error and refuses to start if any of them are missing. All seven must be configured before your first deployment.

Variable	Default	Purpose
NEXTAUTH_SECRET	"dev-secret" in development only	JWT signing key for session tokens. Generate a strong value with openssl rand -base64 32.
AZURE_AD_CLIENT_ID	None	OAuth client ID from your Azure App Registration.
AZURE_AD_CLIENT_SECRET	None	OAuth client secret from Azure Certificates & secrets.
AZURE_AD_TENANT_ID	None	Directory (tenant) ID from your Azure App Registration.
NEXT_PUBLIC_SUPABASE_URL	None	The public Supabase project URL. Bundled into the browser at build time.
NEXT_PUBLIC_SUPABASE_ANON_KEY	None	The browser facing Supabase API key. Bundled into the browser at build time.
SUPABASE_SERVICE_ROLE_KEY	None	Server side Supabase key that bypasses row level security for server actions and the audit log. Never expose this to the browser.

Optional but recommended

These variables enable the optional integrations that make the system feature complete. The application runs without them, but specific features will be limited or will fall back to demo data.

Variable	Default	What happens if missing	Purpose
GEMINI_API_KEY	None	AI Reading Assistant returns a fallback error message instead of a real reply	API key from Google AI Studio.
YOUTUBE_API_KEY	None	Learning Hub shows a Demo mode banner with sample videos	API key from Google Cloud Console for the YouTube Data API.
SIP2_BASE_URL	None	SIP2 sync is skipped silently; Supabase remains the source of truth	HTTP base URL of the legacy library SIP2 service.
SIP2_API_KEY	None	Same as above	Authentication token for the SIP2 service.

Advanced configuration

These variables fine tune behaviour for advanced deployments. The defaults are sensible for most installations, so these variables can usually be left unset. Set them only if you need to override specific behaviour.

Variable	Default	Purpose
NEXTAUTH_URL	Auto detected	Override the callback URL when running behind a proxy or non standard host.
AUTH_SECRET	Same as NEXTAUTH_SECRET	NextAuth v5 alias accepted as a fallback.
AUTH_URL	Same as NEXTAUTH_URL	NextAuth v5 alias accepted as a fallback.
AUTH_AZURE_AD_CLIENT_ID	Same as AZURE_AD_CLIENT_ID	NextAuth v5 alias for the Azure client ID.
AUTH_AZURE_AD_CLIENT_SECRET	Same as AZURE_AD_CLIENT_SECRET	NextAuth v5 alias for the Azure client secret.
AUTH_AZURE_AD_TENANT_ID	Same as AZURE_AD_TENANT_ID	NextAuth v5 alias for the Azure tenant ID.
SUPABASE_URL	Falls back to NEXT_PUBLIC_SUPABASE_URL	Server side override, useful when the public URL differs from the internal URL.
GEMINI_MODEL	gemini-2.5-flash-lite	Specific Gemini model to use for chat completions.
GEMINI_API_BASE_URL	https://generativelanguage.googleapis.com/v1beta	Override for the Gemini API endpoint, useful for custom proxies.
YOUTUBE_TRENDING_REGION_CODE	MY	ISO region code for trending video lookups.
YOUTUBE_TRENDING_CATEGORY_ID	28	Numeric YouTube category ID (28 is Science and Technology).
SIP2_TIMEOUT_MS	5000	Timeout in milliseconds for each SIP2 request.
SIP2_INSTITUTION_ID	LIB001	Institution identifier sent in each SIP2 message.
SIP2_TERMINAL_PASSWORD	(varies, see warning below)	SIP2 terminal password.
SIP2_PATRON_PASSWORD	(varies, see warning below)	SIP2 patron password.
BRAVE_SEARCH_API_KEY	None	API key for Brave Search, used by the external resources route to surface web links.
GOOGLE_BOOKS_API_KEY	None	API key for Google Books, used only by the fill-cover-images.mjs script when seeding cover images.

Development only

These variables enable a development bypass that lets a developer skip Microsoft sign in entirely while running locally, which is useful for testing all three role views without needing three real Microsoft accounts. The bypass only activates when `NODE_ENV` is set to development; it will not activate in production no matter how the variables are set. Do not set these variables on a production deployment.

Variable	Default	Purpose
<code>DEV_BYPASS_AUTH</code>	unset	Set to 1, true, yes, or on to enable the bypass. Honoured only when <code>NODE_ENV</code> is development.
<code>DEV_BYPASS_ROLE</code>	user	Role to assume when bypass is active. Accepts admin, staff, librarian, user, or student.
<code>DEV_BYPASS_EMAIL</code>	library.dev@example.com	Email for the synthetic user.
<code>DEV_BYPASS_NAME</code>	Library Developer	Display name for the synthetic user.
<code>DEV_BYPASS_USER_ID</code>	dev-user	User ID for the synthetic user.

Important warnings

The development bypass must never be enabled in a production deployment. The only safeguard against accidental activation is the `NODE_ENV` check, so it is critical that `NODE_ENV` remains set to production on Vercel and on any other production host. Setting `NODE_ENV` to development on a production deployment combined with `DEV_BYPASS_AUTH` would let any visitor sign in as the synthetic admin user without credentials, so this combination must be avoided.

The SIP2 terminal and patron password defaults are currently inconsistent in the codebase. The server actions in `app/dashboard/actions.ts` default these passwords to empty strings, while the API route at `app/api/checkout/route.ts` defaults them to placeholder values. To avoid confusion, set both `SIP2_TERMINAL_PASSWORD` and `SIP2_PATRON_PASSWORD` explicitly in your environment file with values supplied by the library SIP2 administrator.

2.5 DESIGN & IMPLEMENTATION DETAILS

2.5.1 Architectural Overview

This sub-section explains the main design ideas and key build choices that guide the rest of section 4.5. The architecture diagram and request flow are in section 2.2.1, and the full technology list is in section 2.2.3, so the focus here is on the reasons rather than repeating those. Several parts of the stack were fixed by the client from the start: TypeScript as the project language, Supabase as the database, Microsoft Azure AD for authentication, and integration with the legacy library SIP2 service. Within these constraints, three principles guided the team's design decisions, and each one came with a trade off the team chose on purpose. The first principle is to use TypeScript everywhere, not only on the front end. Because TypeScript is the required language, the team chose to use it across the whole stack so the front end and the back end share the same type system. Server Components, Server Actions, API routes, and shared utility libraries all use one type system, so a type defined once in `app/lib/supabase/types.ts` is automatically checked everywhere it is used. The trade off is the loss of flexibility to add a separate service in another language (such as Python for machine learning). The team accepted this to keep the system simple and easy for new developers to learn. The second principle is to use managed services rather than running our own infrastructure. Two of the managed services in the stack — Supabase and Azure AD — were already client requirements. The team extended this approach by choosing Vercel for deployment, so the whole platform layer is operated by external providers. This let a small student team release a production ready system without using sprint time on server setup, monitoring, or scaling. The trade off is vendor lock in: moving away from Supabase or Vercel would mean rewriting large parts of the data layer and the deployment pipeline. The third principle is that Supabase is the source of truth and external integrations are best effort. All main data (loans, holds, users) lives in Supabase. The legacy SIP2 service is kept in sync with Supabase but the system does not depend on it for correctness, the Gemini API can fail without breaking the application, and the YouTube API switches to demo data when it is not available. The trade off is that the SIP2 emulator can go out of sync with Supabase if many failures pile up; this is acceptable because the legacy ILS is going to be replaced soon, not kept as a system that has to stay in sync. The rest of section 2.5 covers the database schema (2.5.2), the key technical decisions that put these principles into action (2.5.3), four main features in detail (2.5.4), and the security and permissions design (2.5.5).

2.5.2 Database Design

This sub-section explains the design of the project's database. The complete schema is captured in `supabase/schema.sql` in the repository, and section 2.4.4 covers where that file lives in the project structure. Here, the focus is on the design decisions behind the schema and what they say about how the system handles books, loans, users, and notifications.

The database has 27 tables in total. The core business tables are listed in the table below, with their main role and key relationships. Less central tables (audit history, community features, user interests) are not listed here because they support secondary features rather than the main lending workflow.

Table	Role	Key relationships
Users	The main identity record. One row per user, created automatically on first Microsoft sign in.	Referenced by almost every other table
UserProfile	Extended profile data (faculty, year, bio, interests). Separated from Users so the auth flow can create a bare Users row at first sign in.	One to one with Users
Books	Bibliographic record for each title (ISBN, author, publisher). One row per unique title.	Parent of Copies
Copies	A physical copy of a book with its own barcode and status. Many copies per title.	Belongs to Books, referenced by Loans
Loans	Active and historical borrow transactions. <code>returned_at</code> IS NULL means still on loan.	Belongs to Copies, Users
Holds	The reserve queue for a title when no copies are available.	Belongs to Books, Users
DamageReports	Reports filed when a copy is returned damaged or marked lost.	Belongs to Loans, Copies
Notifications	The notification feed for users and staff.	Optional <code>target_user_id</code> link to Users

NotificationReads	Tracks which user has read which notification.	Composite primary key on (notification_id, user_id)
NotificationFlags	Per user starring of notifications.	Composite primary key on (notification_id, user_id)
GeneralChatHistory	Conversation history for the AI Reading Assistant, persisted across sessions.	Belongs to Users

Three key design decisions in the schema

The first is the separation of Books and Copies into two tables. A Book row is the bibliographic record (one per ISBN), and Copies holds the physical objects on the shelf. This split follows standard library catalogue design (similar to how MARC records and items are separated in real library systems). Without the split, updating a title would mean touching every copy, and tracking "three copies in, two on loan" would need either complex aggregation queries or denormalised counters that can drift out of sync. With the split, the AvailableCopies view (defined in schema.sql) handles availability counting cleanly.

The second is the search_books stored procedure. Library catalogue search needs to handle typos ("introduction to programming") and partial matches ("intro programming"). A simple SQL LIKE query cannot do this without becoming very slow on a large table. The team's solution uses PostgreSQL's pg_trgm extension to do fuzzy matching: the search_books function calls word_similarity(query, title) and word_similarity(query, author) with a 0.3 similarity threshold, combined with an ILIKE fallback on title, author, ISBN, classification, and publisher. The function is backed by two GIN indexes (books_title_trgm_idx and books_author_trgm_idx, both using gin_trgm_ops), which make the fuzzy matching fast even on the full catalogue. Without these indexes, every search would be a full table scan.

The third is the heavy use of Postgres enum types. Roles, copy statuses, hold statuses, notification channels, profile visibility, and several community related fields are all enums (UserRole, CopyStatus, HoldStatus, NotificationChannel, ProfileVisibility, and others). The benefit is that Postgres rejects invalid values at insert time, which means the application does not need to add its own validation on every write. The trade off is that adding a new enum value requires a migration with ALTER TYPE ... ADD VALUE. The team accepted this trade off because the set of roles and statuses changes rarely, so the cost of the migration is low.

Schema fix during Sprint 3

One database change made during this sprint deserves a specific note because it shows how production schema drift can hide subtle bugs. The NotificationReads table was originally defined with a composite primary key on (notification_id, user_id), so each user can independently mark each notification as read. But the live database had ended up with a single column primary key on notification_id alone, meaning only one user could mark a given notification as read. The mark all read feature was silently failing: the upsert call returned HTTP 200, but no rows were inserted because the conflict resolution targeted a non-existent composite key. The migration 20260507_fix_notification_reads_pk.sql drops the wrong primary key and adds the correct composite key. Without writing migrations to compare the live database against the source schema, the team would not have found this drift.

Performance indexes

The schema includes several indexes that are not just for primary keys or foreign keys but specifically for query performance. These are summarised in the table below.

Index	Type	Purpose
books_title_trgm_idx	GIN with gin_trgm_ops	Fast fuzzy title search in search_books
books_author_trgm_idx	GIN with gin_trgm_ops	Fast fuzzy author search in search_books
books_embedding_idx	ivfflat with vector_cosine_ops	Approximate nearest neighbour search for AI book recommendations
idx_loans_overdue_active	Partial B-tree (WHERE returned_at IS NULL)	Fast overdue scan that skips all returned loans
idx_notifications_created_at	B-tree DESC	Feeds the notification list in reverse chronological order
idx_general_chat_history_user_created	Composite (user_id, created_at)	Fast retrieval of a user's chat history in time order

The partial index on Loans is especially useful: most loans in a long running library system are already returned, so a normal index on due_at would scan a lot of dead rows when looking for overdue items. The partial index only contains active loans, keeping it small even after years of operation.

Known design compromises

Three compromises in the schema deserve to be flagged honestly so they can be addressed in future sprints.

First, 25 of the 27 tables have row level security disabled. The application avoids this by routing all writes through server side code that uses the Supabase service role key, so the anon and authenticated keys cannot reach those tables in practice. However, this is architectural trust, not a database level guardrail. If the service role key ever leaked, it would give full unrestricted access to the entire database. Enabling RLS with policies for each role and each table is in the Sprint 4 backlog.

Second, the schema includes a small number of preserved spelling errors. The StgBooksRows table has columns named `avaliable_copies`, `copies_avaliable`, and `avaliable` (all should be "available"), and `BookRecommendation.update_at` and `CommunityPostComments.update_at` should be `updated_at`. These are kept as is because they exist in the live production database and the application code already references them by these names. Fixing them would require a migration plus updates across the codebase. The team chose to leave them and document the issue rather than risk breaking dependent queries during a critical sprint.

Third, two notification tables (Notification singular and Notifications plural) coexist. The legacy Notification table uses a Postgres enum for the type column, while the active Notifications table uses a text column with a CHECK constraint. The application uses the plural table only; the singular one is left in place for safety in case any process still reads from it. Cleaning this up is also on the Sprint 4 backlog.

2.5.3 Key Technical Decisions

This sub-section documents the five most important technical decisions the team made during the project, beyond the constraints set by the client. Each decision is described with what was chosen, why, and the trade off the team accepted. These decisions explain how the principles in section 4.5.1 were put into action.

Decision 1: Server Actions for mutations, REST APIs for async operations

Next.js offers two ways to call server side code from the client: Server Actions (called from React forms) and REST API routes (called through fetch). Both were available because the team chose Next.js, but the team had to decide which to use for what. The choice was to use Server Actions for all circulation mutations (checkout, checkin, renew, place hold, file damage report, add or edit a book) and REST API routes for anything called asynchronously without a form, mainly the notification bell that polls every few seconds and the AI Reading Assistant chat.

The reason for this split is that Server Actions integrate cleanly with React's `useActionState` hook, so a checkout form can show success or error messages inline without writing any client side fetch logic. They also call `revalidatePath` after a mutation to refresh the server rendered cache in the same request. REST routes do not have these advantages but they can be called any time, not only from form submissions, which is why the notification poller and the chat use them.

The trade off is that Server Actions are tightly coupled to Next.js. If the application ever needed a separate mobile client or external API consumer, only the REST routes would be available out of the box; the Server Action logic would have to be moved to REST routes first.

Decision 2: Azure AD as the identity provider, role lookup from Supabase

Microsoft Azure AD was a client requirement. Within that requirement, the team had to decide how to integrate it and where the user role information should live. The team chose to use NextAuth (version 5 beta) as the auth library and to read each user's role (user, staff, or admin) from the Supabase Users table rather than from Azure AD claims.

The reason for using NextAuth is that it ships with an Azure AD provider built in, which handles the full OAuth flow without custom code. The reason for reading the role from Supabase is more interesting: it lets a Swinburne admin promote a student to staff or admin by updating one row in the Users table, without needing to make changes inside Azure AD. The role is re-fetched from Supabase on every JWT rotation, so role changes take effect on the user's next session refresh without requiring a full sign out and sign in cycle.

The trade off is that the source of truth for permissions is split. Azure AD knows who you are, but Supabase decides what you can do. If these go out of sync (for example, an Azure AD account is disabled but the Supabase row stays active), the application could behave inconsistently. The team accepted this because the alternative would have required Azure AD tenant administrator access to push role information through Azure, which the team did not have.

Decision 3: Disable row level security, use the service role key on the server

Supabase supports row level security (RLS) policies at the database level: each table can have policies that say "an authenticated user can only see rows where user_id equals their own ID." This is a strong security model that pushes access control into the database. The team chose not to enable RLS on most tables and instead enforce access control in the application layer using the Supabase service role key.

The reason is the cost of writing correct RLS policies for a system with three roles (user, staff, admin) and tables with complex relationships. For example, a staff member needs to see any loan, but a user can only see their own; a hold record should be visible to the user who placed it and to all staff but not to other users. Writing policies for every table, every operation (select, insert, update, delete), and every role would require dozens of policy statements and careful testing of each combination. Instead, the team enforces these rules in server actions and API routes through helper functions called `requireUser`, `requireStaff`, and `requireAdmin`, which check the JWT role before any database call.

The trade off is significant and the team is honest about it: there is no database level guardrail. If a server action has an input validation bug, no RLS policy will catch it. If the `SUPABASE_SERVICE_ROLE_KEY` ever leaks (through a logging mistake, a stolen developer machine, or a misconfigured environment variable), the attacker has full unrestricted access to every row in every table. The team reduces this risk by keeping the key only in server side environment variables, never in any `NEXT_PUBLIC_` variable, and never logging it. Enabling RLS for the 25 currently unprotected tables is in the Sprint 4 backlog.

Decision 4: Treat SIP2 as best effort, not authoritative

The client required the system to integrate with the legacy library system through SIP2, which is the standard library protocol for circulation. The team had to decide whether SIP2 should be authoritative (a transaction must succeed in SIP2 before being committed in the application) or auxiliary (the application is authoritative and SIP2 is updated as a side effect, allowed to fail). The team chose auxiliary, with a 5 second timeout on every SIP2 call.

The reason is reliability and user experience. The SIP2 service used during development is a thin wrapper around an emulator that is occasionally slow or unavailable. If checkout were blocked on a successful SIP2 response, users could be stuck on a "Processing..." screen for 30 seconds during a SIP2 outage, and a failed SIP2 call would mean the user could not borrow the book even though Supabase recorded the transaction successfully. Treating SIP2 as auxiliary keeps the user experience smooth and lets the system keep working during SIP2 outages.

The trade off is that the SIP2 emulator can drift out of sync with Supabase if outages happen often. Currently, SIP2 failures are only logged through `console.error`; there is no retry queue and no scheduled job that reconciles the two systems. This is acceptable in the project's context because the legacy ILS is going to be replaced by Supabase, not kept indefinitely as a parallel system. If the legacy ILS were staying in production, a stronger synchronisation strategy would be needed.

Decision 5: Gemini 2.5-flash as the default AI model

For the AI Reading Assistant, the team chose Google Gemini and specifically the Flash Lite variant (gemini-2.5-flash-lite). The model is configured through the GEMINI_MODEL environment variable, which means swapping to a different Gemini model later (Flash, Pro, or any future release) only requires changing the env value and redeploying, without any code changes.

The reason for choosing Flash Lite over Flash, Gemini 2.5 Pro, or other more capable models comes down to task fit and cost. The Reading Assistant's system prompt asks the model to do two things: classify the user's intent into one of six categories (find_books, answer, both, greeting, off_topic, loan_status) and either extract search terms or write a short reply. These tasks are structured (the output is a constrained JSON object, maxOutputTokens is set to 1024 in the request) and do not benefit from extended reasoning. Flash Lite does not run thinking mode by default, so it returns answers faster and at the lowest cost in the Gemini 2.5 family. Testing showed that Flash Lite produces correct intent classifications and reasonable replies for the kinds of questions students actually ask. Paying for a more capable model would not improve the user experience for this workload, only the API costs.

The trade off is that there is currently no fallback. If the Gemini API returns a 403, the module level geminiDisabled flag is set and all subsequent calls return null until the server is restarted; users see a static error message instead of a generated response. The team chose this simple behaviour over a more complex multi provider fallback chain because the AI features were judged a useful enhancement rather than a critical path; the rest of the application keeps working even when the Assistant is down.

2.5.4 Core Feature Implementation

This sub-section walks through the implementation of four core features in detail. For each feature, this includes the relevant file locations, the technical highlights worth calling out, and an honest note on any known gaps or limitations. The four features were chosen because they are the most technically substantive parts of the system: the AI Reading Assistant, the camera barcode scanner, the role-based middleware that protects every dashboard route, and the notification system.

Feature 1: AI Reading Assistant

The Reading Assistant is implemented across three locations: `app/lib/recommendations/ai.ts` (Gemini integration and prompt construction), `app/api/reading-assistant/route.ts` (the REST endpoint that the chat UI calls), and `app/ui/dashboard/readingAssistant/` (the chat interface itself).

The Assistant uses a single round trip with Gemini for each user message. Rather than making two calls (one to classify intent, another to generate a response), the team chose to send a system prompt that asks Gemini to return a single structured JSON object containing the intent, the reply text, the search terms, and an optional follow up question. This design relies on Gemini's JSON output mode, where the model is constrained to return valid JSON only.

The complete system prompt sent to Gemini for each request is reproduced below, exactly as it appears in the `buildUnifiedSystemPrompt` function in `ai.ts`. The placeholders `${todayStr}`, `${studentCtx}`, `${loansBlock}`, and `${returnsBlock}` are filled in at runtime with the current date, the student's profile context (faculty, year, interests, recent borrowing history), the active loans, and the books returned within the last 14 days.

Prompt
You are a helpful university library assistant. Your job is to help students find books and answer academic questions. You must respond ONLY with a valid JSON object — no markdown, no extra text, no code fences. Classify the student's message into one of these intents: - "find_books": student wants book recommendations - "answer": student has an academic question - "both": student asks a question AND wants books - "greeting": hi, hello, how are you, or other small talk - "off_topic": requests unrelated to studying or books - "loan_status": questions about the student's current loans, due dates, overdue items ("what's due tomorrow?", "any overdue?", "what books do I have?") Response format: <pre>{ "intent": "find_books" "answer" "both" "greeting" "off_topic" "loan_status", "reply": "Your natural, friendly response. Plain text only — no asterisks, no bullet points, no markdown.", "searchTerms": ["term1", "term2"], "followUpQuestion": "One short follow-up question (only for find_books or both, else empty string)" }</pre>

Rules:

- reply must always be plain text. Never use **bold**, *italic*, bullet points, or markdown.
- For "find_books" or "both": searchTerms must be the SUBJECT/TOPIC the student wants — proper noun phrases as they would appear in a book title or table of contents. NEVER include filler verbs (give, show, find, recommend, suggest, want), quantifiers (some, any, a few), or pronouns (me, my). Expand acronyms (AI → "artificial intelligence", ML → "machine learning", DB → "database", OOP → "object-oriented programming"). Use 2-6 multi-word phrases when possible. Examples:
 - "give me some AI books" → searchTerms: ["artificial intelligence", "machine learning", "neural networks"]
 - "I need books for my marketing assignment" → searchTerms: ["marketing strategy", "consumer behavior", "brand management"]
- For "answer" or "both": give a concise academic explanation (2-4 sentences max).
- For "greeting": reply warmly and invite the student to ask for books or academic help. searchTerms must be empty.
- For "off_topic": politely decline and redirect to books or academic topics. searchTerms must be empty.
- For "loan_status": answer using ONLY the "Currently borrowed" data below. Never invent dates or titles. Use today's date for "due tomorrow", "due soon", "overdue" reasoning. searchTerms must be empty.
- Never recommend a book that appears in "Currently borrowed".
- When the student asks for a recommendation and recent-return data exists below, prefer adjacent or follow-up topics to those books and name the connection in reply (e.g. "Since you just finished X, here's something on Y").
- When the student asks for recommendations without specifying a topic, infer topics from recent returns, recently borrowed history, and known interests.
- You can answer questions from ALL academic fields.
- Never reveal what AI model you are. You are the library assistant.
- Today's date: \${todayStr}.\${studentCtx}\${loansBlock}\${returnsBlock}

Three things in this prompt are worth pointing out. First, the searchTerms rule explicitly lists filler words and pronouns that the model must strip out, with worked examples for both ambiguous

queries ("give me some AI books") and assignment style queries ("books for my marketing assignment"). This is necessary because the search terms are passed straight into the search_books RPC discussed in section 4.5.2; if the model returns "give me books", the trigram search returns nothing useful. Second, the loan status branch grounds the model's answers in real data from the database by injecting the user's current loans into the prompt rather than letting the model infer them, which prevents the model from inventing due dates. Third, the prompt forbids the model from revealing what underlying AI it is, so the assistant always presents itself as the library assistant rather than as Gemini.

Conversation history is persisted in the GeneralChatHistory table (one row per turn, tagged with sender = 'user' or 'assistant'). The persistTurn function in route.ts writes both the user message and the assistant reply to the database immediately after each request. The last 10 turns are loaded back from the database when the chat panel mounts and are sent to Gemini as conversation history on every subsequent request. The 10 turn limit is held in a constant called READING_ASSISTANT_HISTORY_LIMIT in the policy file, so it can be tuned without changing the API code.

The chat panel offers six quick prompt chips that pre-fill the input field with common questions: "How do I renew a loan?", "What happens if a book is overdue?", "Find me a fantasy novel", "Recommend something light for the weekend", "How do I place a hold?", and "Computer science textbooks I can borrow now". These cover the three most common user intents (loan policy questions, casual recommendations, and topic based recommendations).

If the Gemini API call fails for any reason, the chat panel catches the error on the client side and displays a static fallback message: "Sorry — I couldn't reach the AI just now. Please try again in a moment, or ask a librarian directly." There is no automatic retry and no fallback to a non AI alternative such as a keyword only search. This keeps the failure mode simple and predictable.

One known gap in the current implementation: the /api/reading-assistant route does not check the user's role before forwarding the request to Gemini. Any authenticated user (including students) can call the endpoint, and there is no rate limit. If a student were to script the endpoint, they could consume Gemini API quota at a rate the team has not budgeted for. Adding a per user rate limit and a role gate is in the Sprint 4 backlog.

Feature 2: Camera Barcode Scanner

The barcode scanner is implemented in lib/barcodeScanner.ts (the scanning logic) and app/ui/dashboard/cameraScanner.tsx (the camera viewfinder UI). The scanner is used by both the user side cameraScan route and the staff side checkout flow, so it had to work on both desktop browsers and mobile phones held over a printed barcode.

The scanner supports both 1D and 2D barcode formats: CODE_128, CODE_39, CODE_93, CODABAR, ITF, EAN_13, EAN_8, UPC_A, UPC_E, QR_CODE, DATA_MATRIX, PDF_417, and AZTEC. The TRY_HARDER flag is enabled, which tells the underlying decoder to try slower but more reliable algorithms when the fast path fails.

There is a three step decode strategy. First, the scanner tries the browser's native BarcodeDetector API, which is available on Android Chrome and other Chromium based browsers and is fast because it can use hardware acceleration. If BarcodeDetector is not available (Safari and Firefox do not currently implement it), the scanner falls back to the @zxing/browser JavaScript library, which works everywhere but is slower because it runs entirely in JavaScript. The third option is always available regardless: a manual text input box sits next to the camera viewfinder, so a user who cannot get a clean scan can simply type the barcode number.

The most interesting engineering detail in the scanner is the quiet zone padding. ZXing is stricter than the original Java reference implementation about requiring whitespace around a barcode (the "quiet zone"). Self-printed barcodes that are cropped tightly to the bars often fail to decode in ZXing because the scanner cannot find the start and end markers. To work around this, the scanner pads every captured frame with extra white space before passing it to ZXing: 10 percent on each edge for live camera frames and 12 percent for uploaded images. The padding is added by drawing the captured image onto a slightly larger canvas with a white background, which ZXing then sees as a properly framed barcode.

For uploaded images (rather than live camera frames), the scanner also runs at multiple resolutions in sequence: 1800 pixels on the long edge first, then 1200, 2400, 900, and the original size if all else fails. The reason is that ZXing's HybridBinarizer struggles with very large images such as 12 megapixel phone photos, where the global threshold gets confused by large uniform areas (a blank wall behind the book). Downscaling to around 1800 pixels usually hits the sweet spot where the barcode is large enough to read but small enough for the binarizer to handle.

The dev server is configured to bind to 0.0.0.0 rather than localhost, which means a developer can run the app on their laptop and open it on a phone on the same Wi-Fi network using the laptop's local IP address. This is specifically to test the camera scanner against real mobile cameras during development, without needing to deploy to Vercel for every change.

Feature 3: Role based Middleware

The middleware is a single file at the project root: middleware.ts. It runs on every request that matches the configured matcher and decides whether the request is allowed to proceed, redirected to the sign in page, or redirected to a "not allowed" page if the user lacks the required role.

The middleware protects six prefixes: /dashboard, /profile, /api/checkin, /api/checkout, /api/sip2, and /api/books. Two notable API routes are deliberately not in this list: /api/notifications and /api/reading-assistant. They implement their own auth checks inside their route handlers because they need finer control over the response (for example, the chat endpoint returns a specific error JSON shape).

Within the protected prefixes, role guard rules apply to specific sub paths. The rules are evaluated in order, and the first matching rule wins:

Path prefix	Allowed roles
/dashboard/admin/books	staff, admin
/dashboard/admin/overdue	staff, admin
/dashboard/admin (other)	admin
/dashboard/staff	staff, admin

The two staff accessible sub paths under /dashboard/admin (the books catalogue and the overdue list) are intentional carve outs: staff need to look up book records and see overdue items, but they should not be able to access the rest of the admin area such as user management.

Role information for the current request comes from the JWT attached to the request by NextAuth's auth() wrapper. The middleware reads sessionUser.role (or sessionUser.roles[0] as a fallback) and passes it through a normalisation function called resolveRole, which maps "librarian" to "staff" and "student" to "user". The normalisation handles the case where Azure AD or older code emits role names that are different from the current canonical set.

The DEV_BYPASS feature deserves a specific note because it is the team's biggest single source of truth assumption. When DEV_BYPASS_AUTH is set in the environment, the middleware short circuits the entire NextAuth check and synthesises a user with the role specified in DEV_BYPASS_ROLE. The bypass is gated behind two checks: NODE_ENV must be "development", and DEV_BYPASS_AUTH must be one of the literal strings "1", "true", "yes", or "on". Next.js sets NODE_ENV to "production" automatically at build time for production deployments, which means the bypass is structurally impossible in a production build without changing the source code. This is documented in section 2.4.5 alongside the relevant warning.

Feature 4: Notification system

The notification system is implemented in app/lib/supabase/notifications.ts (the data access functions) and several routes under app/api/notifications/ (the REST endpoints called by the bell component and the due check job). Notifications are stored in the Notifications table (plural, the active one), with read state tracked in NotificationReads and per user starring in NotificationFlags as described in section 4.5.2.

The notification feature with the most interesting logic is the due soon reminder. When a user opens the dashboard, the front end calls /api/notifications/due-check, which scans the user's active loans, looks for any due within the configured warning window (the current default is 3 days), and creates a "due_soon" notification for each one — but only if a due_soon notification has not already been created for that same loan on the same calendar day.

This deduplication is implemented with a metadata check rather than a date column. Each due_soon notification is created with a metadata object containing the loan ID and the current date in YYYY-MM-DD form. Before creating a new notification, the code queries Notifications for any existing row with type = 'due_soon', target_user_id = the current user, and metadata containing the same loan ID and date. If found, the new notification is skipped. The guard resets at midnight rather than on a rolling 24 hour window, which means a user who logs in at 11:59 PM

and again at 12:01 AM might briefly see two reminders for the same loan, but in practice the deduplication is good enough.

Not every notification type declared in the schema is wired up. The table below shows what is actually implemented versus what only exists as a value in the type column's CHECK constraint.

Notification type	Active in Notifications table	Wired to a trigger in code
checkout	Yes	Yes — fires in checkoutBookAction
checkin	Yes	Yes — fires in checkinBookAction
loan_confirmed	Yes	Yes — user-targeted, fires in checkoutBookAction
due_soon	Yes	Yes — /api/notifications/due-check endpoint
hold_placed	Yes	Yes — /api/notifications/hold-placed/route.ts
hold_ready	No (only in legacy Notification enum)	Not wired
overdue	No (not in any active CHECK constraint)	Not wired

Two notification types (hold_ready and overdue) exist in the legacy schema but were not wired up in this sprint. They are scheduled for Sprint 4. This honest accounting matters because a stakeholder reading only the schema might assume these features work; documenting the gap prevents that misunderstanding.

The per user starring feature is fully implemented. Each user can star (flag) any notification they have access to, and the starred state is stored as one row per (notification_id, user_id) pair in NotificationFlags. The endpoint PATCH /api/notifications accepts a body with notificationId and a boolean flagged value, and toggleNotificationFlag in notifications.ts handles the upsert.

For performance, the read state tracking is chunked. When the bell component asks "which of these notifications have I read?", the data layer batches the notification IDs in chunks of 50 to avoid hitting PostgreSQL URL length limits with large IN clauses. This matters because the notification feed can grow to thousands of rows over time, and a single URL with all those IDs would fail.

2.5.5 Security and Permissions

This sub-section summarises how the system protects user data and enforces role permissions, pulling together the design decisions discussed earlier in section 4.5. The security model is built in layers, and every layer is enforced on the server side. The browser is never trusted to enforce a permission check, even for visual changes such as hiding admin menu items.

The first layer is identity. Users sign in through Microsoft Azure AD using NextAuth, and a JWT is issued on each successful sign in. The JWT carries the user's email, ID, and role. Role information comes from Supabase rather than from Azure AD claims, as discussed in section 4.5.3 Decision 2.

The second layer is the middleware (section 4.5.4 Feature 3). Every request to a protected prefix (/dashboard, /profile, /api/checkin, /api/checkout, /api/sip2, /api/books) is intercepted by middleware.ts. The middleware verifies the JWT exists and has not expired, then applies role guard rules to redirect any user without the right role to a "not allowed" page.

The third layer is server side enforcement inside server actions and API routes. Every server action begins with a call to `requireUser()`, `requireStaff()`, or `requireAdmin()` to assert the caller has the right role before any database query is run. This is the layer that catches anything the middleware does not, notably /api/notifications and /api/reading-assistant, which sit outside the middleware matcher and are checked inside the route handlers.

The fourth layer is the Supabase service role key. All writes go through the server side client created with `SUPABASE_SERVICE_ROLE_KEY`. This key is held only in server side environment variables, never in any `NEXT_PUBLIC_` variable, never logged, and never exposed to the browser bundle. The key bypasses row level security (because RLS is disabled on most tables, see section 4.5.3 Decision 3) but is unreachable from the browser even if the anon key is leaked.

The fifth layer is the dev bypass safeguard. The development bypass for skipping Microsoft sign in is gated behind two checks: `NODE_ENV` must equal "development" (Next.js sets it to "production" automatically in production builds), and `DEV_BYPASS_AUTH` must be one of "1", "true", "yes", or "on". This means the bypass is structurally impossible to enable in a production deployment without changing the source code.

Known security gaps

The team has identified six gaps in the current security model that are scheduled for Sprint 4. Documenting these honestly matters because pretending they do not exist would mask real risk. The full list is below.

#	Gap	Risk	Planned fix
1	Row level security disabled on 25 of 27 tables	Full unrestricted database access if SUPABASE_SERVICE_ROLE_KEY leaks	Add per role RLS policies to all 25 tables
2	/api/reading-assistant has no role check or rate limit	Any authenticated user can consume Gemini API quota	Add requireUser() and a per user rate limit
3	Notifications.target_user_id is text without FK to Users(id)	Deleted users leave orphan notifications	Add FK constraint with ON DELETE CASCADE
4	AiChatHistory.user_id has no FK to Users(id)	Same as above	Same as above
5	NotificationFlags.user_id is text not uuid	Type inconsistency with Users.id	Migration to align column types
6	Duplicate indexes on UserInterests	Minor write overhead, no security impact	Drop one of the duplicate indexes

Three of these (1, 2, 3) carry real risk and are top priority for Sprint 4. The others are correctness or cleanliness issues rather than security holes, but they are tracked alongside the higher risk items so they are not forgotten.

2.6 TESTING

1. Testing Strategy

Sprint 3 testing combines automated tests run in continuous integration with structured manual testing across each user role. The automated layer focuses on three concerns: pure utility functions where regression is easy to catch, the shape of API and database calls (where Supabase and Gemini SDKs are mocked so the test asserts the call is built correctly without contacting the real services), and component rendering tests for the primitives most likely to drift across releases. Manual testing carries the remainder of feature verification: for every merged branch, the responsible team member exercises the user flows on the role they affect (student borrow and return, staff checkout and history export, admin overdue list and user management) before the branch is merged into the consolidated working branch.

The team chose this division because Sprint 3 was UI heavy and integration heavy. End to end manual verification surfaces issues such as inconsistent layouts, broken navigation, or missing role guards faster than writing exhaustive unit tests for every component would. Automated tests are reserved for logic that is easy to assert in isolation and expensive to break silently. Examples include CSV escape rules, ISBN-10 to ISBN-13 equivalence, the Gemini call shape, and the composite primary key handling on notification reads.

2. Test Cases

The codebase contains 23 test files under the `__tests__` directory at the repository root, run by Jest 30 (jsdom environment by default) with `@testing-library/react` for component tests. The files cluster into five groups:

Group	Count	Representative tests
Utility / logic	5	<code>computeRoleCounts</code> (admin role tabs), <code>toCsv</code> (CSV export with BOM and comma/quote/newline escaping), <code>computeNextBarcodes</code> (SWI-XXXXX generation with padding and overflow), <code>isbnMatch</code> and <code>normalizeIsbn</code> (ISBN-10 to ISBN-13 equivalence), <code>isLikelyImageUrl</code> and <code>validateImageUrl</code> validators
Database query shape	2	<code>fetchRecentLoansByUser</code> and <code>fetchRecentlyReturnedLoans</code> (filter and mapping correctness against mocked Supabase)
AI behaviour	5	<code>buildUnifiedSystemPrompt</code> renders today's date and the <code>loan_status</code> rules; the Gemini call places the prompt in <code>systemInstruction</code> (not <code>contents</code>); <code>classifyAndExtract</code> injects active loans and recent returns into the system instruction; the <code>AIIntent</code> type contains <code>loan_status</code> ; the <code>readingAssistant</code> system prompt covers all FAQ sections plus the no emoji rule
API integration	3	<code>/api/generalChatHistory</code> POST / GET / DELETE round trip; <code>/api/reading-assistant</code> POST pipeline; <code>markNotificationsReadByIds</code> upsert with composite primary key conflict target
Component rendering	8	<code>NotificationsFilter</code> (search, sort, apply, reset), <code>BarcodePreview</code> (single, range, loading), <code>IsbnLookupBox</code> (lookup click, disabled state, pending), <code>ReminderButton</code> (24 hour cooldown), <code>SignOutButton</code> , <code>ShimmerButton</code> , <code>LoginClient</code> (title and <code>signIn</code> invocation), <code>HomePage</code> (landing copy)

In the continuous integration pipeline (.github/workflows/test.yml), every push to any branch triggers a two stage workflow. The first job runs pnpm build, which surfaces TypeScript errors because strict is enabled in tsconfig.json. The second job depends on a successful build and runs pnpm test against all 23 files. Pull requests inherit this workflow because every push lands on the runner regardless of whether a pull request exists.

3. Bugs & Fixes

Sprint 3 produced more than thirty commits whose messages mark a bug fix. The following table presents six representative cases, each chosen to illustrate a different class of issue that the project encountered during the sprint:

Date	Fix	What was wrong
May 7	NotificationReads composite primary key migration (e07b4e3)	The original NotificationReads table used a single column primary key that allowed duplicate (notification_id, user_id) pairs, so the same notification could appear unread for the same user after being marked. The migration changes the primary key to the composite (notification_id, user_id)
May 7	ISBN-10 / ISBN-13 equivalence in active loan search (229c42c)	Returning a book by ISBN failed when the database stored the ISBN-13 but the user scanned the ISBN-10 (or vice versa). The fix normalises both values and matches across the two encodings
May 6	Resolve type errors after merge (a3f32ce)	The integration of one of the feature branches produced TypeScript build failures. The fix realigns the affected types so pnpm build passes
May 1	Camera reliability for printed barcode scanning (75976c9)	Printed barcodes on glossy paper failed to decode reliably on the live camera scan. The fix adds multiple resolution attempts and tighter quiet zone padding
April 10	Azure AD sign out and account switching (c8f70f1)	The previous sign out flow did not clear the Azure AD session, so signing out and signing back in skipped the account picker. The fix adds the post logout redirect that forces account selection
April 10	Case sensitive import for Linux on Vercel (fe11c12)	A LoginClient import worked locally on macOS (case insensitive file system) but failed the Vercel production build (case sensitive Linux runner). The fix normalises the import casing

The pattern across these six cases shows where the project most often broke during Sprint 3: at integration points (post merge type errors), at platform boundaries (Linux case sensitivity on the Vercel runner, Azure AD sign out lifecycle), at API contract edges (ISBN encoding equivalence), at hardware boundaries (camera scanner reliability on glossy paper), and at database integrity rules (composite primary keys). Each has been resolved in the Sprint 3 build and is verified either by the corresponding automated test or by manual walkthrough on the affected role.

2.7 LIMITATIONS & FUTURE IMPROVEMENTS

2.7.1 Current Limitations

The Sprint 3 build has several known gaps. The items below are deliberate compromises to ship the sprint on time, technical debt from the consolidation work, or features whose UI scaffolding exists but whose backend triggers were not completed within the sprint window. They are grouped by the layer they affect.

Security. Row Level Security (RLS) is disabled on twenty five of the twenty seven Supabase tables. Server side enforcement happens through the service role key in the application layer, which means a misconfigured route could expose data that RLS would otherwise block. The `/api/reading-assistant` endpoint does not currently enforce a role check or apply rate limiting per user, so any authenticated user can call the AI without restriction. The codebase also contains a `DEV_BYPASS` environment variable that skips authentication when set; it is intended only for local development, and production deployments must ensure the variable is unset.

Data. The codebase still contains both a `Notification` and a `Notifications` table. The application uses the plural form for all current code paths; the older singular table is no longer referenced but has not yet been dropped.

Notifications. Two notification types are scaffolded but not wired. The `hold_ready` type, which would alert a user when a placed hold becomes available for pickup, has UI components and database rows but no trigger logic. The `overdue` type, which would generate the daily reminder for items past their due date, is in the same state.

Integrations. The SIP2 integration runs in best effort mode with a five second timeout. The library management system can drift out of sync with the application database when the SIP2 service is unreachable, and there is no scheduled reconciliation job to detect or correct that drift.

Testing. Component tests cover the primitives and a few page level smoke tests. The redesigned dashboards and the consolidated reading-assistant page have no automated end to end coverage. Verification at this level relies on manual testing.

2.7.2 Future Enhancements

The following items are tracked for Sprint 4 or for later iterations, ordered by the area they target:

Area	Enhancement
Security	Enable RLS on the remaining twenty five tables and migrate the service role calls to use the user's session JWT where possible
Security	Add a role check and a rate limit per user to the /api/reading-assistant route
Security	Replace the DEV_BYPASS flag with documented test accounts that exist only in the development Supabase project
Notifications	Implement the hold_ready trigger using a Supabase database webhook on the Holds table state transition
Notifications	Implement the overdue daily job using a scheduled Vercel Cron Function that scans active loans past their due date
Database	Drop the unused Notification (singular) table after verifying no code paths reference it
Integrations	Add a nightly SIP2 reconciliation job that compares the application loan tables against the library management system and reports drift
Testing	Add Playwright end to end tests for the four critical flows (login, borrow, return, hold pickup) so that regressions in the redesigned pages are caught automatically
AI	Cache the Gemini response for common FAQ style questions to reduce cost and latency
Mobile	Build a lightweight mobile entry point optimised for the camera scanner workflow on staff and student devices

2.8 CONCLUSION

Sprint 3 delivered the planned scope of the library self checkout system. Students can borrow and return books through the web interface, staff can manage circulation operations and run overdue follow ups, and administrators can manage users, books, and the catalogue without direct database access. The build is functionally complete for the Sprint 3 backlog.

The Sprint 3 contribution beyond Sprint 2 spans four areas. Visually, the dashboard has been redesigned end to end into a unified design system with self hosted typography, a warm cream and charcoal palette, dark mode support, and responsive layouts that work on narrow viewports. Functionally, the previously separate AI surfaces (chat, recommendations, FAQ, help) have been consolidated into a single Reading Assistant page with multi turn conversation, intent classification, and grounded answers about the current user's loans and recent returns. Operationally, four new admin and staff pages (the overdue list with multi facet filters, the user management page with role tabs and a per user detail panel, the books catalogue with the add book form, and the staff loan history page) move circulation tasks out of the Supabase studio and into the dashboard. On the engineering side, several long standing issues were resolved during the sprint: the NotificationReads primary key was corrected, the ISBN-10 and ISBN-13 mismatch in active loan search was fixed, the camera scanner was made reliable for printed barcodes on glossy paper, the Azure AD sign out flow was fixed to force account selection on the next login, and the post merge build errors that had blocked the Vercel deployment were resolved.

What remains is captured in section 2.7. The most material outstanding items scoped for Sprint 4 are the Row Level Security migration across the remaining twenty five Supabase tables, the implementation of the `hold_ready` and `overdue` notification triggers, and the addition of end to end automated tests for the redesigned pages. The team's confidence in the Sprint 4 backlog is high because each of these items has a defined entry point in the codebase rather than requiring exploratory design.

The team's overall conclusion is that the system is ready for a controlled pilot deployment. A pilot would expose the application to real circulation patterns, validate the SIP2 best effort sync model under actual library load, and surface any usability issues that the manual testing rounds did not catch. The findings from the pilot would feed directly into the Sprint 4 backlog and inform any final hardening before broader rollout.

3 Sprint Progress

3.1 SPRINT DELIVERABLES

Among these four weeks of Sprint 3 progress, we tried to upgrade our entire system as much as we can before we start user acceptance testing, we were adding more practical features and redesign our entire system UI to make it highlight out the main features of our system. During our upgrading process, there are a lot of bugs found when merging branches, so we are forced to spend more time debugging and have to extend the implementation process. We planned to do the UAT later after we finalized the latest version and done internal testing. In conclusion, this part mainly talked about the changes we have made compared to the last Sprint.

3.1.1 UI Upgrade

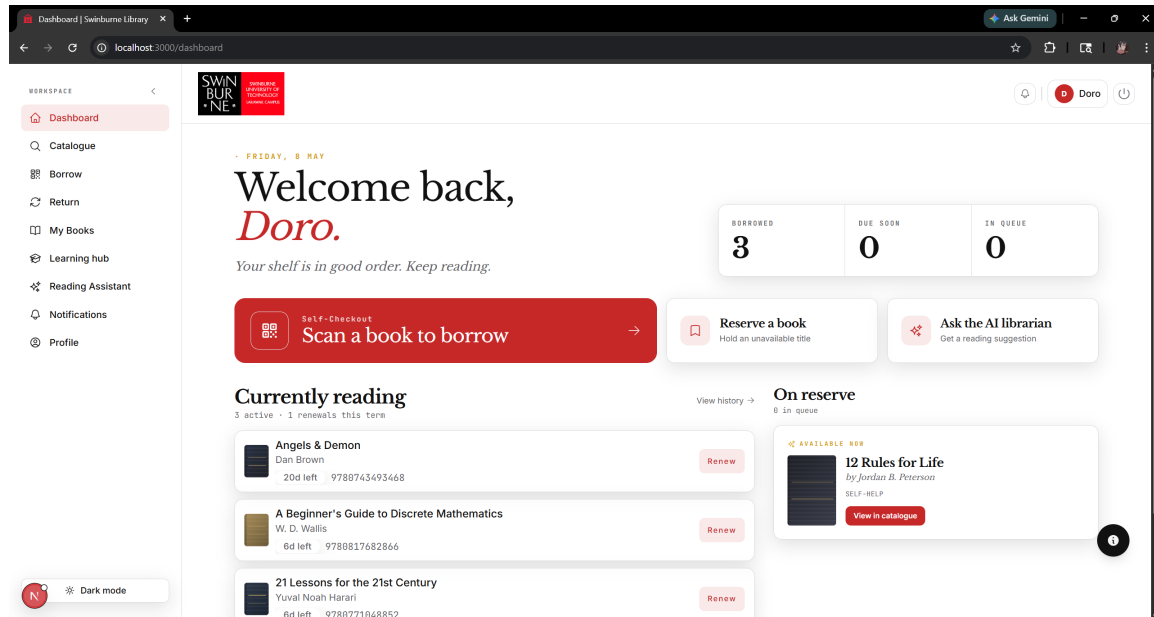


Figure 46 User Role Upgraded UI

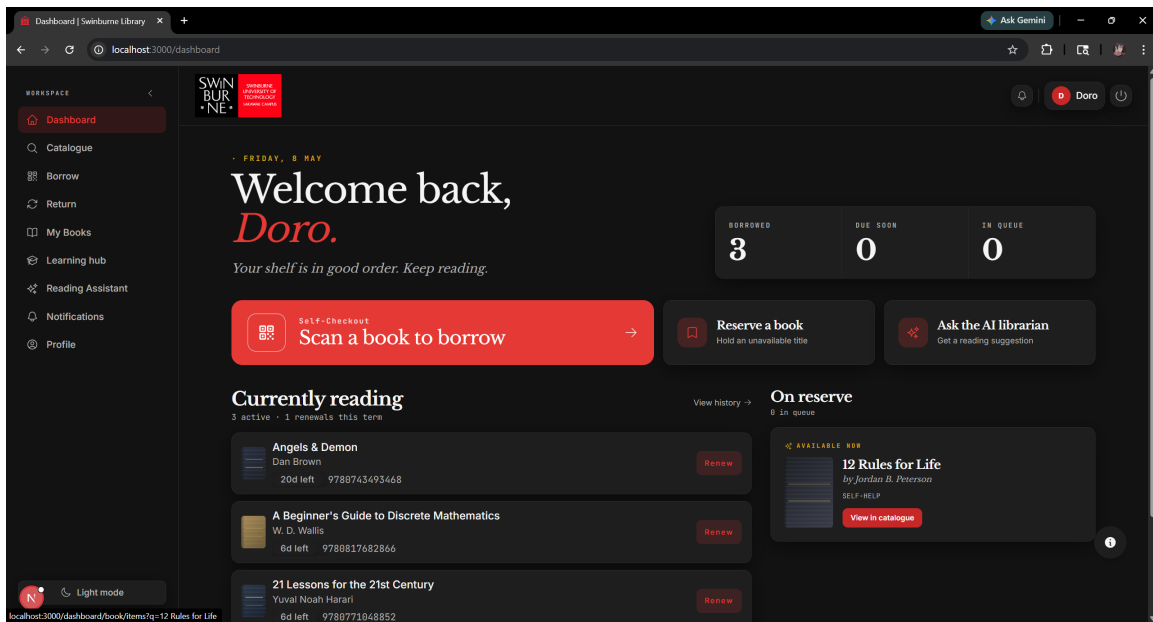


Figure 47 User Role Upgraded UI (dark mode)

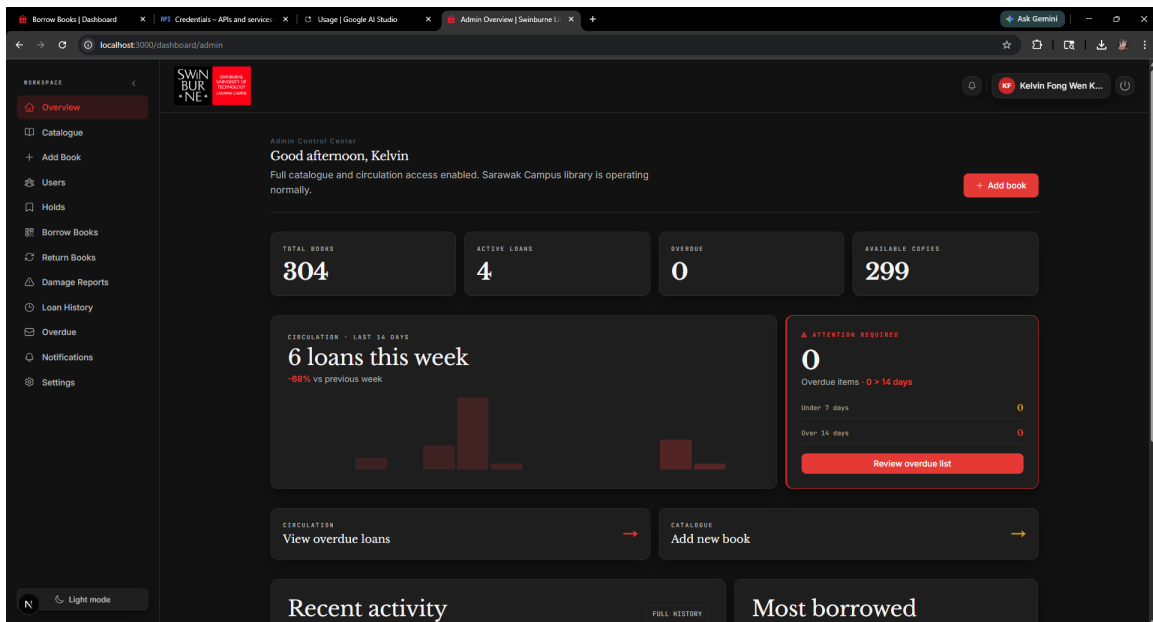


Figure 48 Dashboard (Admin View)

Got the full picture. Here's the actual UI Upgrade write-up for this branch: UI Upgrade This sprint focused on rebuilding two of the most heavily used surfaces on the site — the AI chat and the admin user-management page — and tightening up the global navigation. The biggest visible change is the new Reading Assistant. The previous monolithic chat component was retired and replaced with a redesigned conversational page composed of four small, focused parts: a Composer for typing, a MessageBubble for each turn, a BookList for inline book recommendations,

and a QuickPrompts row for one-tap shortcuts. Conversation history now persists across sessions via a new API endpoint, so students can return to the page and pick up where they left off. The chat also includes always-visible prompt chips and a Clear Chat button, replacing the older flow where prompts disappeared after the first message and there was no way to reset the conversation. Behind the scenes, the assistant pulls in the student's profile, active loans, and recently returned books before answering, so its replies feel personalised — but this is a chat-side polish, not a separate panel. The Admin Users page was rebuilt from scratch. The old `usersManager.tsx` was deleted entirely, and in its place is a cleaner read-only list (`UsersList`) with role-count tabs across the top, a tap-to-open detail flow, and a dedicated detail route at `/dashboard/admin/users/[id]`. Adding a user now happens in a modal `AddUserDialog` rather than a separate page, and editing a user happens on the new detail page through `UserDetailForm`, which surfaces the user's profile, their recent activity, and a clearly marked danger zone for delete actions. Reusable profile-field helpers were extracted so the form stays consistent with the rest of the site. A `RoleSelect` component replaced the older badge-plus-dropdown duplicate that used to live in the table. The sidebar navigation was tidied up too. The role and name badge that used to sit at the top has been removed (it duplicated information already shown on the profile page), and the collapse toggle is now in line with the rest of the navigation rather than floating awkwardly at the bottom. Together these changes give the sidebar a cleaner, less cluttered look at every screen size. A handful of smaller polish commits round out the sprint: the unnecessary search bar in one of the dashboard surfaces was removed, button placements were normalised across pages, the profile page got a visual update, the `BookCover` component was refined, and a sweep of theme refinements brought consistent spacing and colours across the redesigned surfaces.

3.1.2 AI-Fine-Tuning

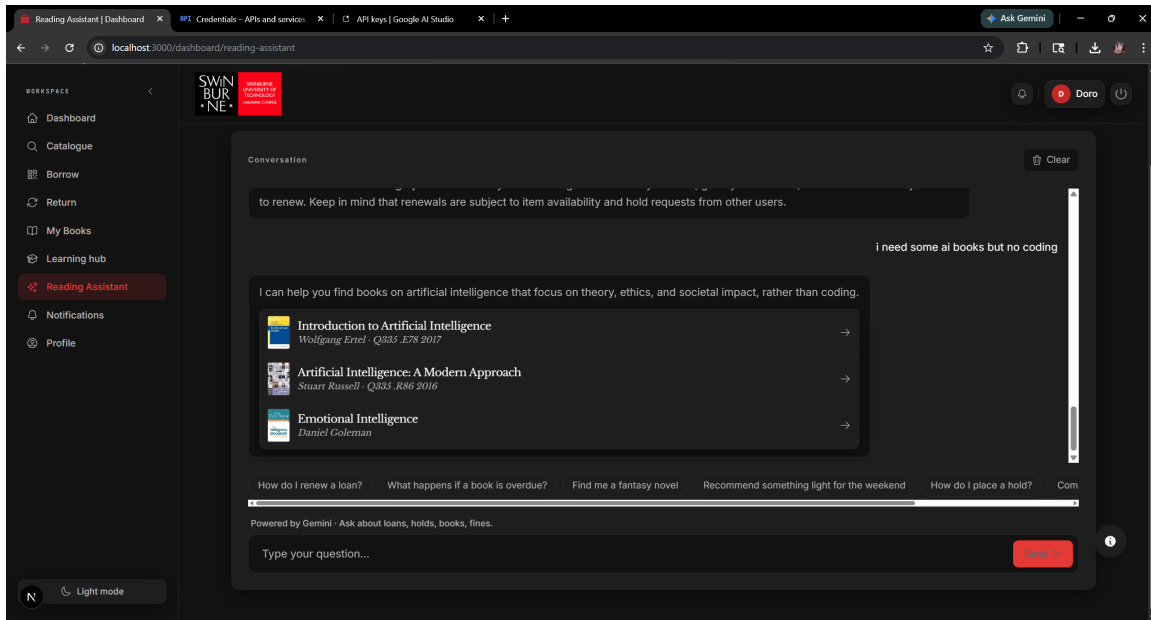


Figure 49 AI assistant is now more intelligent to detect keyword and user demand

We made the AI noticeably smarter and more reliable, such as smarter book searches. The system now understands the meaning of what users are asking, not just the exact words. So, if user search "AI books", it correctly finds books on machine learning and neural networks even if those titles never literally say "AI". Better topic understanding. We taught the AI with example inputs and outputs, so it pulls clean topics from messy phrasing such as "give me some AI books" gets read as artificial intelligence, machine learning, neural networks, instead of "give", "ai", "books". Acronym expansion. Common short forms are automatically expanded: AI → artificial intelligence, ML → machine learning, DB → database. Helps the search find more relevant books. Cleaner keyword extraction. Filler words like "give", "some", "recommend", "looking" are now ignored, so the AI focuses on the actual topics. We also added personal recommendations that explain themselves. When user ask "recommend books based on my faculty and interests", the system uses user's real profile (faculty, saved interests, recent borrows) and tells user exactly why each book was picked — e.g. "Based on your faculty in IT and your recent borrows of 'Deep Learning'...". Smarter quick prompts. When a user is asking for assignment books such as "what's available now", or "something interesting" each gets its own tailored response instead of a generic one. No duplicate suggestions. The AI won't recommend a book you've already borrowed. Lighter on resources. We trimmed wasteful background AI calls, so each chat message uses 3× less of our free monthly quota. AI provider switch. A toggle at the top of the chat lets user choose between the local AI (running on a computer) and Google's Gemini. If one breaks, the other steps in.

3.1.3 AI Chat History

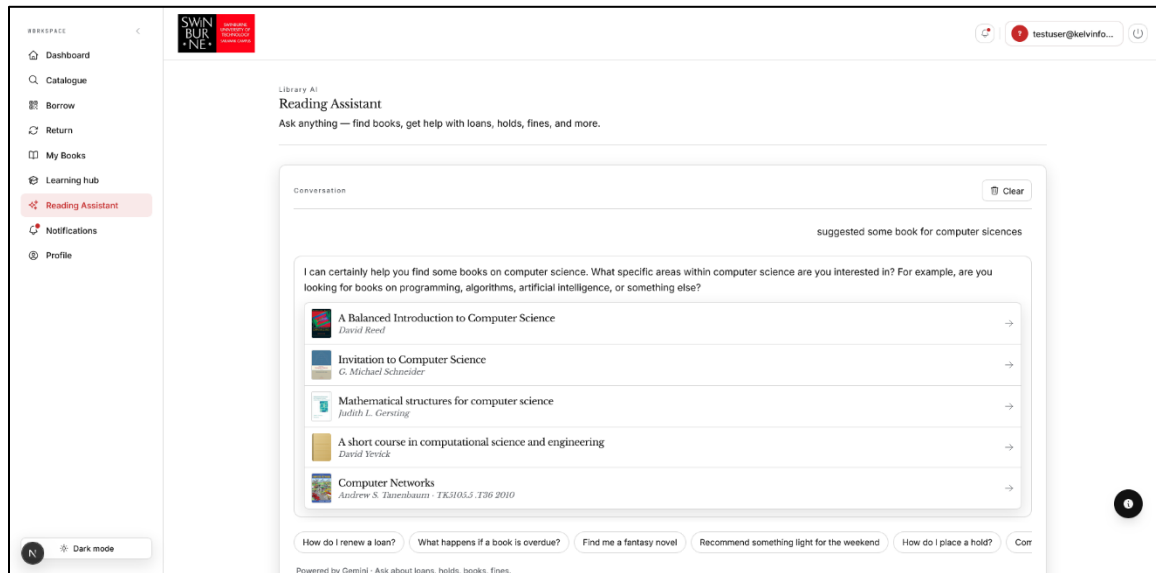


Figure 50: AI Chat Page

The AI chat page (Reading Assistant) uses a chat history storage system that allows users to save, load, and clear their conversations. This system works by interacting with a Supabase database, where each chat message is stored with details like user ID, message ID, sender (user or assistant), message text, and a timestamp. When a user sends a message, it's saved to the database, and when they open the chat, their previous messages are loaded. Users can also clear their entire chat history, which removes all their messages from the database. This functionality helps users keep track of past interactions, refer to previous recommendations, and ensures a personalized experience, making the AI assistant more helpful and efficient.

3.1.4 My Books

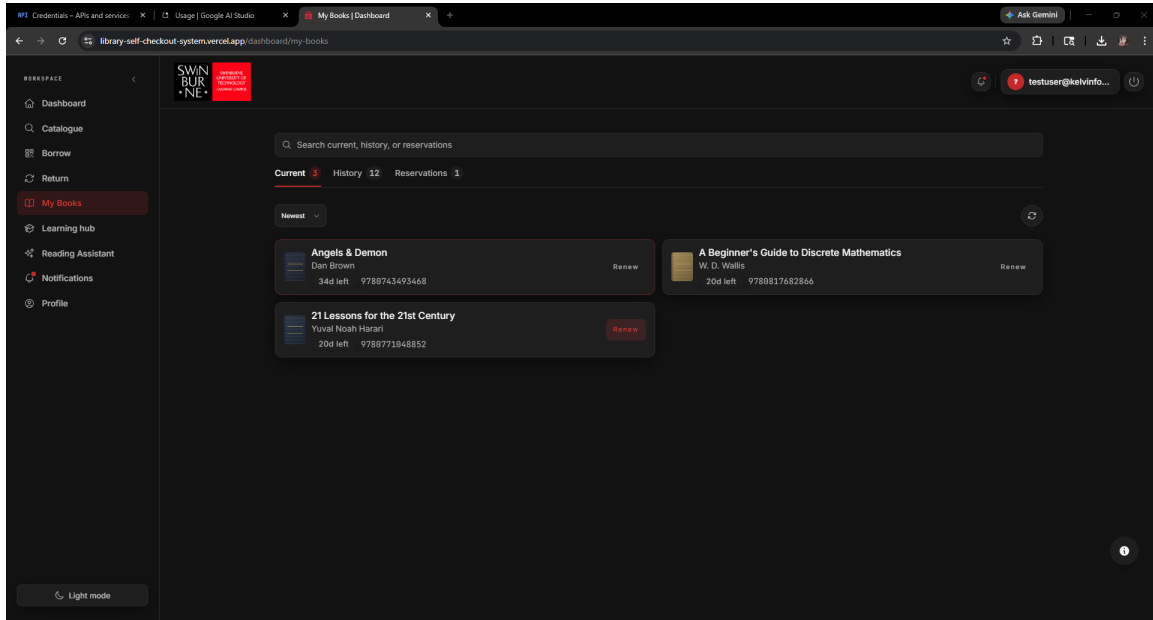


Figure 51 My Books: Current Loan

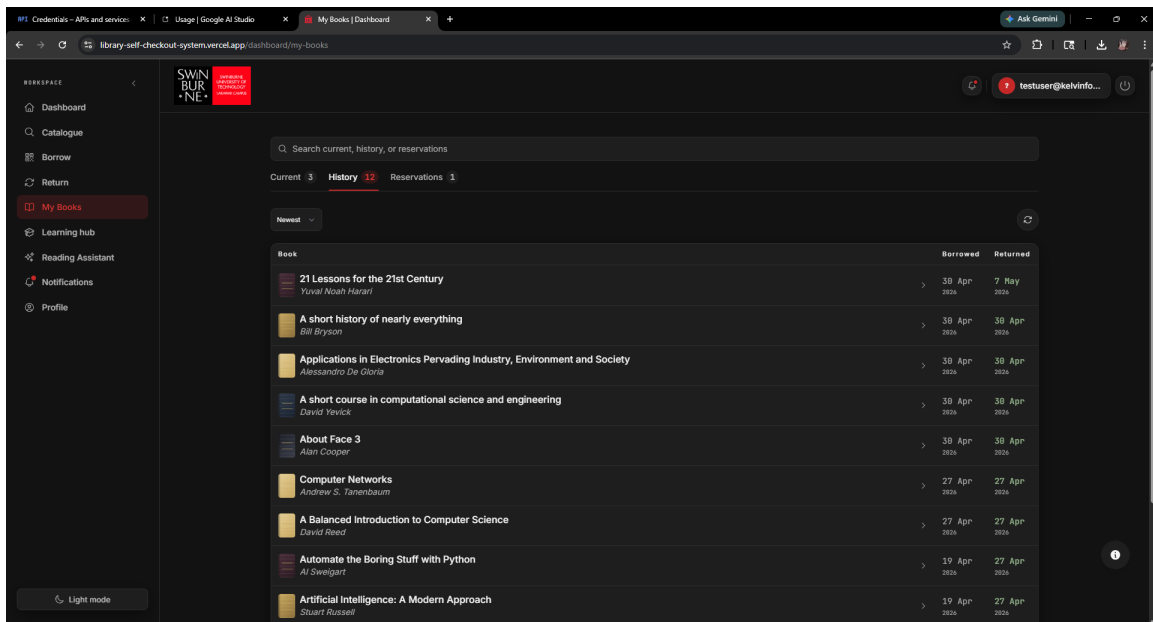


Figure 52 My Books: Borrowed History

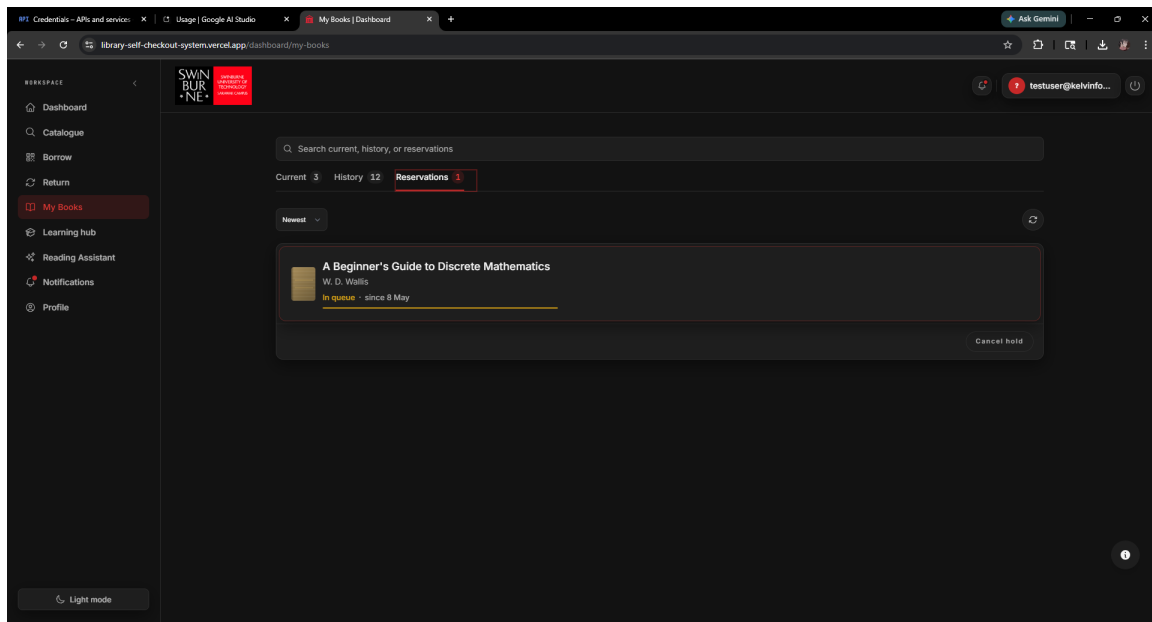


Figure 53 My Books: Reserve Book(s)

The My Books page at /dashboard/my-books is the student's personal hub for everything they currently have on loan, have borrowed in the past, or have reserved. It's restricted to student-role accounts — staff and admins are automatically redirected to their own dashboard, since they have separate views with broader scope. The page is organized into three tabs in a single switcher: Current, History, and Reservations. Each tab shows a count badge so the student can see at a glance how many active loans, past loans, or pending holds they have. The Current tab lists every book the student is presently borrowing, with renewal controls that check live hold counts on each book — if someone else is waiting for the same title, the renew button is disabled to keep the queue fair. The History tab shows the last 30 completed loans, with each entry expandable to reveal the full loan details. The Reservations tab lists active holds and offers a one-click cancel action that runs as a secure server action and immediately revalidates the page. Across all three tabs, students can use a single search bar at the top to filter by book title, author, or barcode, and a sort dropdown to reorder by Newest, Oldest, Title A–Z, or Title Z–A. A small refresh button sits at the right-hand side of the controls for when the student wants to pull the latest server state without navigating away. The active tab is also persisted in the URL via a ?tab= query parameter, so students can bookmark or share a direct link to their History or Reservations view, and keyboard users can move between tabs using the arrow keys. Visually, the page reuses shared primitives — LoanCard, HoldCard, and BookCover — so the styling stays consistent with the rest of the student dashboard. Each book card shows the cover (or a coloured gradient placeholder if no image is available), the title, author, key dates, and the contextual action available for that state (renew for current loans, cancel for reservations).

3.1.5 Notification

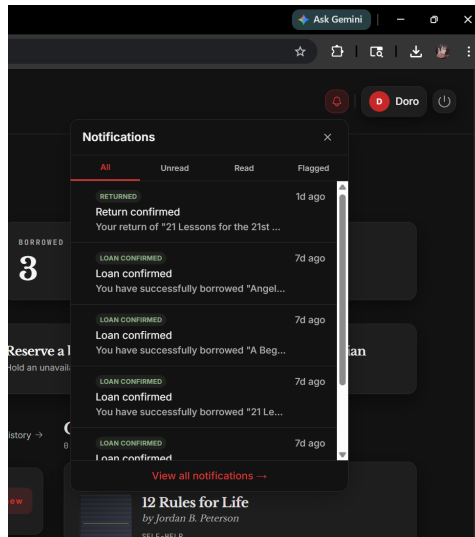


Figure 54 Notification window

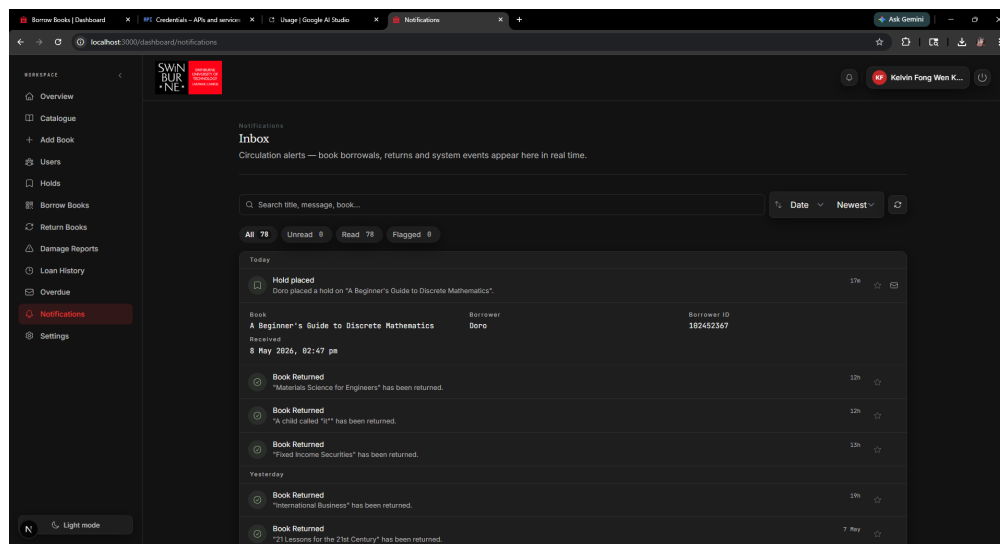


Figure 55 Notification page (message details)

New notifications small window added, allow user to check the notification message by clicking the “bell” button at the top right corner. Convenient for user to check the notification message without needing to go to the Notification page. User can still go to Notification page to check the details of the notification message

3.1.6 Reading Assistant

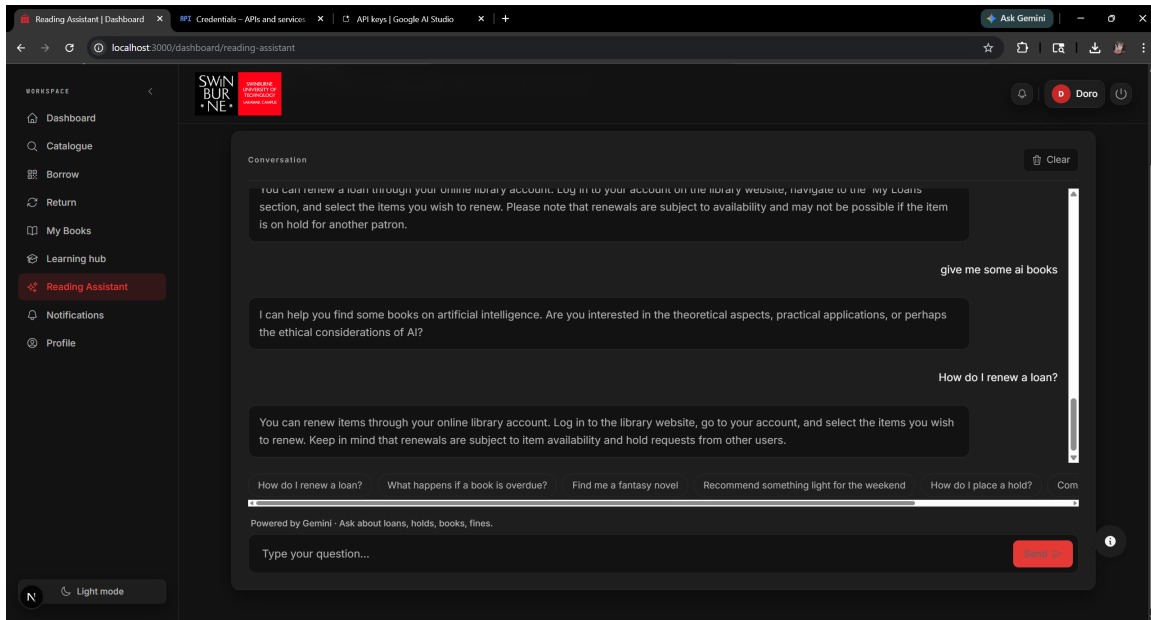


Figure 56 Reading Assistant AI help to answer renewal problems

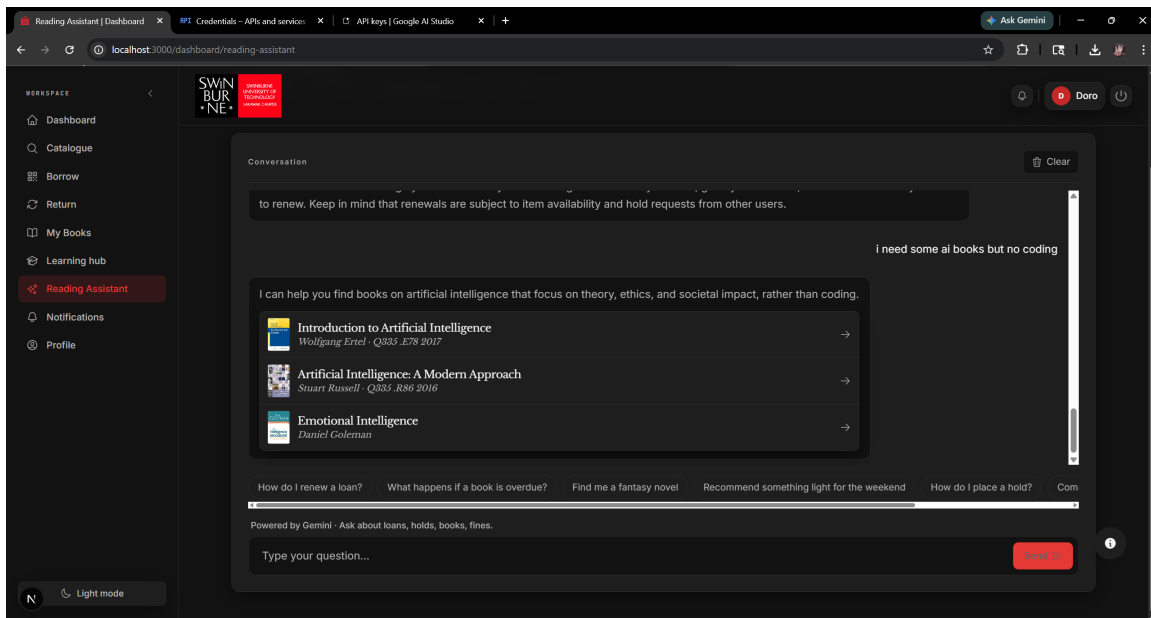


Figure 57 AI chat recommendation system

Cleaner code structure so the chat, the learning path, and the web resources stay independent, future updates to one won't accidentally break the others. Automatic follow-ups, which means as soon as the chat picks up user topic of interest, it automatically pulls a learning path and web resources for it, so user don't need to ask twice. We also make it smarter for greeting detection.

Messages like "yo i need business books" are no longer mistaken for a casual "hi", the chat correctly understands user want book recommendations. Borrow button on every recommendation card, now one click and user will be at the checkout with the book already chosen. And now the AI chat assistant can also answer technical questions like book due date renewal steps.

3.1.7 Learning Hub

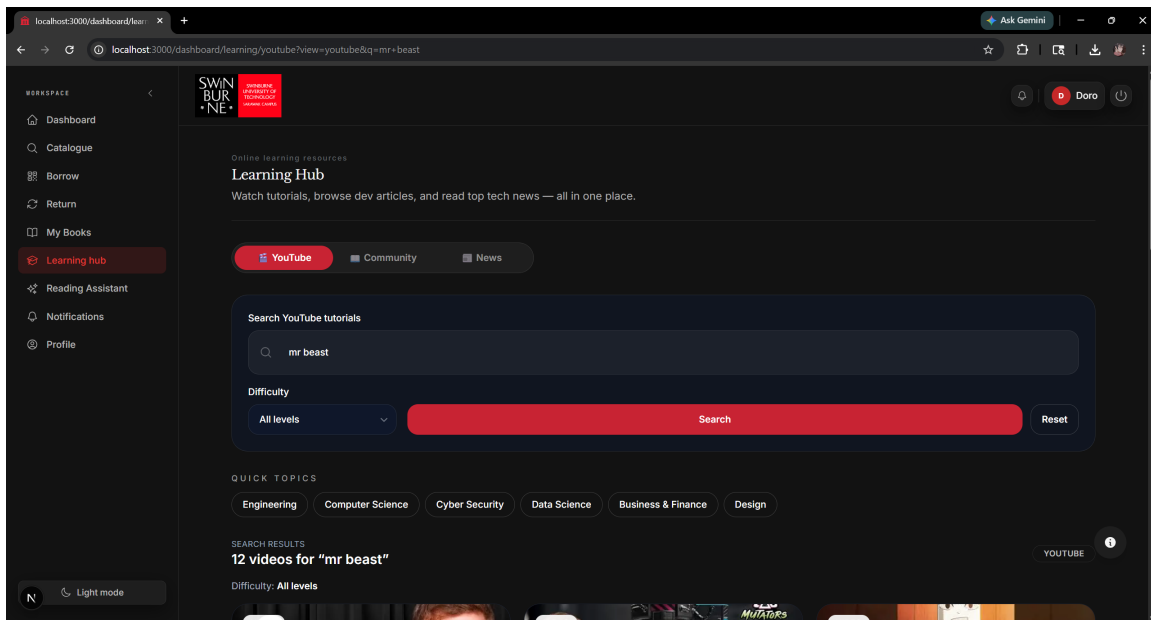


Figure 58 Learning Hub page

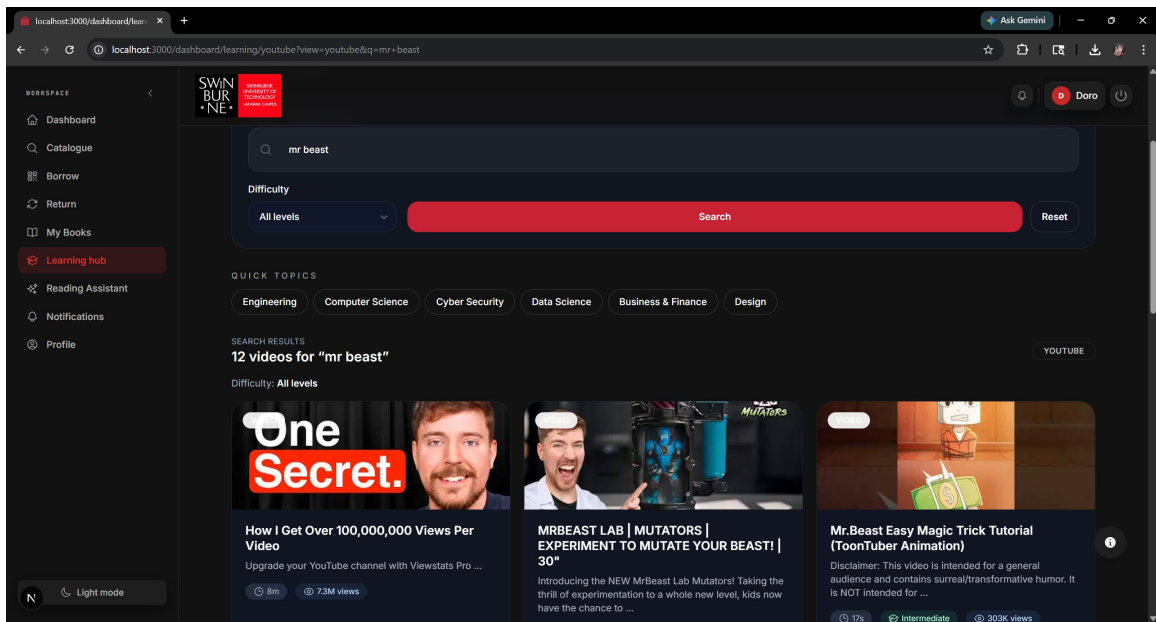


Figure 59 Learning Hub page search features (only show academic result)

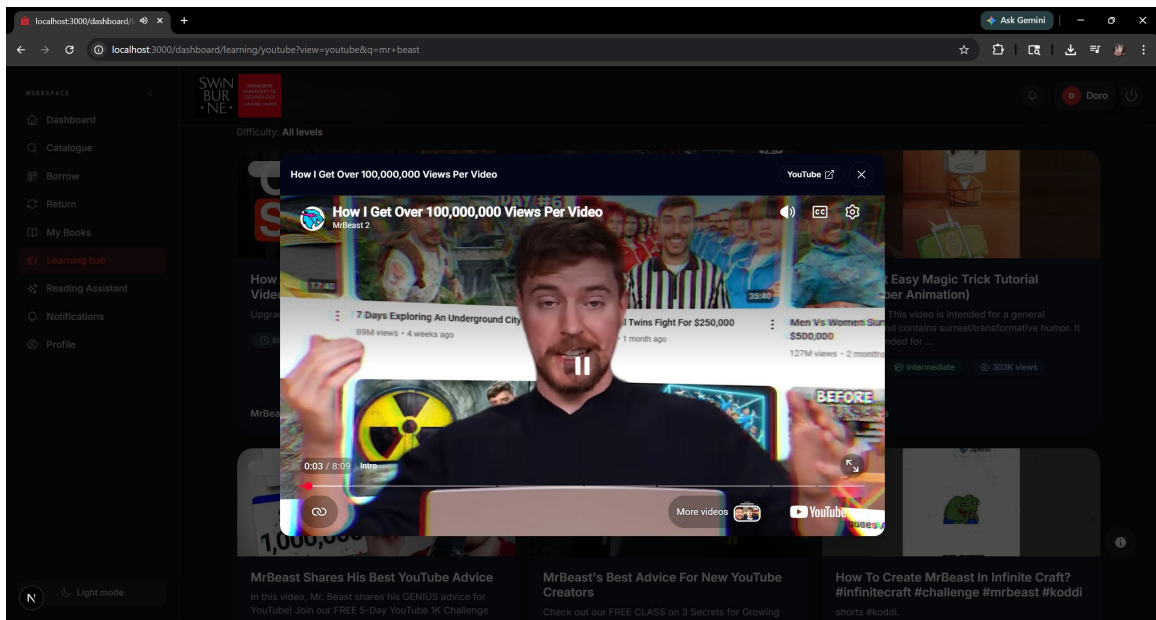


Figure 60 Learning Hub video play window

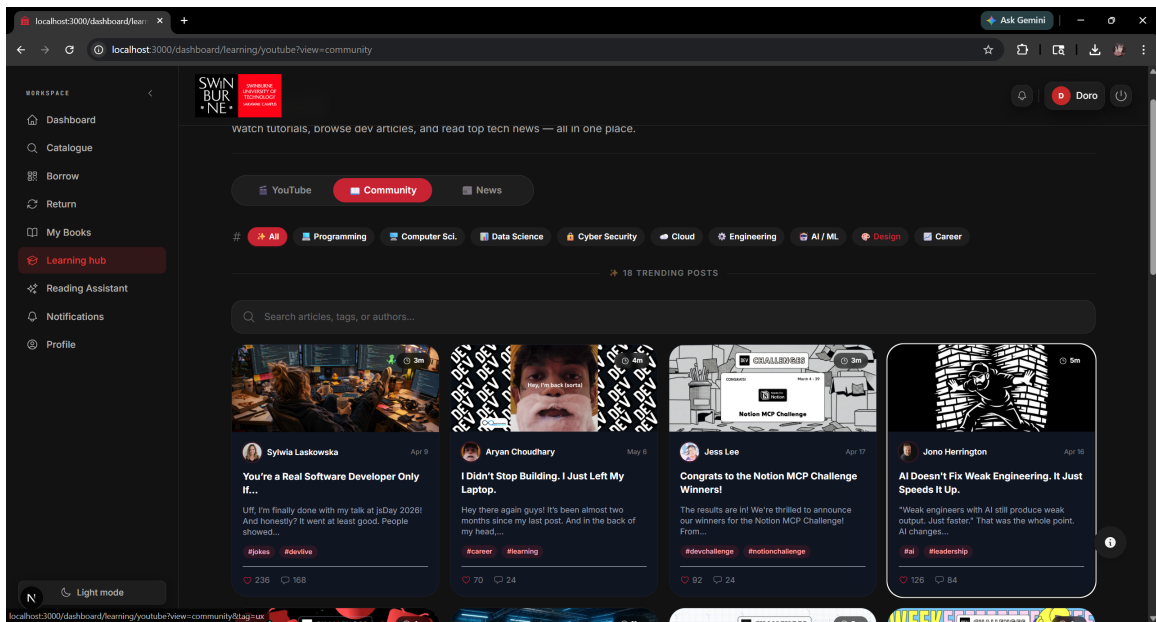


Figure 61 Learning Hub Community section

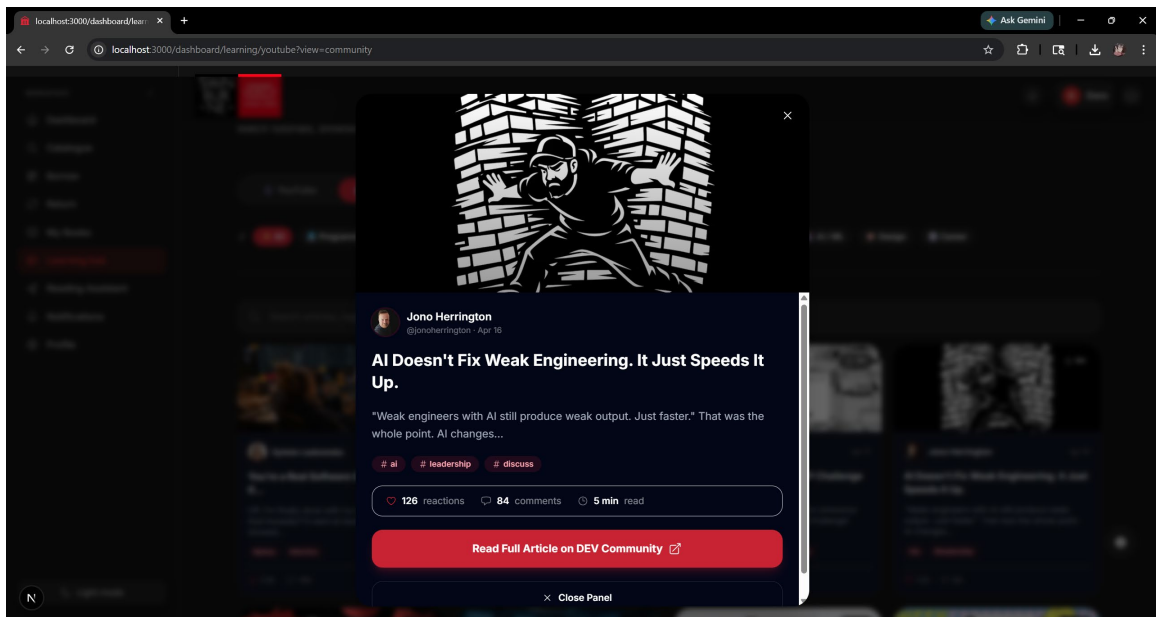


Figure 62 Community articles check details window

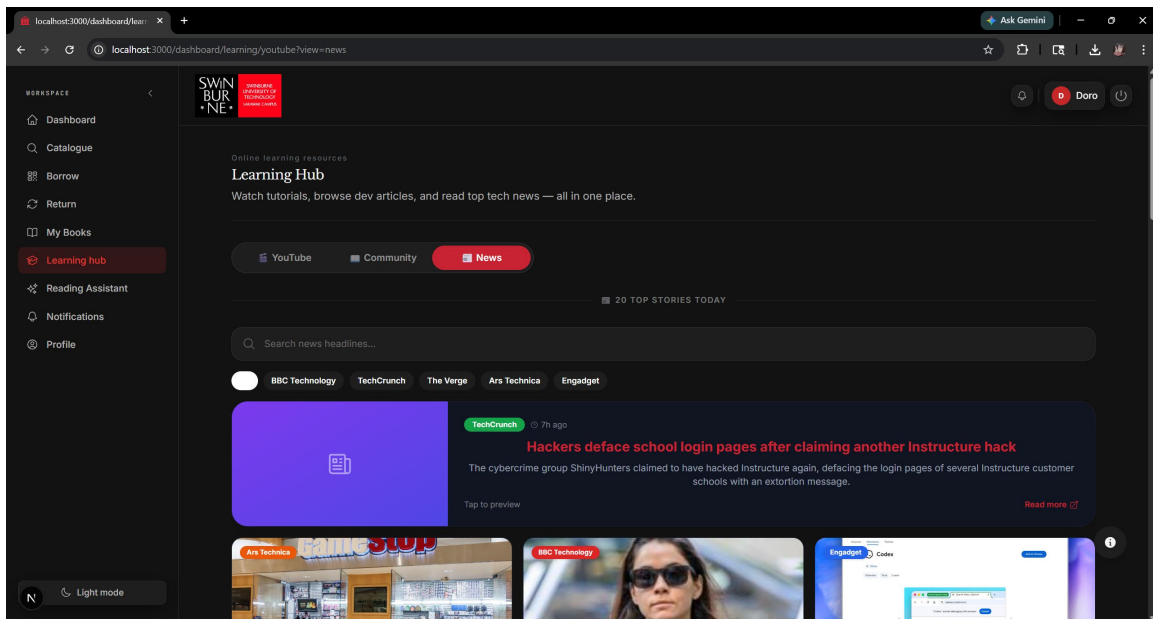


Figure 63 Learning Hub News section

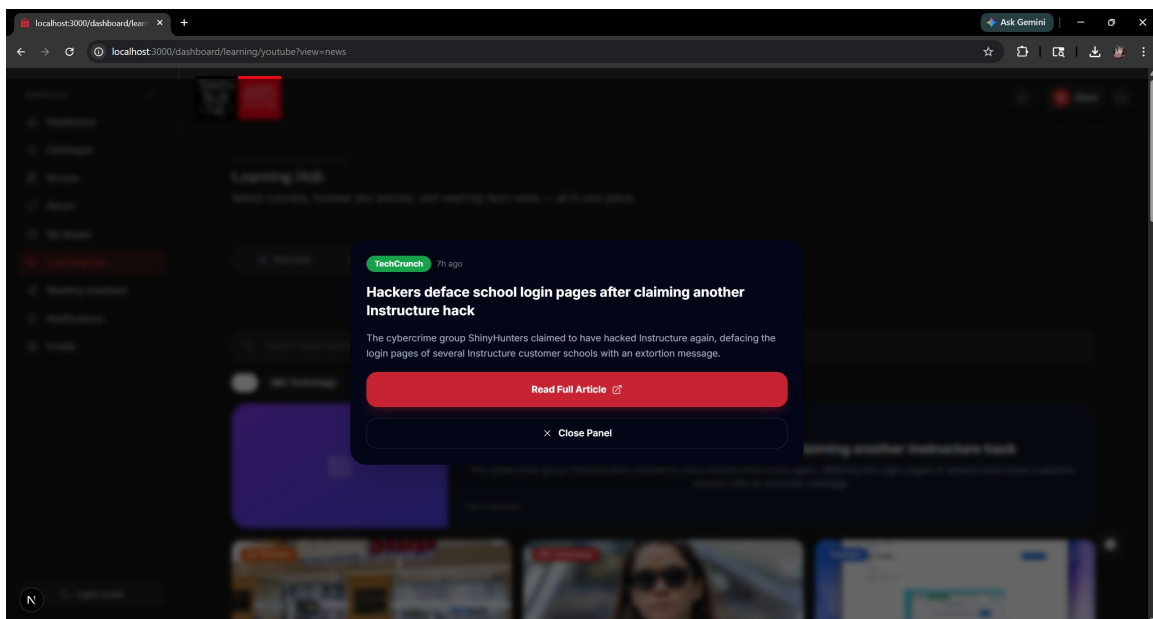


Figure 64 Learning Hub news details

The Learning Hub is a dedicated self-study page available to student accounts at /dashboard/learning. It acts as a one-stop browsing destination for online learning content, complementing the AI chat by giving students a place to explore tutorials, articles, and tech news on their own terms — without needing to type a query. The page is organised into three tabs in a single pill-style switcher. The **YouTube** tab is the default landing view: students can search for tutorial videos by topic and filter results by difficulty (Beginner, Intermediate, Advanced, or All

levels). Six one-click "quick topic" chips cover the most common faculties — Engineering, Computer Science, Cyber Security, Data Science, Business & Finance, and Design — and a "Trending" feed appears when no search has been entered yet. Live videos come from YouTube's official Data API v3; if no API key is configured, the page gracefully falls back to bundled sample videos with a clear "Demo mode" notice. The **Community** tab pulls trending articles from DEV.to, the developer community blog, and lets students filter by emoji-tagged categories such as Programming, Cyber Security, Cloud, AI/ML, Design, and Career. The **News** tab aggregates the latest technology stories from five major RSS feeds — BBC Technology, TechCrunch, The Verge, Ars Technica, and Engadget — interleaving sources so the feed feels varied. Both tabs cache their results (1 hour for articles, 30 minutes for news) to keep the page fast and reduce external bandwidth. Access is restricted to student-role accounts only; staff and admin users are automatically redirected back to their own dashboards. The Learning Hub is intentionally separate from the AI-generated learning path that appears on the recommendations page — the chat-based learning path is a focused, three-stage reading order for a specific topic, while the Learning Hub here is an open browsing space for general curiosity and ongoing self-study.

3.2 REPOSITORY AND WORKFLOW EVIDENCE

3.2.1 GitHub

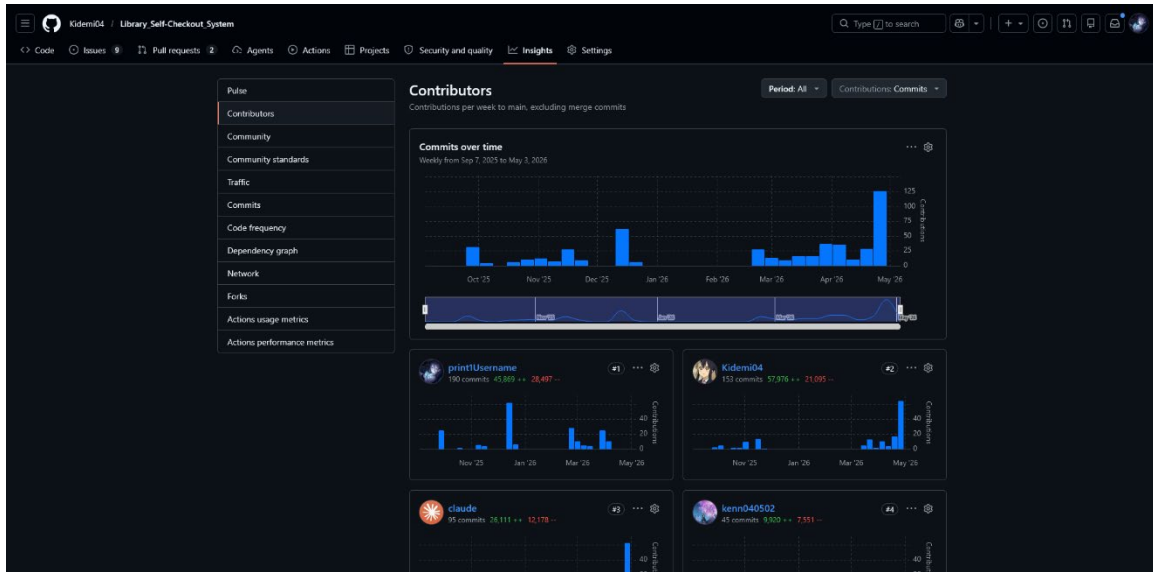


Figure 65 GitHub Contribution

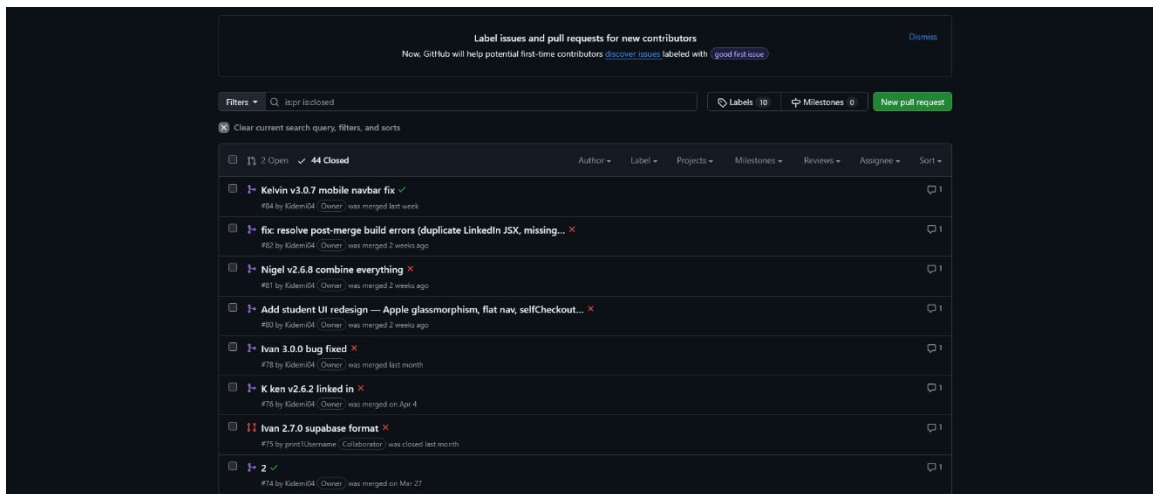


Figure 66: GitHub Pull Requests

3.2.2 WhatsApp Chat

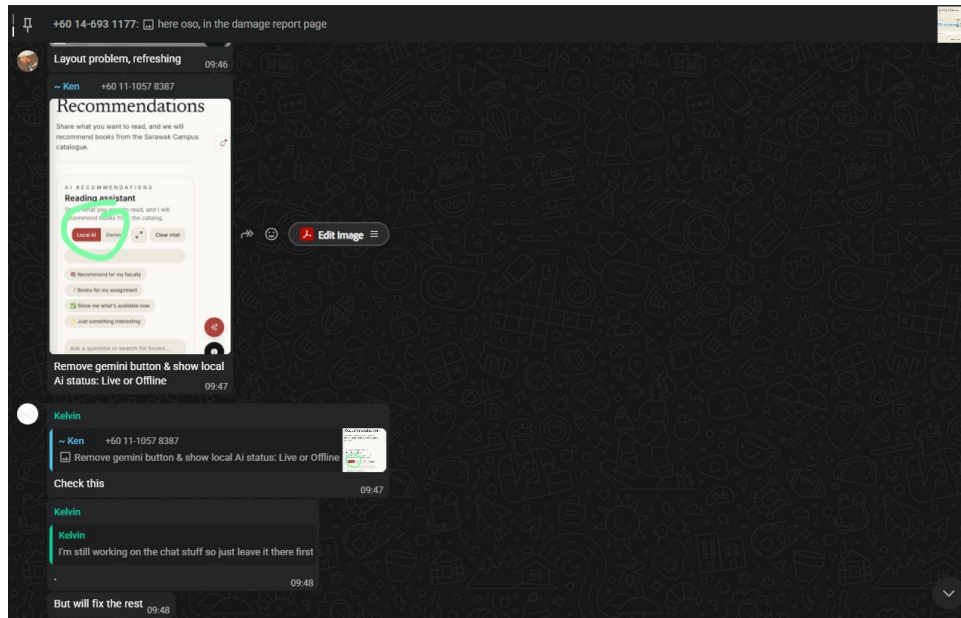


Figure 67 WhatsApp chat discussion (1)

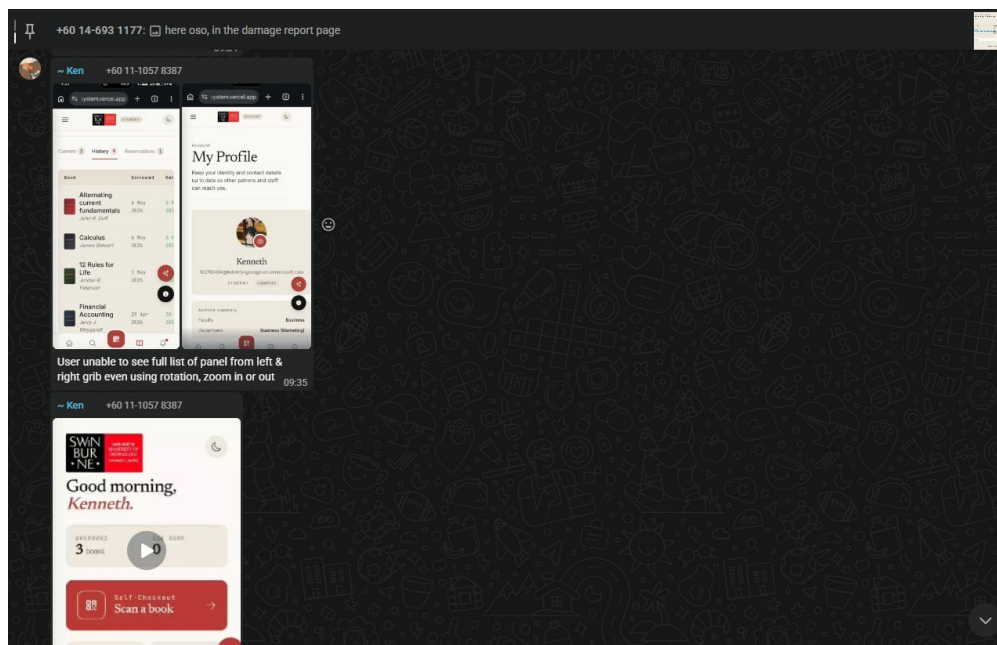


Figure 68 WhatsApp chat discussion (2)

3.2.3 Microsoft Teams

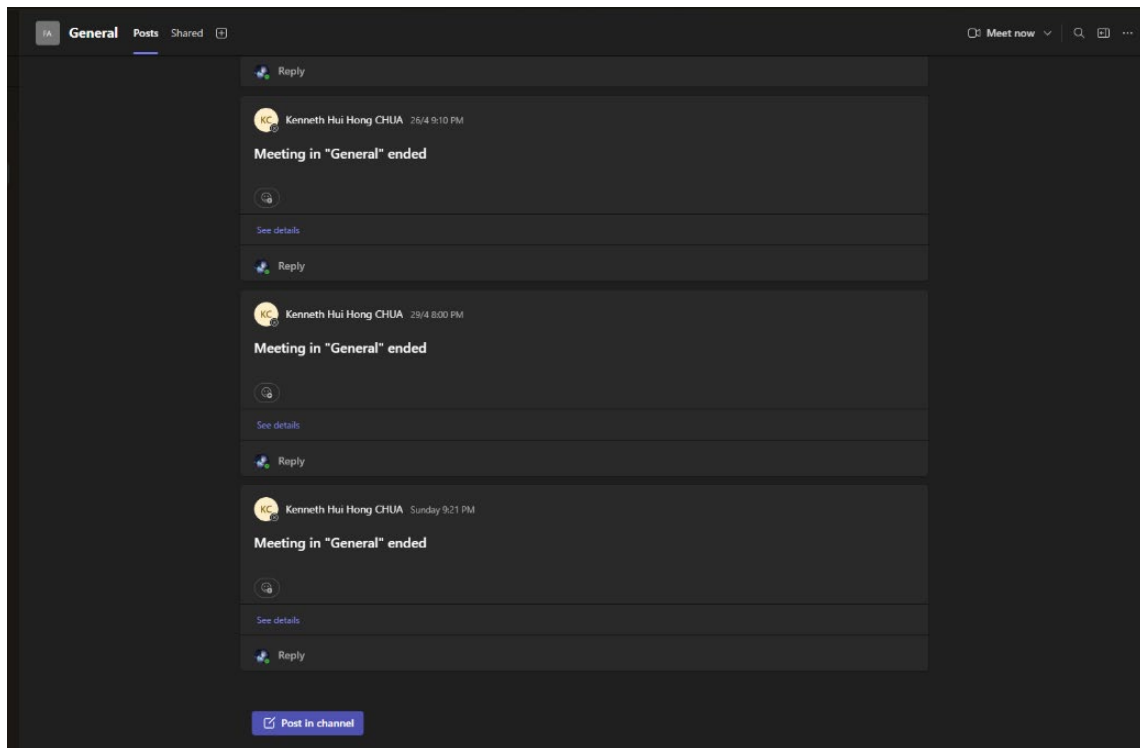


Figure 69 Microsoft Teams Meeting evidence

3.2.4 Linear

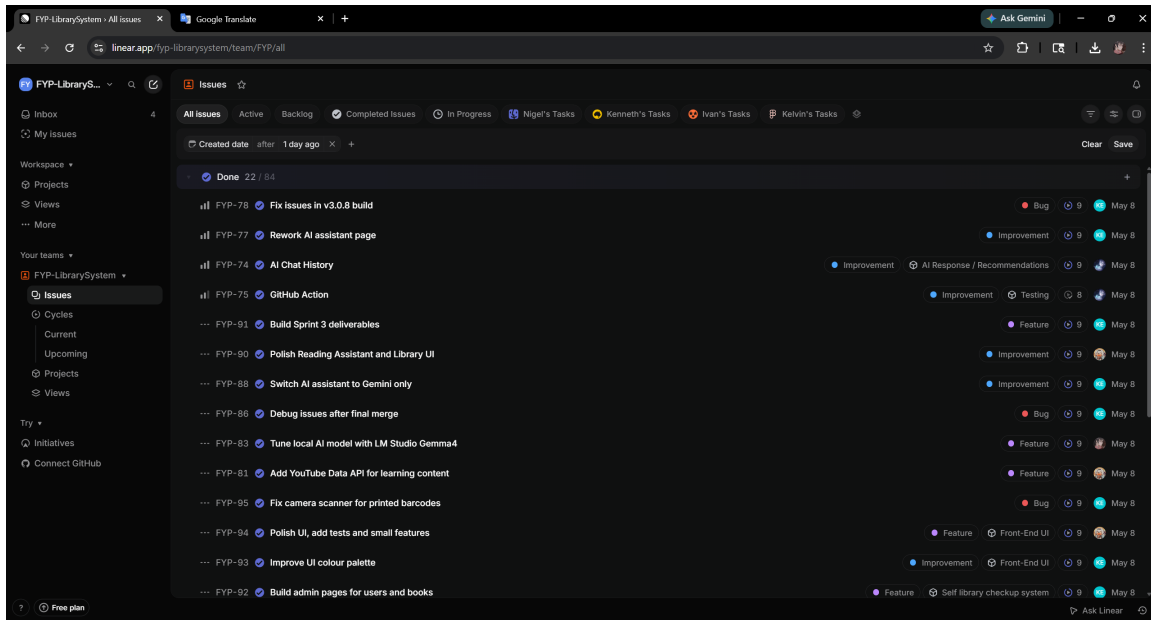


Figure 70 Linear Done Issues (1)

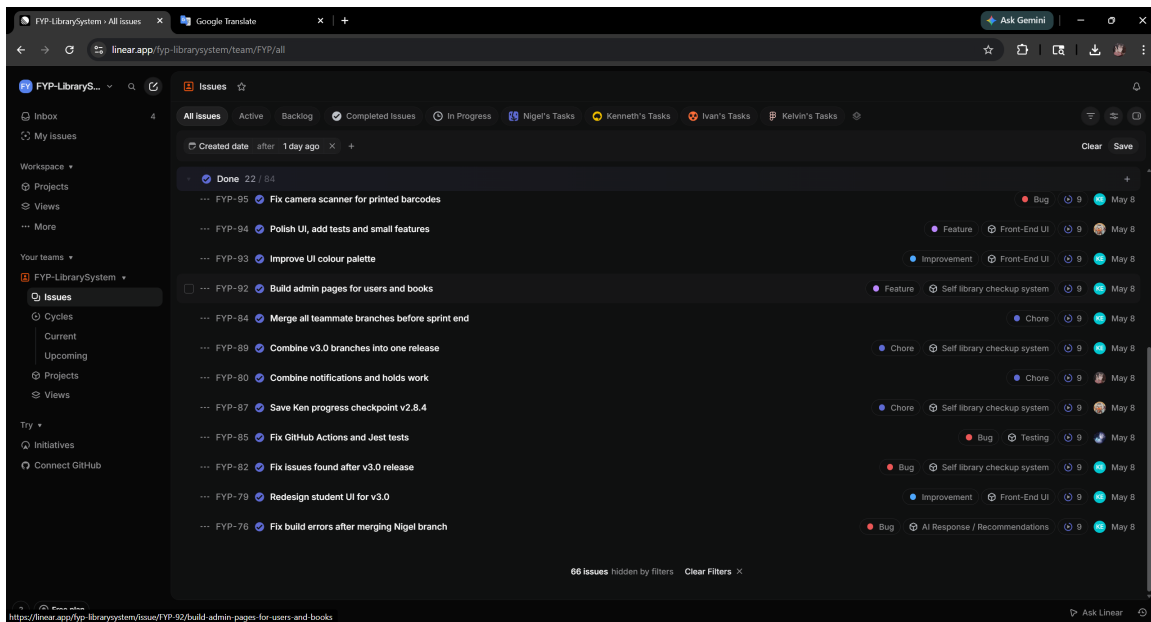


Figure 71 Linear Done Issues (2)

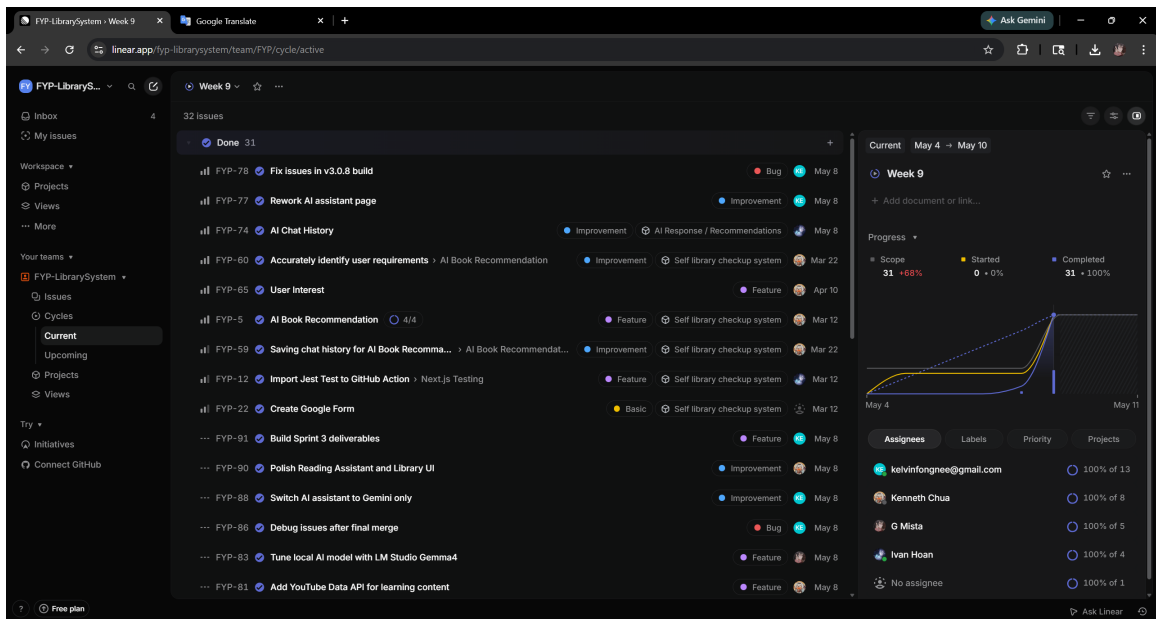


Figure 72 Linear Current Issue

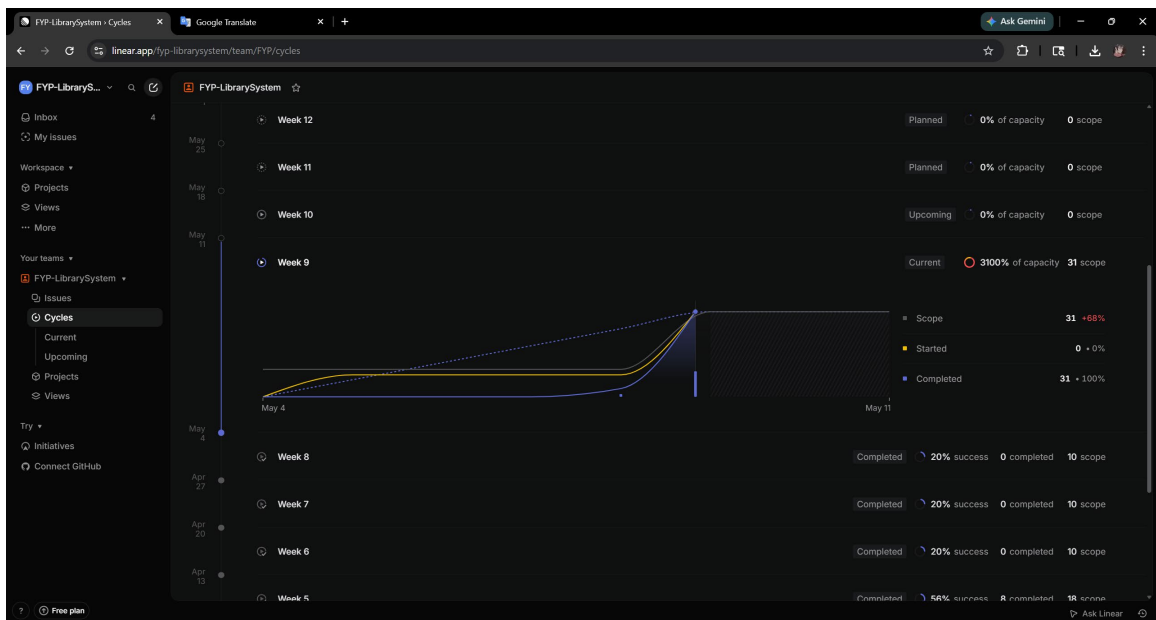


Figure 73 Linear Cycles

4 Sprint Review

4.1 DEMONSTRATION SESSION AND CLIENT COMMUNICATION

During this sprint, we were originally planning to do UAT around week 7, but we discover that our system is not practical enough and still have some problems, so we decided to upgrade our system. This costs us a lot of time to upgrading our system and fix the errors, because of this we have to extend the UAT phase to the next Sprint. But we still managed to let our friends (Swinburne students) to test out our prototype, even though it is not the final version yet, so we somehow still have collected some feedback and made some improvements based on it. So, for the demonstration session, we let our friends to use our device to test out our system in face-to-face way, and they will straight up give us advice in oral since we are not able to adjust our Google Form in time. We had let 5 Swinburne students to test out our system, each of them took about 10 minutes to try every feature in our system by having our teammates guiding them to make sure they don't accidentally mess up important settings. The entire demonstration session is conducted smoothly and successfully; we had record their feedback but not in a formal way (Google Form).

During this print, we are not able to perform our final complete version to client and our unit convenor Mr. Brian since we are not able to complete it intime. We decide to perform our final product to them in the next Sprint with a formal way, such as online presentation video to demonstrate our system and features to them.

4.2 FEEDBACK RECEIVED

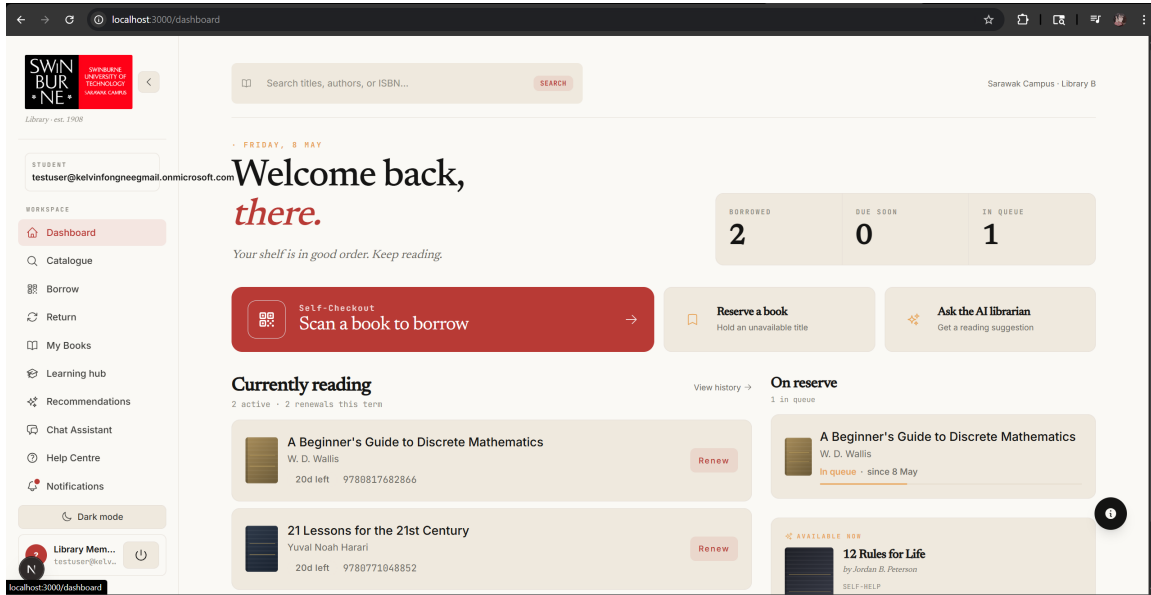


Figure 74 Old version dashboard (light mode)

From the tester feedback, they think that our new UI design is not good enough, such as we were using pearl white as our light mode base colour, one of the testers think that this is not suitable for Swinburne library and suggest us to change it to pure white. Also, they think we separate the page into too many small parts and it is very inconvenient for user because they need to switch page just to use another small features, for example, they think the Chat Assistant page (AI chat to answer technical questions such as steps to borrow book), Recommendations page and Help Centre could combine into 1 pages that let the AI model could do all the jobs combined.

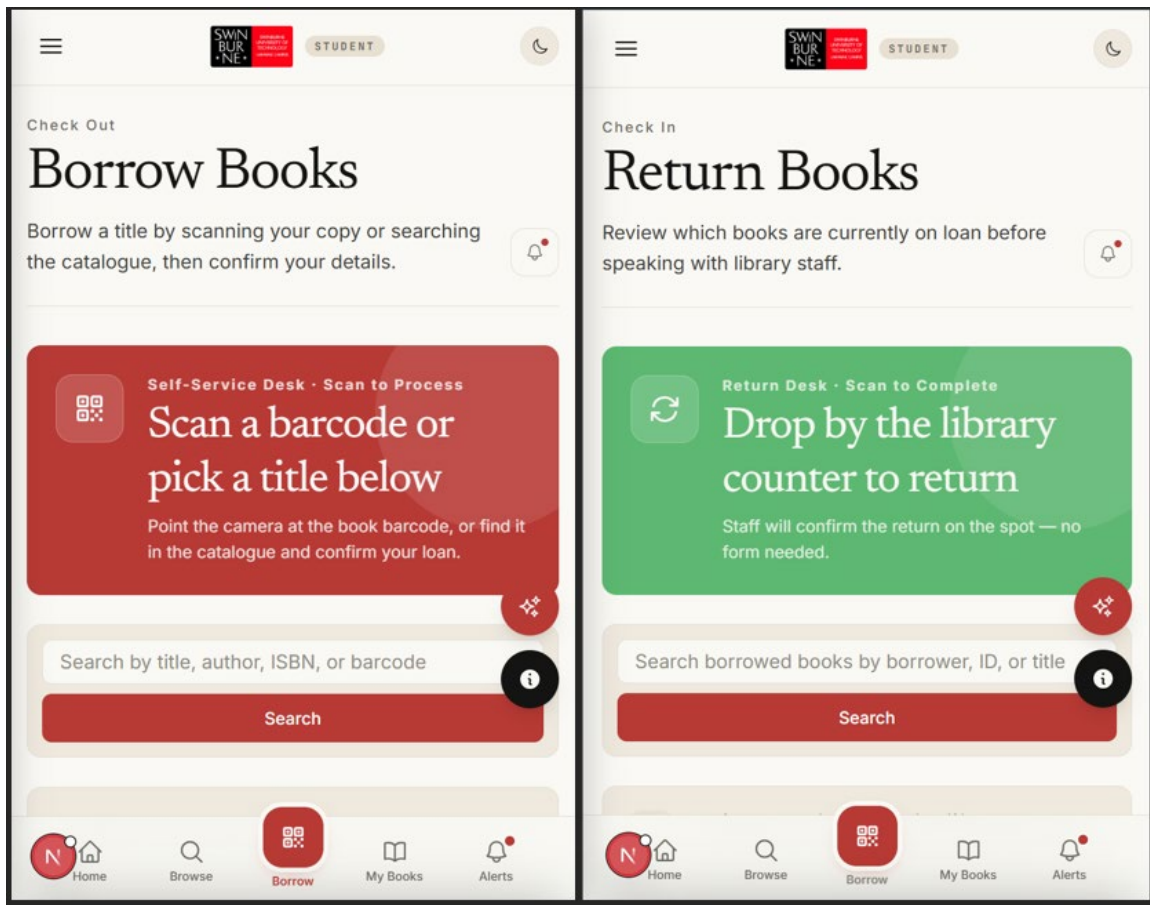


Figure 75 Old version of Borrow and Return Books page (mobile view)

For the “Borrow Book” and “Return Book” pages the big red and green grid in in top of the pages are very confusing, it looks like a button for them to click but it is just a showing panel that shows instructions to the users, so they suggest us to change it. Then, the search bar is both these two pages are also not functioning, they both just bring user back to Book catalogue page which is a duplicate feature as the search bar in Catalogue page.

For our client part, they finally answer our questions, they want us to focus on the main purpose of this system, which is the process of Check-in and Check-out. Also, they want required us to provide presentation video to updates our progress to them instead of face-to-face meetings to save more times.

4.3 CRITICAL ANALYSIS OF PROGRESS

Progress Achieved

We rebuilt large portions of the user interface this sprint to make the system feel more cohesive and student-friendly. The most visible change is the redesigned **Reading Assistant** at `/dashboard/reading-assistant` (the old `/dashboard/recommendations` URL now redirects here for backwards compatibility). It is composed of small focused leaf components — Composer for typing, MessageBubble for each turn, BookList for inline book cards, and QuickPrompts for one-tap shortcuts — and conversation history is now persisted server-side via the `/api/generalChatHistory` endpoint, so students can return to the page and pick up exactly where they left off. Sitting beside it is the new **Learning Hub** at `/dashboard/learning`, a one-stop self-study destination organised into three pill-switched tabs: a YouTube tab for tutorial videos with difficulty filtering and quick-topic chips, a Community tab pulling trending developer articles from DEV.to (filterable by Programming, AI/ML, Cyber Security, Cloud, Career and other categories), and a News tab aggregating the latest tech stories from five major RSS feeds — BBC Technology, TechCrunch, The Verge, Ars Technica, and Engadget. Both pages use shared design tokens for spacing, colour, and typography, so the redesigned surfaces feel like a single product rather than a patchwork of features.

The Reading Assistant itself is far smarter than the previous version. It uses **Gemini 2.5 Flash Lite** as its AI backbone, running through a single classify-and-extract call that detects what kind of question the student is asking and routes accordingly. The system now recognises five intents: `find_books`, `answer`, `both`, `greeting`, `off_topic`, and a new `loan_status` intent specifically for questions like *"when is my book due?"* or *"can I renew?"* — answered using the student's real loan data instead of generic AI guesses. Every AI call is now multi-turn, sending the conversation history along with the student's profile (faculty, intake year), active loans, and recently returned books as system instructions, so replies feel personalised — for example, *"Since you've been reading 'Deep Learning', here are some related books on neural networks."* JSON output is forced via Gemini's `responseMimeType` parameter for reliable parsing, and we deduplicate against books the student already has on loan or has recently returned so the same title is never recommended twice.

We invested significant effort in **comparing AI providers** before committing to one. Initially we wired up both a Local LLM path (LM Studio running Gemma 4 e4b on a developer machine) and a Cloud API path (Google Gemini), with a runtime toggle that let us switch between them. After hands-on testing we concluded that the cloud path was clearly the better choice for our project: the Gemini API gave more accurate keyword extraction, produced more natural reply text, returned valid JSON consistently, and required zero infrastructure beyond a single environment variable. The local LLM path, by contrast, demanded that a host PC run continuously as a live server and load a multi-gigabyte model into memory — a setup we cannot realistically guarantee for a deployed library system that needs to be accessible 24/7. Based on that conclusion we ultimately **dropped the LM Studio code path entirely** in a final refactor (refactor(ai): drop LM Studio path, become Gemini-only), simplifying the codebase and removing the always-on hardware dependency. To stay within Gemini's free tier we also trimmed the number of API calls

per chat from three down to one by removing a redundant health-check ping and a separate LinkedIn course generator.

Beyond the AI surfaces, the **admin and student dashboards also received polish**. The admin Users page was rebuilt from scratch — replacing the older monolithic `userManager.tsx` with a cleaner `UsersList` component, role-count tabs, an `AddUserDialog` modal, and a dedicated `/dashboard/admin/users/[id]` detail route showing each user's profile, recent activity, and a clearly-marked danger zone for delete actions. The sidebar navigation was tidied up by removing the duplicate role-and-name badge and inlining the collapse toggle. The student-facing **My Books** page at `/dashboard/my-books` consolidates active loans, loan history, and active reservations into a single tabbed view with shared search, sort (Newest, Oldest, Title A–Z, Title Z–A), and refresh controls, plus URL-persisted tab state and keyboard arrow navigation between tabs. Renewal logic on the Current tab now checks live hold counts so a student cannot extend a loan when other patrons are waiting on the same book, keeping the queue fair.

Finally, we ran **informal user acceptance testing with a small circle of friends** during the sprint to gather early reactions to the new layouts, the chat flow, and the visibility of the Borrow buttons. While the feedback was useful for catching small ergonomic issues — such as the chat panel feeling cramped when no sidebar was rendered alongside it, and the borrow flow being unclear when the button only led to a search results page — none of this counted as formal UAT. The next sprint is therefore reserved for **formal UAT**, where we plan to put together a structured test plan with predefined task scenarios, success criteria, and a feedback collection form to validate the system against real student usage patterns before the project's final delivery.

Challenges faced

The most critical problem we had occurred is the system had a lot of major bugs after merging all the branch from different teammates together, this had severely slowed down our progress, forcing us to postpone UAT.

We had occurred a lot of bugs and error while upgrading our system, such as the search bar problem in Return and Borrow pages that we stated out in above, it should be used as finding out the specific user or records, but instead it just redirect user to the Catalogue page which act the same as the book search bar in the Catalogue page. Furthermore, we some of the buttons are weird in mobile view, it looks like it does not resize properly, and some long words like email address had overflowed. Also, there is a lot of buttons are not-functioning at all while clicking it or it is not functioning as the desired output, such as the Add books button in staff and admin pages and does not redirect to the “Add new book” page, but instead, it redirects users to Catalogue page and do nothing. Not only that, but the camera scanner is also not functioning properly, it might detect human as the barcode. These types of errors and bugs are everywhere in our upgrade versions, so we were mainly fixing them during this sprint.

The biggest obstacle this sprint was the AI quota problem and we had spent too much time on it. Google quietly tightened the free tier for `gemini-2.5-flash-lite` to only **20 requests per day per project**, which our team blew through within a handful of test conversations because the original code was making three Gemini calls per chat message (a health-check ping, the actual classify-

and-extract call, and a separate course-suggestion call). We attempted several workarounds before landing on a working configuration. Our first move was to switch the model to gemini-2.0-flash, which Google's documentation advertised as having a 1,500/day free tier — but on our project the API kept returning limit: 0, locking us out of the model entirely despite the published docs. We then tried creating a brand-new Google Cloud project and generating a fresh API key under it, hoping a clean project would inherit the higher default quotas, but the new project also returned limit: 0 on gemini-2.0-flash, suggesting Google has been silently throttling free-tier model access on a per-project basis with no public explanation.

We also explored switching to a completely different web-search and AI provider stack to reduce reliance on Gemini. We initially tried **Brave Search API** for the article suggestions panel, only to discover their truly-free tier had been removed and any usage now requires a paid subscription. We then evaluated **Serper.dev** (2,500 one-time free queries, then paid) and **Tavily** (1,000/month free) as alternatives — both technically usable but ultimately freemium with monthly costs once exceeded, which was not sustainable for a long-term student project. We considered **Groq** as a free Gemini replacement (14,400 requests/day on Llama 3.3 70B), but settling on a non-Google provider meant maintaining a second API integration and didn't solve the underlying lesson — that any free cloud AI quota is fragile and can change at any time. We even briefly debated rotating between multiple Gemini API keys but rejected this approach because it potentially violated Google's Terms of Service and risked getting all our accounts banned.

Eventually we **circled back to Gemini** as our final choice, but with a different model: gemini-2.5-flash-lite, which still had a usable free quota on our project (around 1,000 requests per day). To make sure we never run out again, we **rebuilt the request flow to use only one Gemini call per chat message** instead of three — removing the redundant health-check ping and the LinkedIn course generator that previously ran in the background. We also added a deterministic fallback path so that even when Gemini hits its daily ceiling or returns an error, the catalogue search, learning path, and external article panels keep working without the AI in the loop. The whole experience was a useful lesson in not relying on third-party free tiers for production-critical features and reinforced our earlier decision to drop the local LLM path: the cloud API was unreliable in its own way, but at least it didn't require an always-on personal PC to function.

5 Retrospect (Critical Review of the Process)

5.1 WHAT WENT WELL

Three areas of the Sprint 3 process worked clearly better than in earlier sprints.

1. Continuous integration with GitHub Actions caught issues early

The CI workflow at `.github/workflows/test.yml` runs both the production build (`Build_Nextjs`) and the Jest test suite on every push. The team set this up at the start of Sprint 3 to replace the earlier process of running tests only on the developer's local machine. Any team member could push and immediately see whether their change passed in the cloud environment, not just on their laptop. This caught several integration issues during the sprint that would have only surfaced after merge under the old process.

2. Clearer division of work and smoother team communication

In Sprint 1 and Sprint 2 the team had occasional confusion about who was working on which feature, and overlapping changes would surface only at merge time. In Sprint 3, the team explicitly assigned each major feature (UI upgrade, AI fine tuning, Learning Hub, notifications, loan history, chat assistant) to a specific owner at the start of the sprint and tracked the work in Linear. WhatsApp handled fast informal coordination and Microsoft Teams handled scheduled meetings. The discipline of writing assignments down at sprint start, rather than relying on a verbal kickoff, made the difference.

3. A unified UI design specification applied across the system

Before Sprint 3, different parts of the dashboard used different colour palettes, button styles, and spacing conventions. During Sprint 3 the team adopted a unified design specification (the warm cream and charcoal palette in `tailwind.config.ts`, with Framer Motion animations for state transitions) and applied it across every page. The design specification was held in a single place rather than duplicated across components, so new components automatically picked it up. The result is a visibly more polished system that the client commented on positively during the demo.

5.2 WHAT DID NOT GO WELL

Two process problems came up repeatedly during Sprint 3 and need to be analysed honestly because they affected the team's pace and the quality of integration work. Both have real, identifiable root causes that point to specific changes for Sprint 4.

1. Estimation was off and several tasks ran beyond their planned duration

Several Sprint 3 tasks took noticeably longer than the team had estimated at sprint planning. The pattern was not random; it concentrated in two types of work: integration work (anything touching SIP2, Azure AD, or Gemini) and UI redesign work (especially when the redesign required restructuring page layouts rather than just restyling existing components).

The root cause was twofold. First, the team's estimation method relied mostly on intuition rather than breaking each task into smaller steps before committing to a duration. A task labelled "implement notifications" might have looked like one item on the sprint board, but in practice it expanded into schema changes, API routes, the bell component, the read state tracking, the dedup logic, and then the integration into every page that needed to show the bell. Each of these was small individually, but the sum was a workload of several days, not the day or two the headline estimate suggested.

Second, the team underestimated discovery work. During Sprint 3 the team identified the NotificationReads composite primary key drift (described in section 4.5.2) that had been silently breaking the mark all read flow. Finding and fixing this kind of issue takes real time, but no one had budgeted hours for "discover and fix bugs in the existing schema". Time spent on these unplanned items came out of the budget for the planned features, which then ran late.

The impact was that some features (notably the Learning Hub and chat assistant polish) were finished close to the demo deadline rather than well in advance. This directly reduced the time available for testing and refinement: the Learning Hub received only informal manual verification rather than structured test cases, and the consolidated reading-assistant page had no automated coverage at all by the time the sprint closed. No feature had to be cut, but the compressed timeline meant that the team carried undiscovered bugs into the demo that a longer testing window would have caught, as the informal UAT session with five student testers subsequently confirmed.

2. Bugs appeared when merging branches together

Several merges during Sprint 3 surfaced bugs that none of the individual branches had on their own. The pattern was that two branches each modified overlapping areas of the codebase or the database schema, and the conflict resolution at merge time produced a state that neither branch had been tested in.

The root cause was that branches lived for too long before merging. A typical Sprint 3 feature branch would be open for multiple days while the owner finished and tested their feature, only

merging once the work was "complete". This makes intuitive sense for individual focus but produces large diffs that are hard to review and easy to merge incorrectly.

A secondary cause was that the database schema sat outside the regular code review process. When two branches both touched the schema (for example, one adding columns to Loans for reminder tracking, another adding the GeneralChatHistory table), there was no automatic way to detect that the migration order would matter. The team noticed these issues only when running the merged code locally and seeing it fail, rather than catching them before the merge.

The impact was lost time at merge points, where one or two team members would spend hours debugging integration problems before the merged branch was ready to push. In a few cases, a merge had to be redone after a hotfix because the first attempt missed an interaction between two changes.

5.3 WHAT CAN BE IMPROVED

Each of the two challenges in section 5.2 points to specific improvements the team can make in Sprint 4 and beyond.

1. Improving estimation accuracy

The team will adopt a stricter rule at sprint planning: any task estimated at more than a single day must be broken into subtasks before the estimate is committed, and each subtask gets its own estimate. The total of the subtask estimates becomes the headline figure. This prevents the "implement notifications" pattern where a single line item on the sprint board hides a workload of several days.

The team will also reserve a fixed percentage of sprint capacity (around 15 percent) for unplanned discovery work and bug fixing. If the unplanned work does not appear, the time gets used to write more tests or pay down technical debt, neither of which is wasted effort. If it does appear (as it did in Sprint 3 with the NotificationReads drift), the time is already budgeted and does not eat into planned feature time.

2. Reducing merge friction

Two changes will reduce the merge problem. First, feature branches should merge to main more frequently, even when the feature is not complete, by hiding incomplete work behind a feature flag rather than letting it sit on a long lived branch. This shortens the merge diff and surfaces conflicts earlier when they are easier to resolve.

Second, the team will add a schema review step to the pull request process. Any pull request that includes a database migration must be reviewed by a second team member before merge, with explicit attention to whether the migration order matters and whether other open branches also touch the schema. This is a process change rather than a tooling change, and costs almost nothing to implement.

5.4 ACTION ITEMS TO BE DONE

The improvements in section 5.3 translate into specific actions for Sprint 4. Each action has a defined owner and a target deadline, listed in the table below.

Action	Owner	Target
Update sprint planning to require subtask breakdown for any task over one day	Kenneth, Kelvin	First planning session of Sprint 4
Reserve 15 percent of sprint capacity for unplanned work in the planning template	Kenneth, Kelvin	First planning session of Sprint 4
Add schema review checkpoint to the pull request template	Kelvin	Week 1 of Sprint 4
Document the feature flag pattern for incomplete features	TBD (assigned at Sprint 4 planning)	Before starting the feature
Enable RLS for the 25 currently unprotected tables	Kelvin, Ivan, Nigel	Sprint 4 backlog (high priority)
Add role gate and rate limit to /api/reading-assistant	Kelvin, Nigel	Sprint 4 backlog
Add foreign key constraints to fix orphan record issues	Kelvin, Ivan, Nigel	Sprint 4 backlog

Where multiple owners are listed, this reflects shared responsibility within Group 12 rather than ambiguity. Kenneth and Kelvin share sprint planning responsibilities, Kelvin handles code review and integration as Tech Lead, the database work is split across Kelvin, Ivan, and Nigel, and AI feature work is shared between Kelvin and Nigel.

5.5 GENERAL OBSERVATIONS FROM SPRINT 3

Beyond the specific things that went well or poorly, three broader observations are worth noting because they affect how Sprint 4 should be structured.

1. The team has reached a sustainable working rhythm

In Sprint 1 the team was still learning each other's working styles, the tools, and the codebase. By Sprint 3 the daily and weekly cadence (Linear updates, GitHub commits, Teams meetings, WhatsApp coordination) has become routine rather than something the team had to consciously enforce. Sprint 4 can therefore focus on improving the work itself rather than improving the workflow.

2. Sprint 3 covered a wide scope

The sprint included the UI redesign across the entire dashboard, AI fine tuning of the Reading Assistant, the Learning Hub video integration, the loan history view, the notification system overhaul, and chat assistant polish. This is a wide scope for a team of four students working in a single sprint. Looking back, fewer features per sprint or more buffer per feature would have eased the estimation issues described in section 5.2.

3. Client engagement was strong

The team showed working features at the demo, the client gave concrete feedback, and the feedback fed into the next sprint backlog rather than being deferred. This part of the process is solid and does not need changing in Sprint 4.

5.6 RISK REGISTRY AND ETHICAL CONSIDERATIONS

No	Risk	Likelihood	Impact	Rating	Responsible	Mitigation & Current Status
1	Supabase service role key exposure could allow full unrestricted database access. RLS is currently disabled on 25 of 27 tables, so the key is the only access guard.	Low	High	Medium	Kelvin	Key stored only in server side environment variables, never in any NEXT_PUBLIC_ variable, never logged. RLS enablement scheduled for Sprint 4 backlog as a structural mitigation.
2	A team member becoming unavailable (illness, exam pressure, family emergency) during critical sprint phases such as demo preparation.	Medium	Medium	Medium	Kenneth, Kelvin	Knowledge sharing through pull request reviews; documentation of every subsystem in section 4 of this report. Each major feature has at least two team members who understand it.
3	Tasks taking longer than estimated at sprint planning, pushing features close to demo deadline and reducing testing time. Already observed in Sprint 3.	Medium	Medium	Medium	Kenneth, Kelvin	Sprint 4 will adopt subtask breakdown rule for any task over one day, plus 15 percent unplanned work buffer. See section 5.3 and 5.4.
4	Feature branches living for multiple days before merging produce large diffs that are hard to review and easy to merge incorrectly, especially when both branches touch the database schema.	Medium	Medium	Medium	Kelvin	Sprint 4 will introduce feature flags for incomplete work and a schema review checkpoint in the pull request template. See section 5.3.
5	The /api/reading-assistant endpoint has no rate limit and	Low	Medium	Low	Kelvin, Nigel	Endpoint is currently only reachable by signed in users. Per user rate limit and explicit role gate

	no role check, so any authenticated user could consume Gemini API quota at an unbudgeted rate.					scheduled for Sprint 4 backlog. See section 4.5.5.
6	The system stores student personal data (loans, profile, chat history with the AI Assistant), which carries privacy and regulatory obligations under university policy.	Low	High	Medium	All team members	Microsoft Azure AD authentication restricts access to verified Swinburne accounts. No personal data is written to logs. Service role key restriction prevents accidental exposure through the browser. RLS rollout in Sprint 4 will add a database level guardrail.
7	The Reading Assistant could suggest nonexistent books or give incorrect loan information, eroding user trust.	Low	Medium	Low	Kelvin, Nigel	The system prompt explicitly forbids the model from inventing dates or titles. The loan_status branch grounds answers in real database data. Book recommendations always pass through search_books which only returns books that exist in the catalogue.
8	The legacy SIP2 service may go out of sync with Supabase if outages happen often, since current SIP2 failures are only logged through console.error with no retry queue.	Medium	Low	Low	Kelvin	Best effort sync design with 5 second timeout already implemented. Supabase is the source of truth, so the system keeps working during SIP2 outages. Acceptable in project context because the legacy ILS is being replaced, not maintained as a parallel system. See section 4.5.3 Decision 4.

6 Lessons Learned

6.1 LESSONS LEARNED

User-Friendly UI:

Designing a user interface that works consistently across both mobile and desktop environments proved to be one of the most challenging aspects of Sprint 3. Selecting a cohesive colour theme that functions equally well in light mode and dark mode required careful consideration of contrast ratios, readability, and visual hierarchy at every breakpoint. The team learned that small decisions such as which shade of cream to use for backgrounds, or how aggressively to round button corners have an outsized effect on whether the interface feels polished or amateurish. Button placement and the logical grouping of interactive elements also required multiple iterations: what felt intuitive to the developer building a feature was not always obvious to a first-time user. The team now understands that UX decisions should be evaluated from the perspective of a student encountering a page for the first time, not from the perspective of someone who built it. Smooth animations through Framer Motion added perceived quality to the experience, but they also added complexity; the team learned to be selective about which state transitions warranted animation and which ones were better left instant so the interface did not feel sluggish.

AI LLM Intelligent Response:

Integrating a large language model into a production system is significantly more complex than calling an API and displaying the response. The team learned that the quality of the AI's output is almost entirely determined by the quality of the system prompt: vague instructions produce vague results, while carefully structured prompts with worked examples, explicit rules, and grounded context produce consistent and trustworthy replies. Keeping the AI grounded in real data, injecting the student's active loans, recent return history, and profile into every request, was essential to making recommendations feel personalised rather than generic. The team also learned that LLM API quotas must be planned for from the start; running out of free quota mid-sprint is a real operational risk. Switching to a lighter model variant (gemini-2.5-flash-lite) and caching per-topic responses proved to be an effective strategy for staying within budget without degrading the user experience. Running the AI service in a 24-hour hosted environment also surfaced latency issues that only appear under real network conditions, not local testing.

Learning Hub:

The original plan for the Learning Hub was to embed LinkedIn Learning as a course platform within the dashboard. This was replaced with a combination of the YouTube Data API v3, the DEV.to community API, and RSS-based technology news feeds after the team determined that LinkedIn Learning required institutional licensing that was outside the project's scope. The replacement approach taught the team an important lesson about platform selection, free, open APIs that aggregate content from active communities are often more practical and more current than gated course platforms. The key challenge was curating the content to remain academically relevant, YouTube's search index is broad, and without careful query construction and category filtering, results drift toward entertainment rather than education. The team learned that a good content aggregation feature requires as much thought about what to exclude as what to include. Looking

ahead, there is an opportunity to broaden the Learning Hub further by integrating platforms that students already use for informal learning, such as Reddit, and by personalising the default feed based on the student's declared faculty and interests rather than showing the same trending content to everyone.

Notification System:

Designing an effective notification system required thinking carefully about what information a user actually needs to see and when. The team learned that notification content must be specific and actionable: a message that simply says "you have a notification" is useless, whereas one that says "your hold on Introduction to Algorithms is ready for pickup until 12 May" is immediately useful. The team also learned that the visual presentation of notifications matters as much as their content. Unread notifications need to stand out clearly without being disruptive, and the history of past notifications must be easy to scan at a glance. Designing the read-state tracking, the deduplication logic to prevent the same reminder from being sent multiple times, and the red dot badge indicator all required more thought than initially anticipated. A key lesson was to think from the user's perspective at every step: what will the student see, what will they understand from it, and what will they do next.

UAT Conduct:

Planning user acceptance testing taught the team that there is a significant difference between internal testing carried out by the developers and external testing carried out by real users. During internal testing, the team designed a structured Google Form survey covering each major feature flow and completed the survey themselves to verify the questions made sense and the system behaved as intended. This exercise revealed gaps in the survey design that would have produced ambiguous data if sent to external testers without revision. For external testing, the team identified three distinct target audiences: students as the primary end users, library staff as the operational users, and library administrators as the management users. Recruiting testers from each group, coordinating a physical testing session at a chosen location on campus, preparing the devices, guiding participants through the system without leading their responses, and managing the time and logistics of multiple concurrent testers all required considerably more preparation than the team had initially estimated. The team learned that UAT logistics should be planned in the sprint backlog alongside the feature work, not treated as an afterthought once the system is ready.

6.2 TO DO AND FUTURE IMPROVEMENT

The most immediate priority for Sprint 4 is to complete the sign-in flow using real Swinburne student Microsoft accounts in the live production environment. Although the authentication infrastructure is in place through Azure AD, the system has so far been validated primarily using development bypass accounts. Connecting the production deployment to the Swinburne Microsoft 365 tenant and verifying that real student accounts are provisioned correctly on first sign-in is a prerequisite for meaningful user acceptance testing.

UI refinement will continue into Sprint 4 with a focus on user experience polish rather than structural redesign. The foundation established in Sprint 3, the design tokens, the responsive layout system, and the Framer Motion animation library, provides a solid base. The remaining work is to tighten interaction details: button feedback timing, micro-animation smoothness on state transitions, and consistency of spacing and alignment across every page. The goal is for every interaction to feel intentional and smooth on both mobile and desktop without requiring the user to think about the interface itself.

The AI Reading Assistant has scope to become significantly more capable in future sprints. The planned improvements include richer system prompt engineering with more worked examples and domain-specific knowledge, the ability to track a student's evolving interests across sessions and surface proactive suggestions, and the potential to pull context from multiple platforms, such as cross-referencing a student's recent Learning Hub activity with their borrowing history, to make recommendations that feel genuinely intelligent rather than purely reactive. The chat assistant could also evolve into a more comprehensive help centre that guides students through all aspects of the library system, not just book discovery.

The Learning Hub has the most obvious room for growth. The current three-tab structure covers YouTube, DEV.to, and RSS news, but students increasingly discover learning content through short-form social media platforms such as TikTok, Instagram, X (formerly Twitter), Reddit, and RedNote. Integrating these sources selectively, filtering for educational creators and topics rather than general trending content, would make the Learning Hub a much richer self-study environment. The longer-term vision is for the Learning Hub to surface content that is personalised to each student's faculty and interests automatically, so that a Computer Science student and a Business student see a meaningfully different default feed without having to configure anything.

Finally, the handover and deployment process needs to be documented thoroughly before the project concludes. This includes writing a clear handover guide for the library IT team covering environment variable management, how to rotate API keys, how to apply database migrations, and how to redeploy the system on Vercel after future updates. Without this documentation, the system risks becoming unmaintainable once the student development team is no longer available. Establishing a clean deployment pipeline and a handover package that a non-developer administrator could follow is a key deliverable for the final sprint.